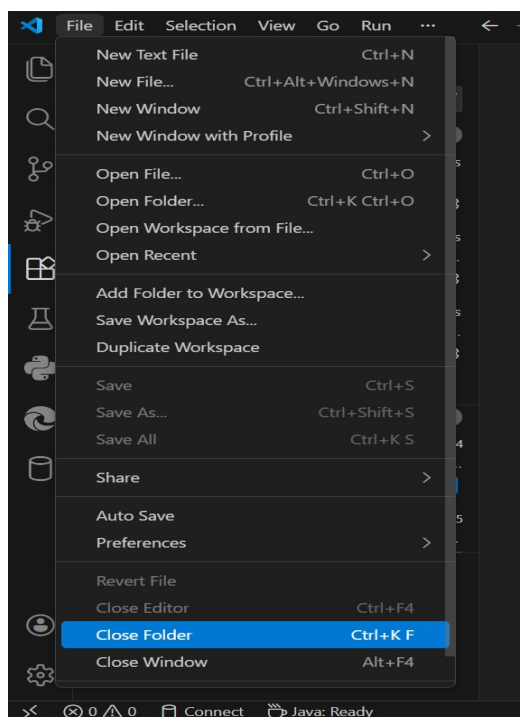


Fase 0: Arquitectura del Proyecto

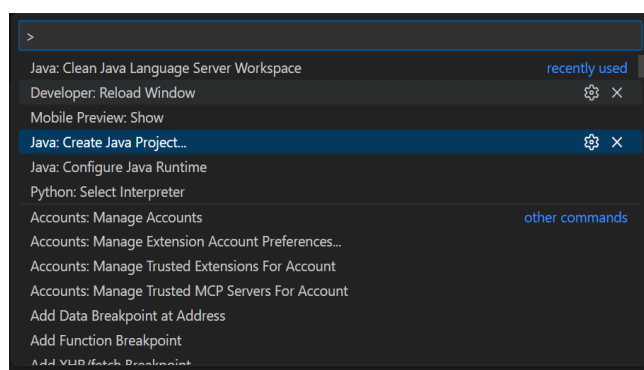
Objetivo: Crear un entorno Java gestionado por Maven para permitir la automatización de pruebas y análisis posterior.

Para garantizar la escalabilidad y la correcta gestión de dependencias desde el inicio, se ha optado por utilizar Apache Maven dentro del entorno de desarrollo integrado (IDE) Visual Studio Code. A continuación, se detalla el procedimiento de inicialización del proyecto:

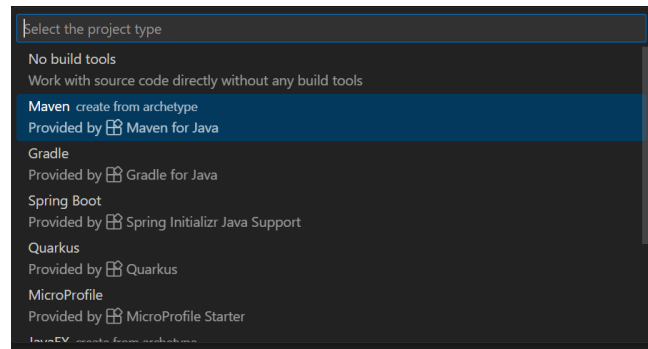
- 1. Preparación del Entorno:** En primer lugar, se asegura un espacio de trabajo limpio en Visual Studio Code cerrando cualquier carpeta o proyecto previo mediante la opción *Close Folder*. Esta acción previene conflictos con configuraciones anteriores ajenas al nuevo proyecto.



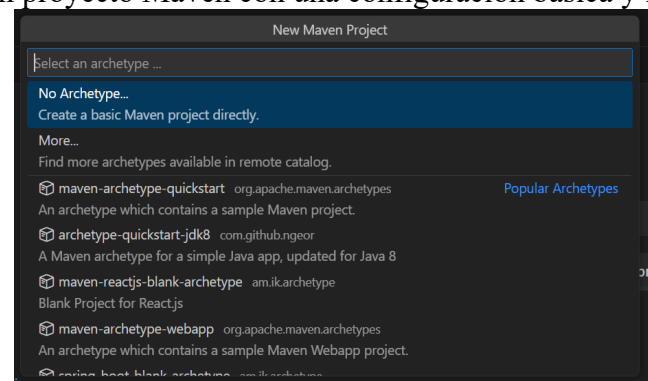
- 2. Inicio de la creación del proyecto:** Se utiliza la paleta de comandos del IDE para invocar la herramienta de generación de proyectos Java (*Java: Create Java Project...*), funcionalidad facilitada por la extensión oficial de Java para VS Code.



3. **Selección de la herramienta de construcción:** Entre las opciones disponibles, se selecciona *Maven*. Esta elección se considera crucial para la arquitectura del proyecto, ya que permitirá gestionar las dependencias y el ciclo de vida del desarrollo de forma estandarizada, facilitando la futura implementación de pruebas unitarias e integración continua.

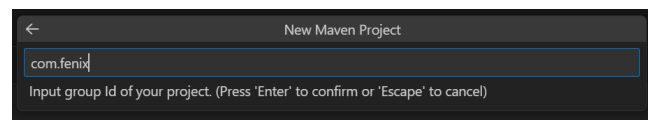


4. **Selección del Arquetipo:** Para mantener un control total sobre la estructura y evitar la generación automática de código innecesario, se escoge la opción **"No Archetype"**. Esto permite iniciar un proyecto Maven con una configuración básica y limpia.

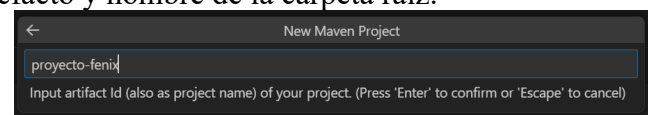


5. **Configuración de Identificadores (GAV):** Se procede a definir las coordenadas del proyecto Maven:

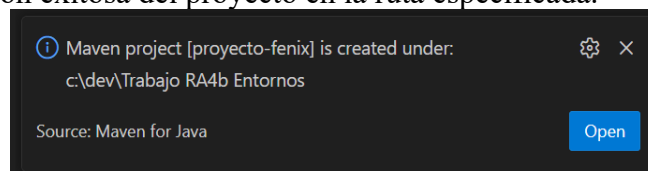
- **Group Id:** Se introduce *"com.fenix"*, siguiendo las convenciones estándar de nomenclatura de paquetes en Java.



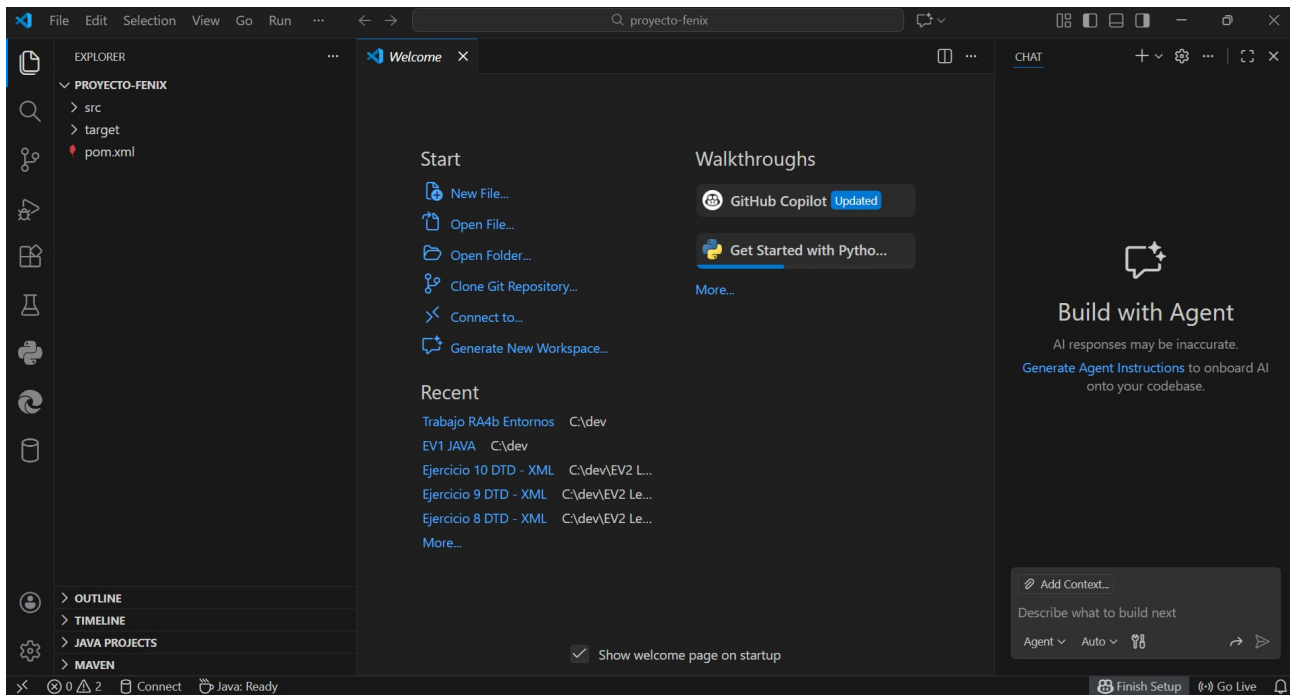
- **Artifact Id:** Se asigna el nombre *"proyecto-fenix"*. Este actuará como identificador único del artefacto y nombre de la carpeta raíz.



6. **Generación del Proyecto:** Se selecciona el directorio de trabajo local donde se alojará el código fuente. Tras la confirmación, Maven genera la estructura de directorios y el IDE notifica la creación exitosa del proyecto en la ruta especificada.



- 7. Estructura Final:** Finalmente, se abre el proyecto generado en el editor. Se puede observar en el explorador de archivos la arquitectura de directorios estándar de Maven ("*src/main/java*", "*pom.xml*", etc.), quedando la estructura lista para recibir el código heredado.

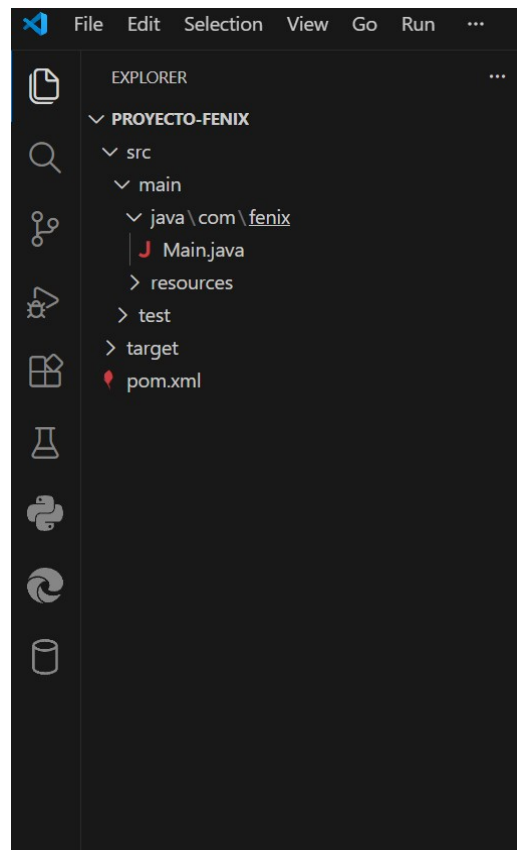


Fase 1: Creación del "Código Heredado"

Objetivo: Integrar la clase "*ProcesadorUsuarios.java*" en la estructura correcta del proyecto Maven, preparándola para su posterior refactorización.

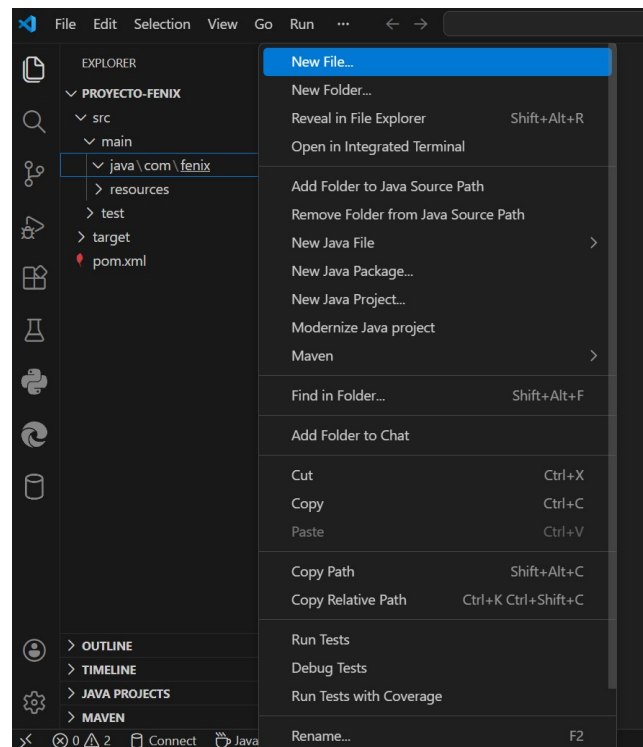
Una vez generada la arquitectura base del proyecto, es necesario limpiar los archivos generados por defecto e introducir el código fuente objetivo de la práctica.

1. Limpieza del Proyecto: Al inspeccionar la estructura inicial en la carpeta "*src/main/java/com/fenix*", se observa la existencia de una clase "*Main.java*" creada automáticamente por el arquetipo o la herramienta del IDE.

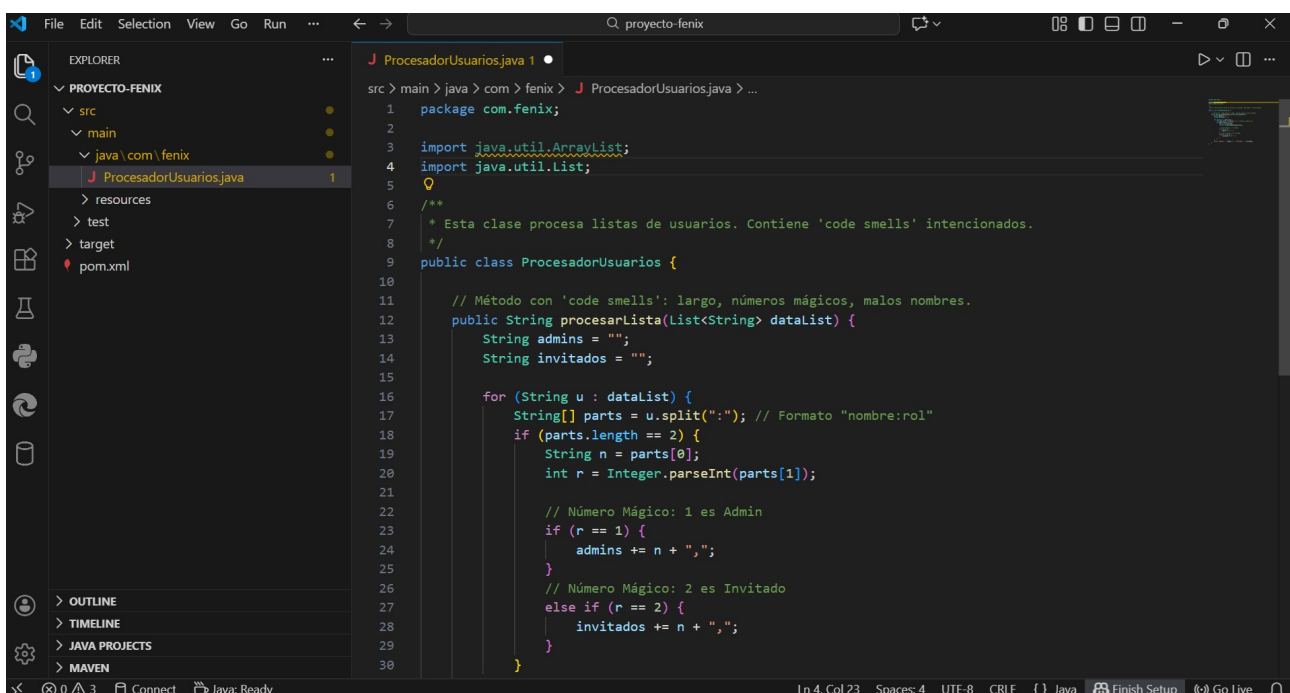


Dado que esta clase no forma parte de los requisitos del sistema heredado, se procede a su eliminación directa para asegurar que el proyecto contenga únicamente los archivos necesarios.

- 2. Creación del Archivo de Clase:** Sobre el paquete *com.fenix* (ubicado en *src/main/java*), se despliega el menú contextual para crear un nuevo archivo. Esta ubicación es crítica para asegurar que Maven compile y gestione la clase correctamente dentro del *classpath* del proyecto.



- 3. Integración del Código Heredado:** Se nombra el archivo como "*ProcesadorUsuarios.java*". A continuación, se copia y pega el código fuente proporcionado en el enunciado de la práctica. En este punto, el editor resalta la sintaxis, permitiendo identificar visualmente la estructura del código, incluyendo los *imports* (*java.util.ArrayList*, *java.util.List*) y la lógica del método *procesarLista*.



- 4. Verificación de Paquete:** Finalmente, se verifica que la declaración del paquete en la primera línea del código (*package com.fenix;*) coincida exactamente con la estructura de carpetas creada. Esta concordancia es obligatoria en Java para evitar errores de compilación.

```
J ProcesadorUsuarios.java 1 X
src > main > java > com > fenix > J ProcesadorUsuarios.java > ProcesadorUsuarios > procesarLista(List<String>)
1 package com.fenix;
```

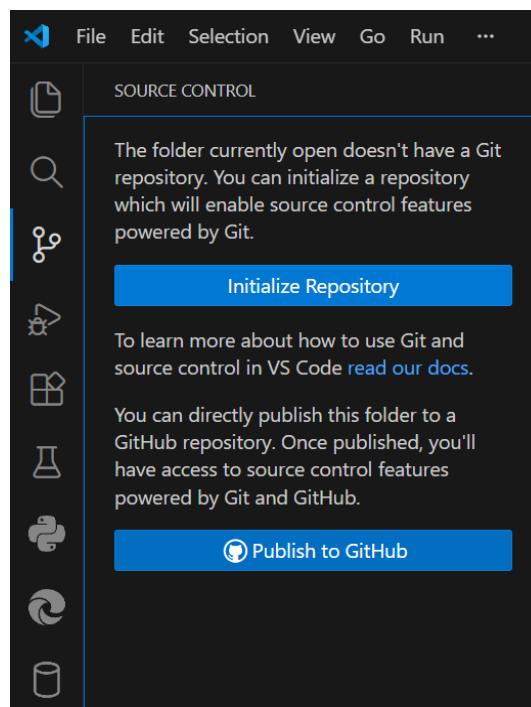
Parte 1: Configuración del proyecto en Git

Tarea 1: Control de versiones local y remoto

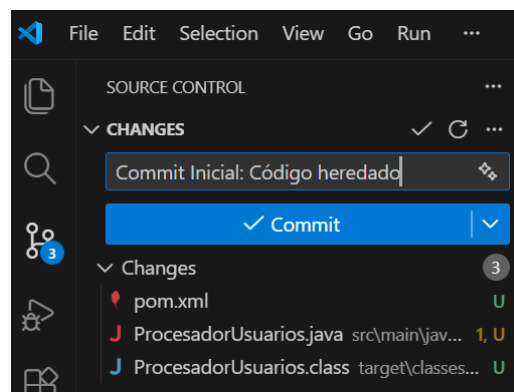
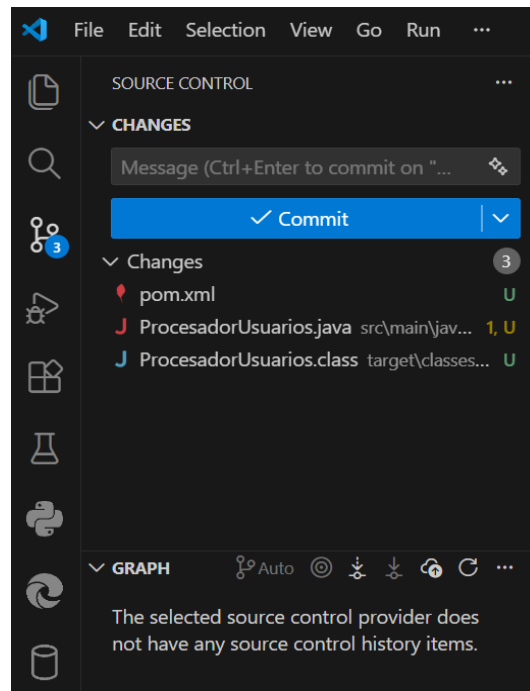
Objetivo: Establecer un sistema de control de versiones robusto que permita gestionar el historial de cambios del proyecto y su respaldo en la nube.

Para cumplir con este requisito fundamental en el desarrollo profesional de software, se configura un repositorio Git local y se vincula con un repositorio remoto alojado en GitHub. A continuación, se detalla el procedimiento técnico seguido:

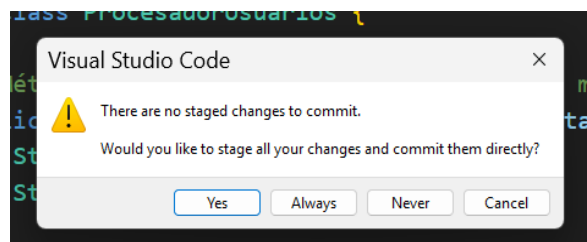
- 1. Inicialización del Repositorio Local:** Desde la vista de control de código fuente (*Source Control*) integrada en Visual Studio Code, se observa que el directorio de trabajo actual no está bajo control de versiones. Se procede a inicializar el repositorio local mediante la acción *Initialize Repository*. Esto crea la estructura interna *.git* necesaria para rastrear los cambios en los archivos.



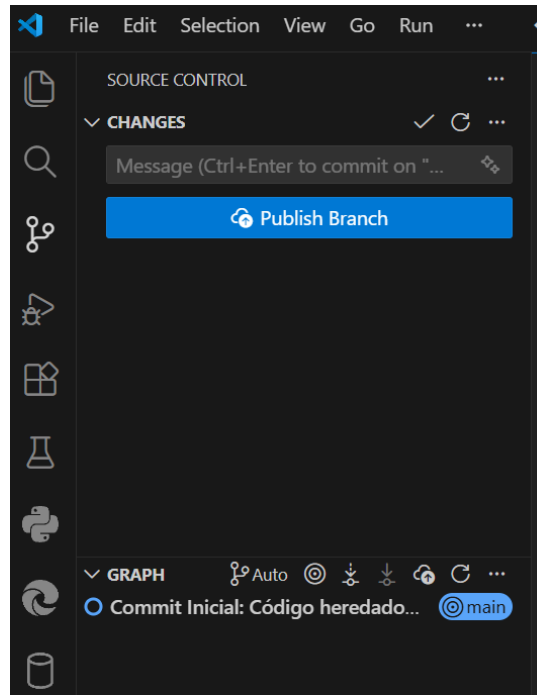
- 2. Preparación de los Cambios (Staging):** Una vez inicializado el repositorio, el sistema detecta los archivos existentes como "nuevos" (marcados con una "U" de *Untracked*). Se introduce el mensaje descriptivo "*Commit Inicial: Código heredado*" en el campo de mensaje. Al pulsar el botón *Commit*, el IDE alerta de que no hay cambios en el área de preparación (*staged changes*).



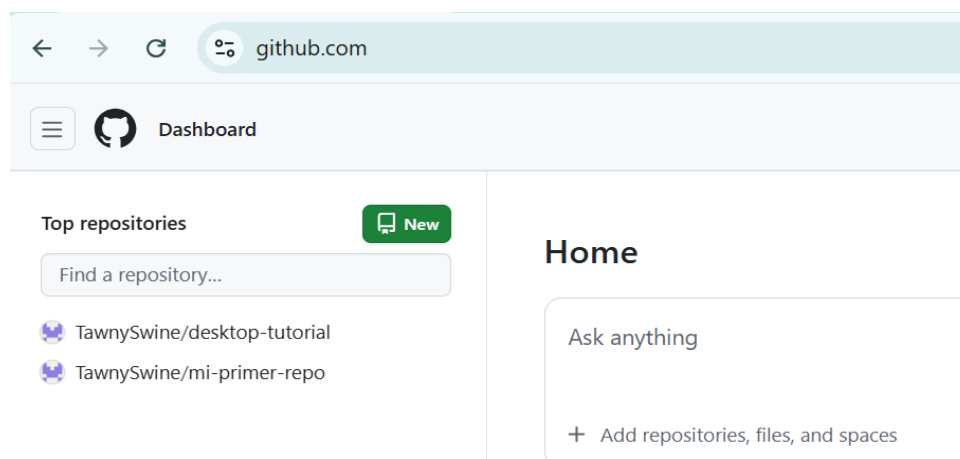
- 3. Confirmación de Cambios (Commit):** El entorno solicita confirmación para añadir automáticamente todos los cambios detectados (*stage all changes*) y realizar el *commit* directamente. Se selecciona la opción *Yes* para confirmar la operación, guardando así el estado inicial del proyecto en el historial local.



4. **Verificación del Estado Local:** Tras el *commit*, la interfaz de control de versiones muestra la opción *Publish Branch*, indicando que los cambios están confirmados localmente pero aún no existen en ningún repositorio remoto. En la sección *GRAPH*, se verifica que el commit "*Commit Inicial: Código heredado*" se ha registrado correctamente en la rama *main*.



5. **Creación del Repositorio Remoto:** Para alojar el código en la nube, se accede a la plataforma GitHub. Desde el panel de control (*Dashboard*), se inicia la creación de un nuevo repositorio.



- 6. Configuración del Repositorio en GitHub:** Se asigna el nombre *proyecto-fenix* al repositorio remoto, asegurando que coincida con el identificador del proyecto local para mantener la consistencia. Se configura como *Public* y se deja vacío (sin añadir *README*, *.gitignore* ni licencia en este paso) para evitar conflictos de historial al subir el código local existente.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 **General**

Owner * TawnySwine / Repository name *
✔ proyecto-fenix is available.

Great repository names are short and memorable. How about [cuddly-dollop](#)?

Description
0 / 350 characters

2 **Configuration**

Choose visibility * Public
Choose who can see and commit to this repository

Add README Off
READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

Add license No license
Licenses explain how others can use your code. [About licenses](#)

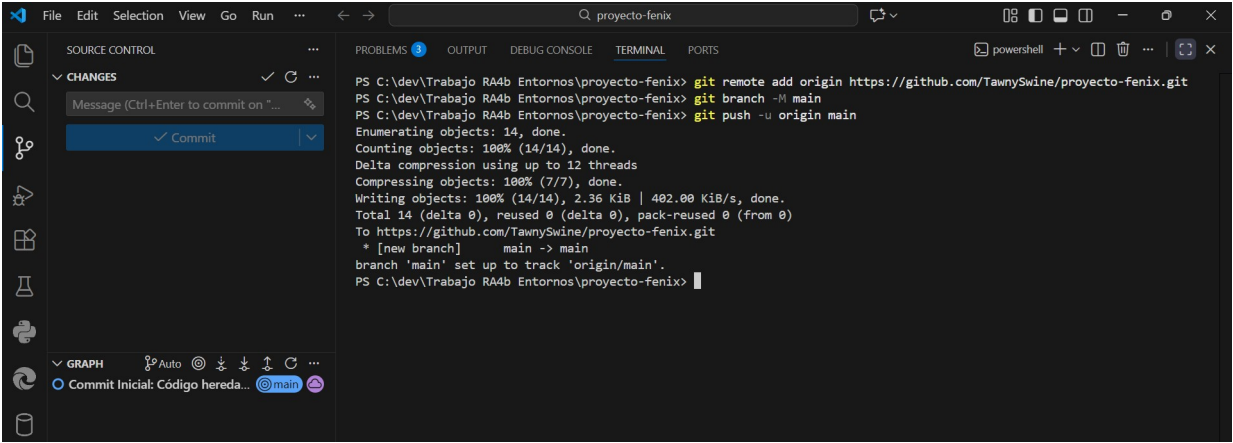
Create repository

- 7. Obtención de la URL del Repositorio:** Una vez creado el repositorio vacío, GitHub proporciona la URL de conexión (<https://github.com/TawnySwine/proyecto-fenix.git>). Esta dirección es necesaria para vincular el entorno local con el remoto.

The screenshot shows a web browser displaying the GitHub repository page for 'TawnySwine/proyecto-fenix'. The URL bar shows 'github.com/TawnySwine/proyecto-fenix'. A search bar overlay is visible. The repository page shows the repository name 'proyecto-fenix' as 'Public'. Below the repository name, there are sections for 'Set up GitHub Copilot', 'Add collaborators to this repository', and 'Quick setup — if you've done this kind of thing before'. The 'Quick setup' section provides instructions for cloning the repository using HTTPS or SSH, and for creating a new repository on the command line. The command line instructions are:

```
echo "# proyecto-fenix" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/TawnySwine/proyecto-fenix.git
git push -u origin main
```

- 8. Vinculación y Subida (Push):** De vuelta en el IDE, se utiliza la terminal integrada para ejecutar los comandos de vinculación y sincronización. Se añade el origen remoto, se asegura el nombre de la rama principal a *main* y se realiza el envío de los cambios (*push*). La salida del comando confirma que la rama local *main* se ha configurado para rastrear la rama remota *origin/main*, completando la sincronización exitosamente.



The screenshot shows the Visual Studio Code interface with the 'TERMINAL' pane active. The terminal displays the following commands and output:

```
PS C:\dev\Trabajo RA4b Entornos\proyecto-fenix> git remote add origin https://github.com/TawnySwine/proyecto-fenix.git
PS C:\dev\Trabajo RA4b Entornos\proyecto-fenix> git branch -M main
PS C:\dev\Trabajo RA4b Entornos\proyecto-fenix> git push -u origin main
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 12 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (14/14), 2.36 KiB | 482.00 KiB/s, done.
Total 14 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/TawnySwine/proyecto-fenix.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\dev\Trabajo RA4b Entornos\proyecto-fenix>
```

The left sidebar shows the 'SOURCE CONTROL' view with a 'Commit' button and a 'GRAPH' view showing the commit history.

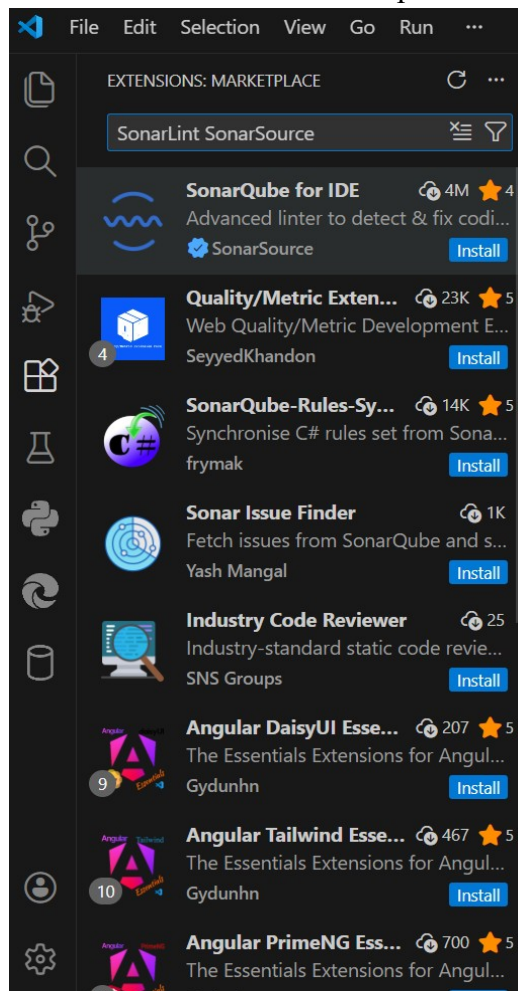
Parte 2: Análisis y pruebas

Tarea 2: Análisis estático de código

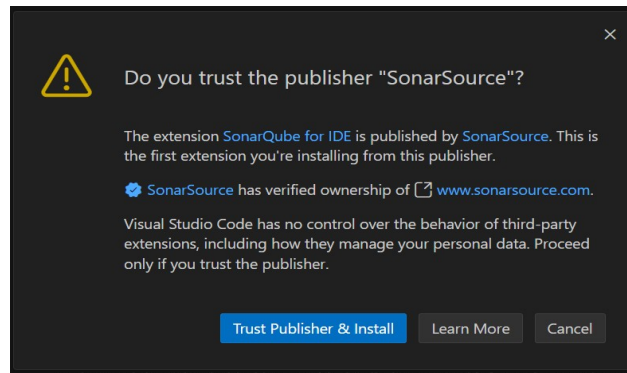
Objetivo: Utilizar herramientas de análisis estático para identificar deuda técnica, *code smells* y posibles errores en el código heredado antes de proceder a su refactorización.

Para cumplir con este objetivo, se procede a la instalación y ejecución de SonarQube for IDE (anteriormente conocido como SonarLint) dentro de Visual Studio Code. A continuación se describe el proceso:

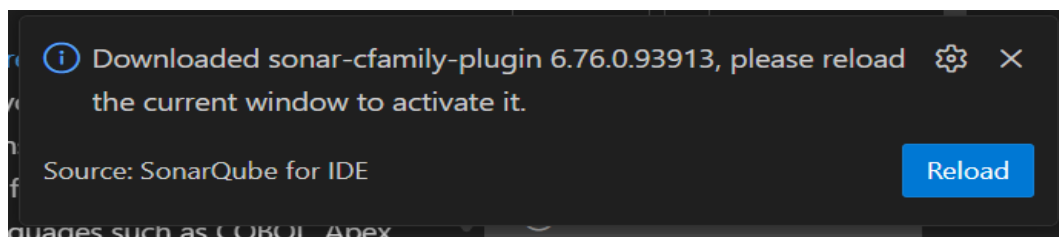
- 1. Instalación del Analizador:** Se accede al *Marketplace* de extensiones del IDE y se busca la extensión oficial "SonarQube for IDE" desarrollada por SonarSource. Se procede a su instalación para habilitar el motor de análisis en tiempo real.



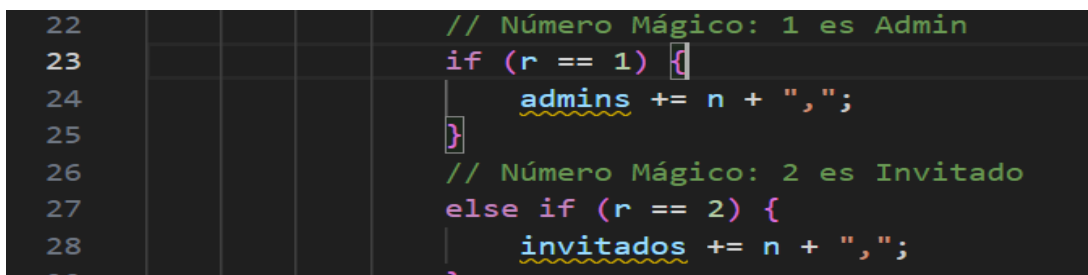
2. **Confirmación de Seguridad:** Durante el proceso, el entorno solicita confirmación de confianza en el editor de la extensión (*SonarSource*). Se confirma la operación para permitir que la herramienta interactúe con el código del proyecto.



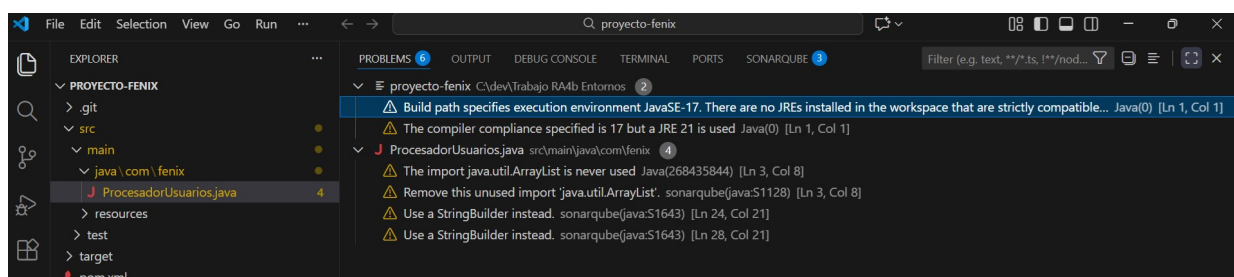
3. **Activación de la Extensión:** Una vez descargados los complementos necesarios (como el *sonar-cfamily-plugin*), el IDE requiere una recarga de la ventana (*Reload*) para activar la extensión correctamente.



4. **Ejecución del Análisis:** Al abrir el archivo *ProcesadorUsuarios.java*, el analizador escanea automáticamente el código. En el editor, se pueden observar subrayados ondulados en amarillo que indican las líneas donde se han detectado violaciones de las reglas de calidad definidas.

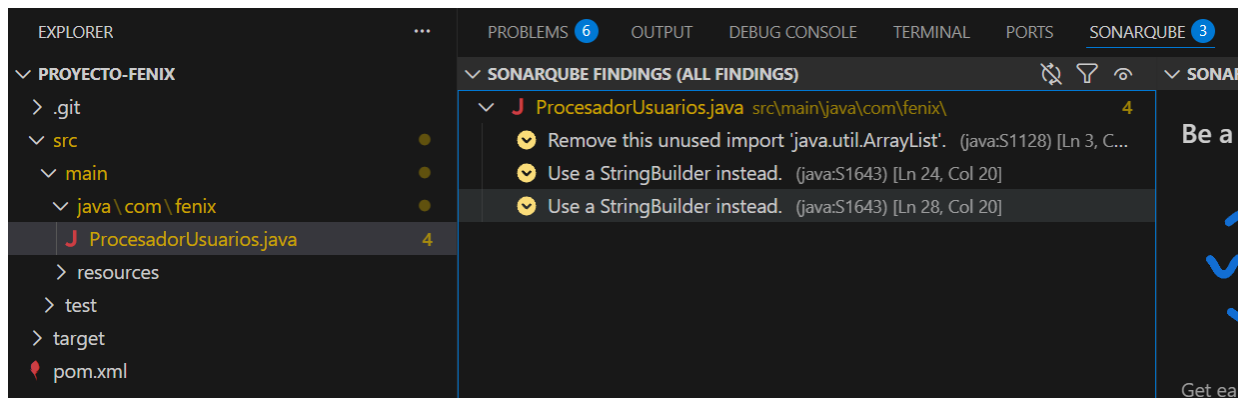


5. **Revisión de Problemas (Panel Problems):** En la pestaña *PROBLEMS* del terminal integrado, se listan todas las advertencias detectadas. Se observa que el analizador ha identificado problemas de mantenibilidad y rendimiento, como importaciones no utilizadas y concatenaciones ineficientes.



6. Detalle de las Reglas (Panel SonarQube): Utilizando el panel dedicado de *SonarQube Findings*, se obtiene un desglose más detallado de las reglas infringidas. Tras el análisis, se destacan los siguientes problemas principales:

- **Regla S1128 (Unused Imports):** Se detectó la importación innecesaria de *java.util.ArrayList*.
- **Regla S1643 (Inefficient String Concatenation):** Se alertó sobre el uso de concatenación de cadenas (+) dentro de un bucle, recomendando el uso de *StringBuilder* para mejorar el rendimiento.
- **Nota sobre Números Mágicos:** Aunque la configuración por defecto priorizó las reglas anteriores, se observa en el código el uso de literales numéricos sin contexto (1 y 2). Se procederá a su refactorización manual en fases posteriores para cumplir con los estándares de código limpio y eliminar estos "números mágicos".

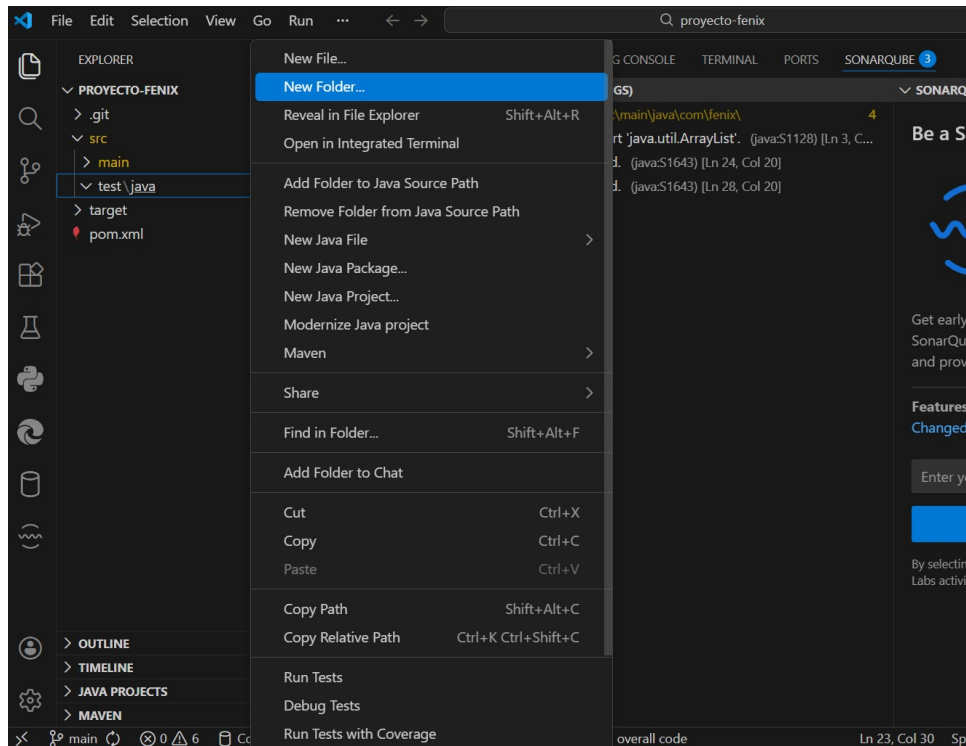


Tarea 3: Pruebas de refactorización

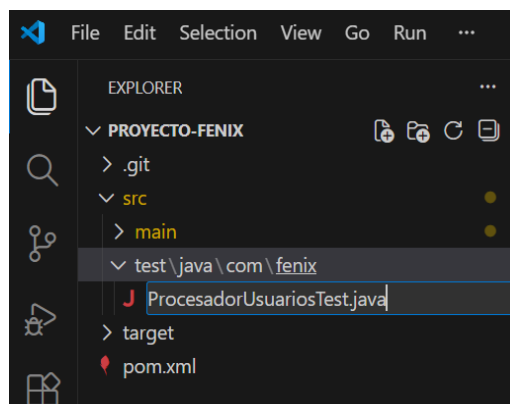
Objetivo: Establecer una "red de seguridad" mediante pruebas unitarias que garanticen que el comportamiento del sistema no se vea alterado durante el proceso de mejora del código.

Antes de realizar cualquier modificación sobre la lógica existente, se debe asegurar que el código actual funciona según lo esperado. Para ello, se implementa una prueba con el framework JUnit.

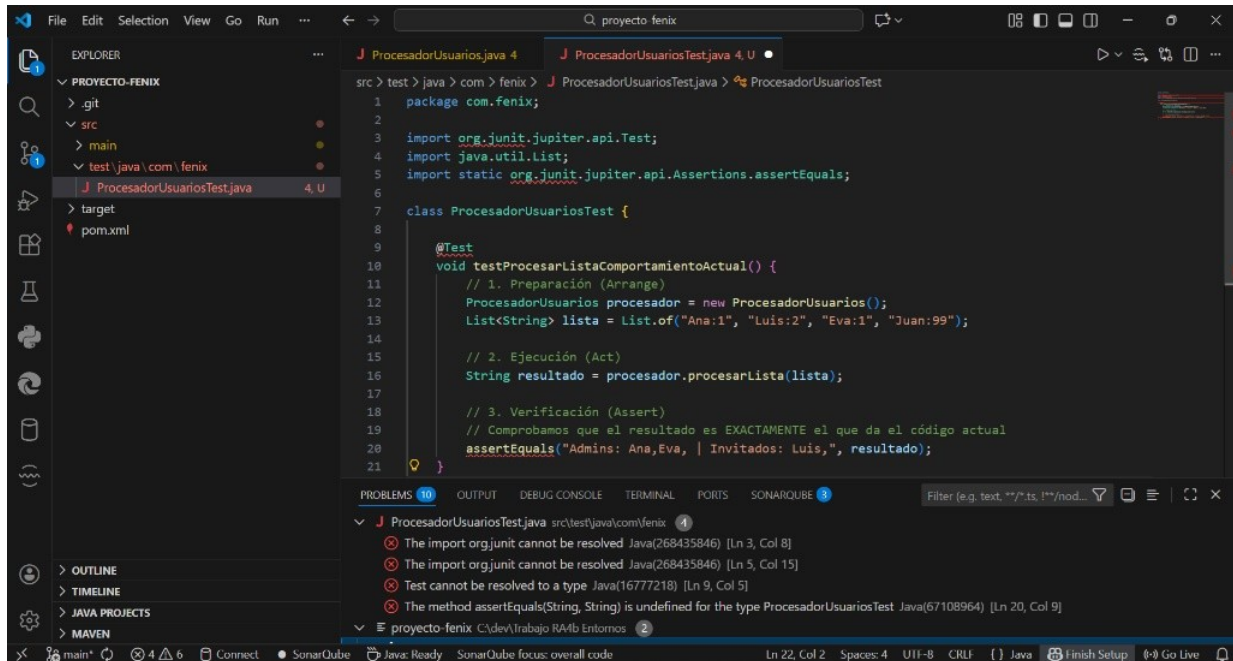
- 1. Preparación de la Estructura de Pruebas:** Siguiendo el estándar de Maven, se replica la estructura de paquetes dentro del directorio de pruebas. Se crea la ruta de carpetas `src/test/java/com/fenix` y se genera el archivo de la clase de prueba.



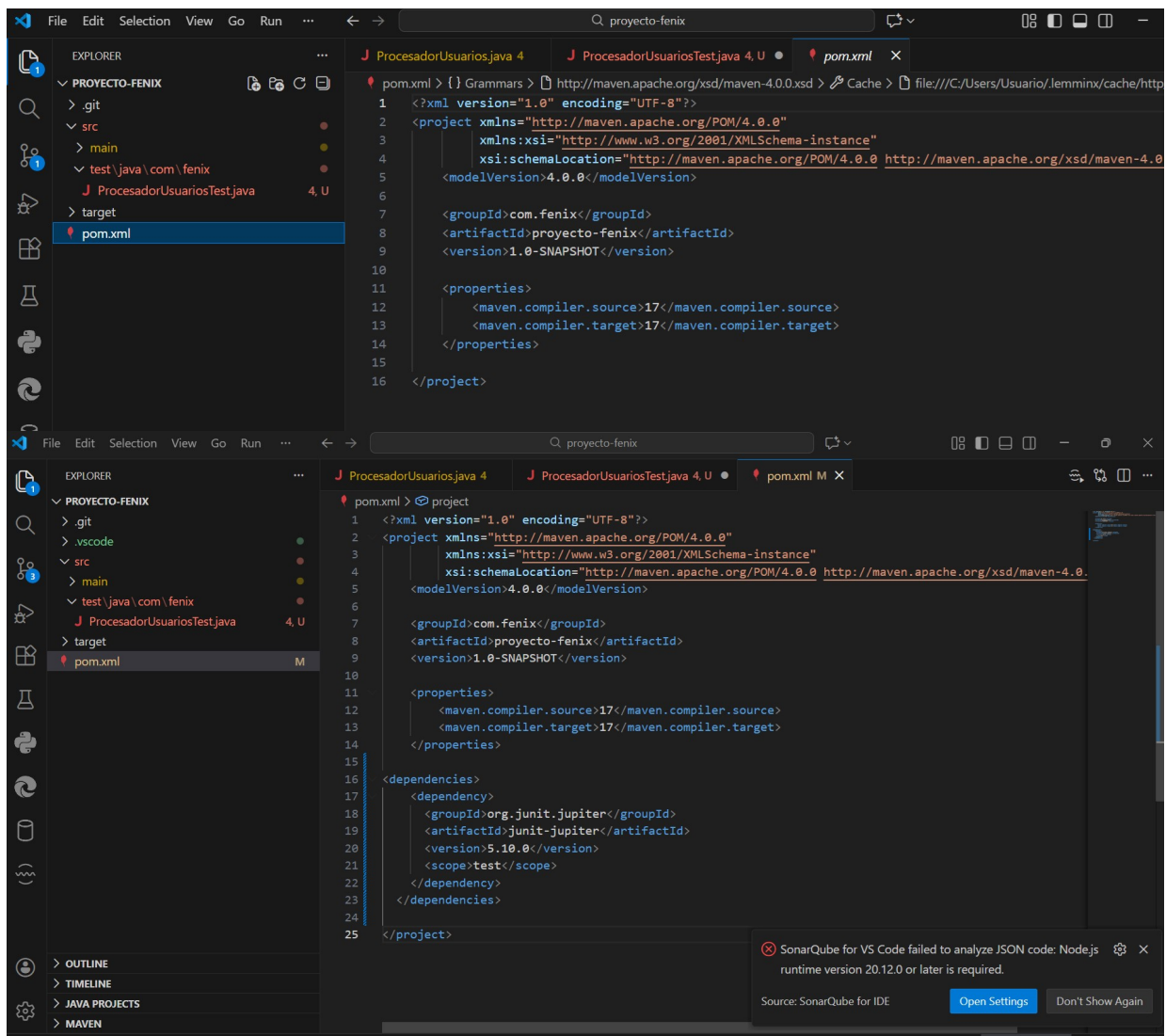
Se nombra al archivo como *ProcesadorUsuariosTest.java*, cumpliendo con la convención de nomenclatura para clases de test en Java.



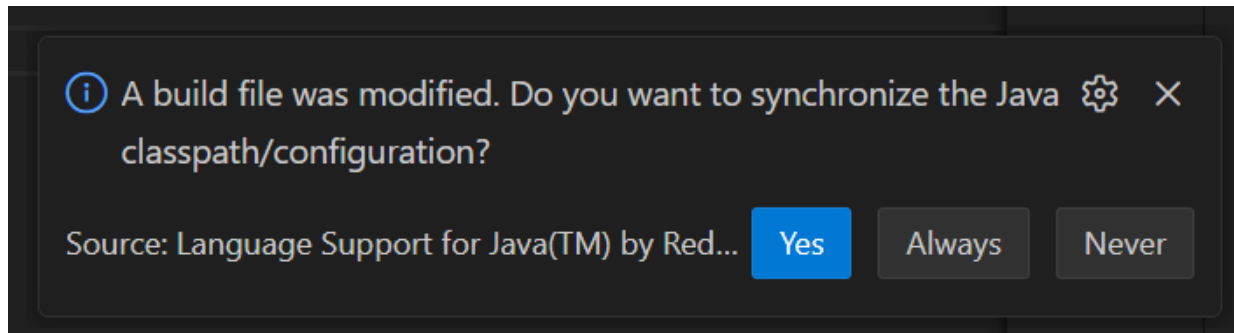
2. **Gestión de Dependencias (pom.xml):** Al intentar redactar el código de prueba utilizando las anotaciones estándar como `@Test` o métodos como `assertEquals`, el entorno de desarrollo marca errores de compilación. Esto se debe a que la librería JUnit no está declarada en el proyecto.



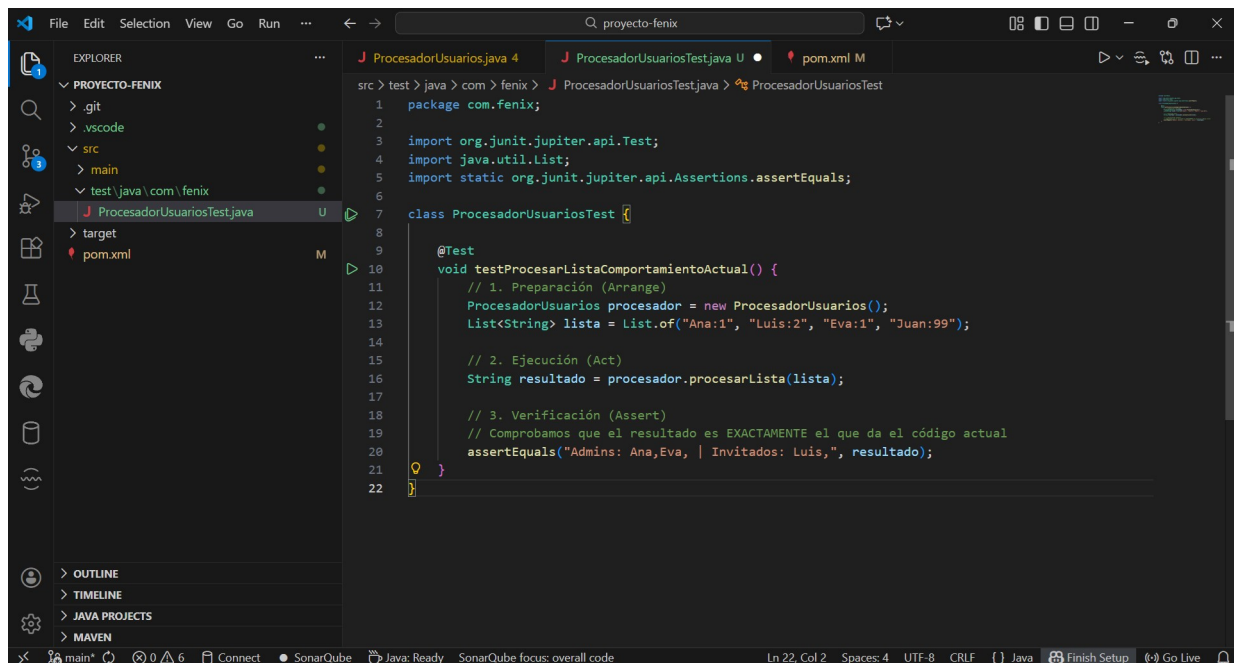
Para solucionar esto, se edita el archivo `pom.xml`. Se añade la dependencia `junit-jupiter` (perteneciente a JUnit 5) dentro de la sección `<dependencies>`, especificando el alcance (`scope`) como `test`.



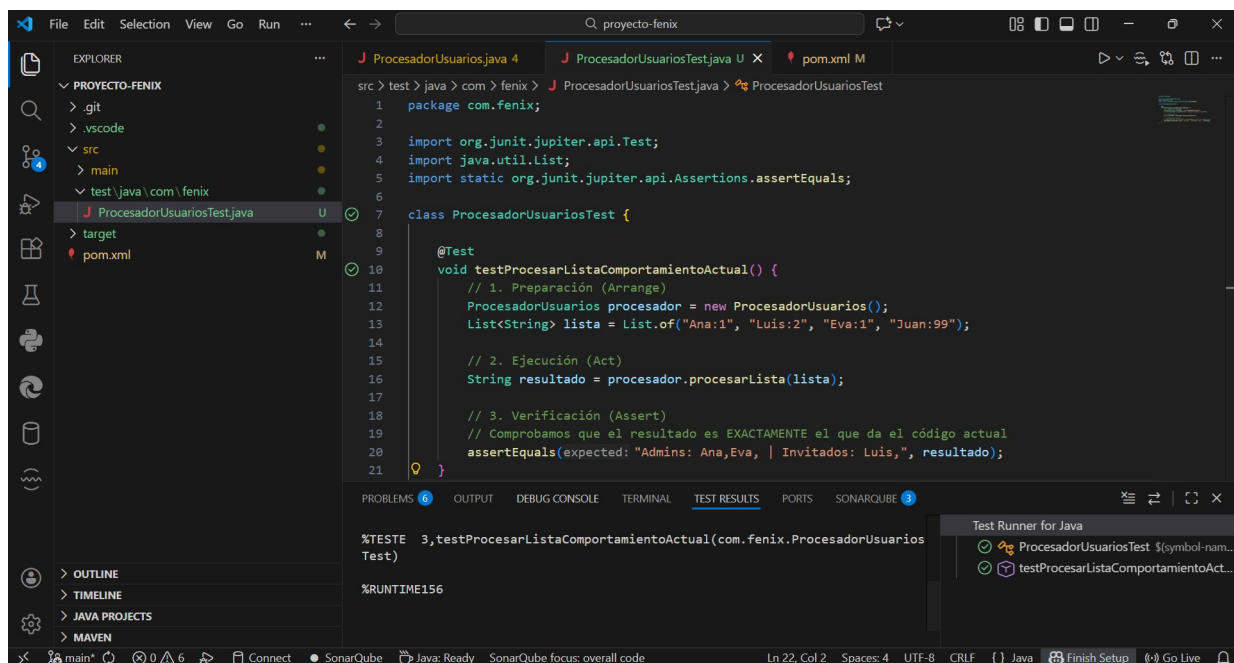
Tras modificar el archivo de configuración, el IDE solicita sincronizar la configuración del classpath de Java para descargar e indexar las nuevas librerías. Se confirma esta acción para aplicar los cambios.



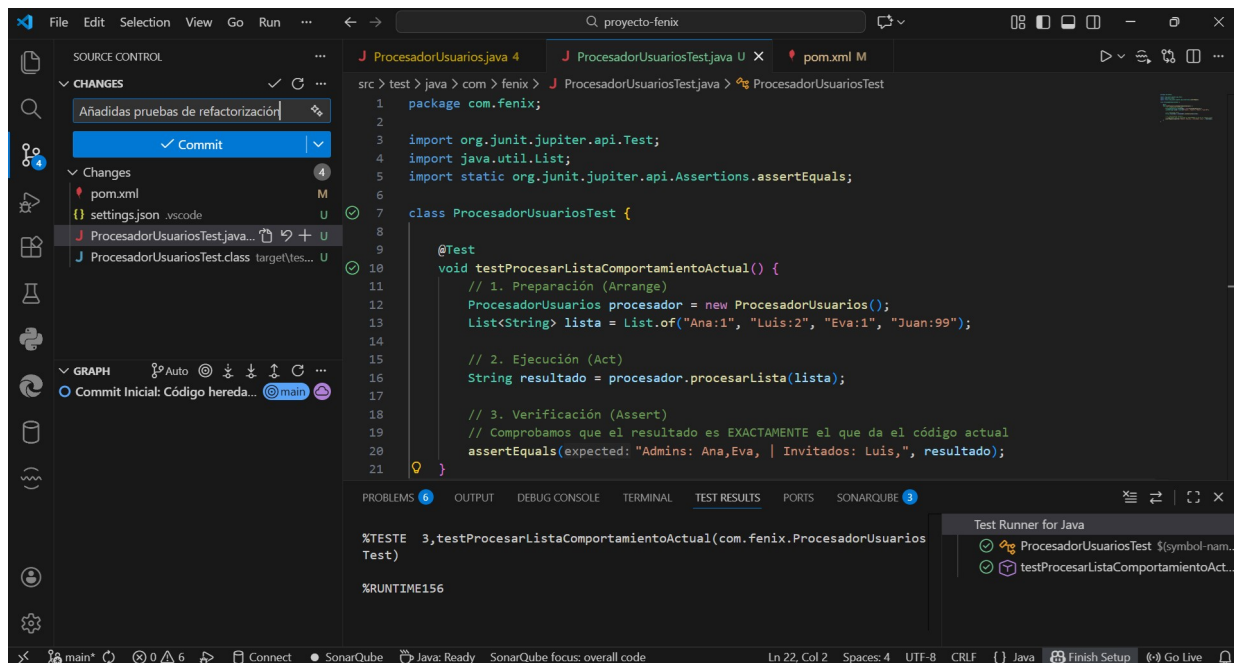
- 3. Implementación de la Prueba Unitaria:** Una vez resueltas las dependencias, los errores de compilación desaparecen. Se implementa el método *testProcesarListaComportamientoActual*, siguiendo el patrón *Arrange-Act-Assert*. Se prepara una lista de entrada con datos conocidos ("Ana:1", "Luis:2"), se ejecuta el método *procesarLista* y se verifica que la cadena resultante coincida exactamente con la salida que genera el código heredado actual.



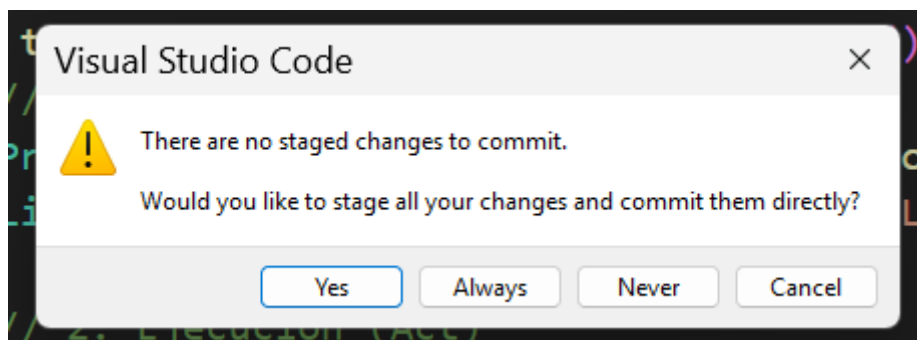
4. **Ejecución y Verificación:** Se ejecuta la prueba desde el propio IDE. El resultado es exitoso (indicador verde), lo que confirma que la prueba captura fielmente el comportamiento actual del sistema, aunque este contenga "code smells". Esta prueba servirá de invariante durante la refactorización.



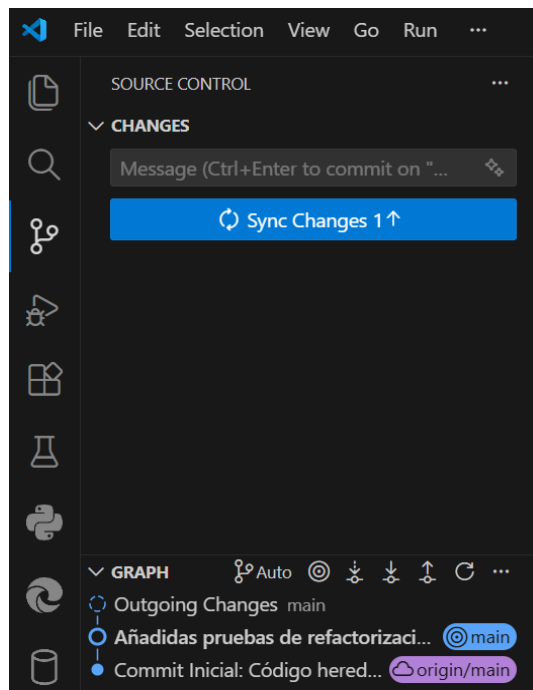
5. **Control de Versiones (Commit):** Con la red de seguridad establecida, se procede a guardar el estado del proyecto. En la vista de control de código fuente, se añaden los archivos modificados (*pom.xml* y *ProcesadorUsuariosTest.java*) al área de preparación. Se introduce el mensaje "Añadidas pruebas de refactorización".



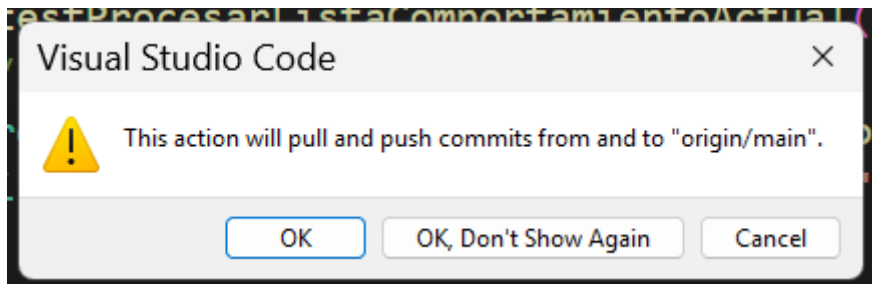
El IDE solicita confirmación para realizar el *stage* de los cambios y el *commit* en un solo paso, acción que se acepta para registrar la nueva funcionalidad en el repositorio local.



- 6. Sincronización con el Repositorio Remoto:** Tras confirmar el *commit*, el sistema de control de versiones indica que la rama local está adelantada respecto a la remota. En el botón de acción principal aparece la opción *Sync Changes*, señalando que existe un cambio pendiente de envío (*outgoing change*). Asimismo, en el gráfico del historial se aprecia que el puntero *main* ha avanzado respecto a *origin/main*.



7. **Envío de Cambios (Push):** Al pulsar el botón de sincronización, el entorno solicita confirmación para realizar las operaciones de *pull* y *push* hacia la rama *origin/main*. Se acepta la operación para asegurar que las pruebas de seguridad queden resguardadas en el repositorio remoto GitHub.



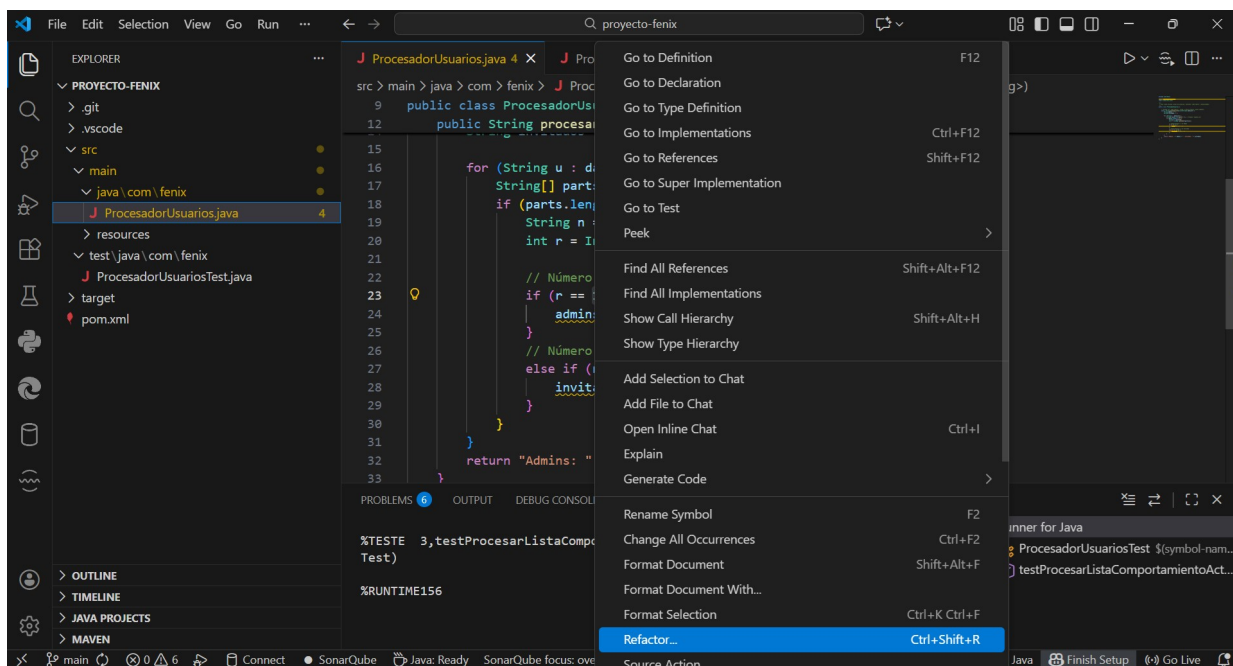
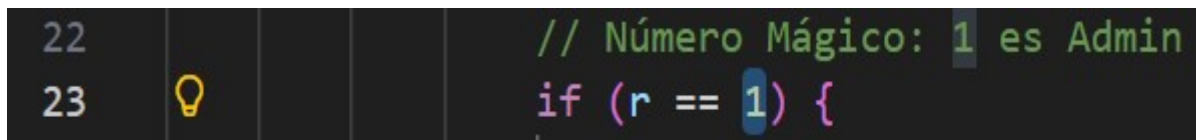
Parte 3: Refactorización y Limpieza

Tarea 4: Aplicar patrones de refactorización

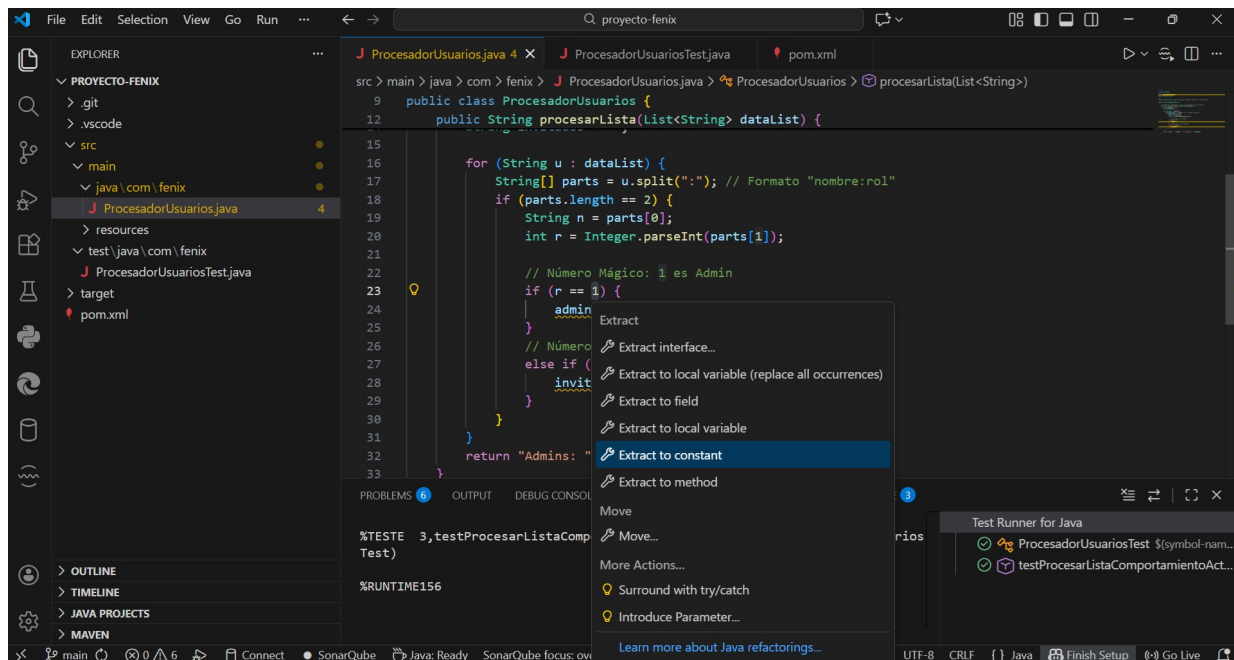
Objetivo: Mejorar la legibilidad y mantenibilidad del código ("Clean Code") sin alterar su funcionalidad externa, apoyándose en la red de seguridad que proporcionan las pruebas unitarias.

Con las pruebas en verde, se procede a eliminar la "deuda técnica" identificada previamente.

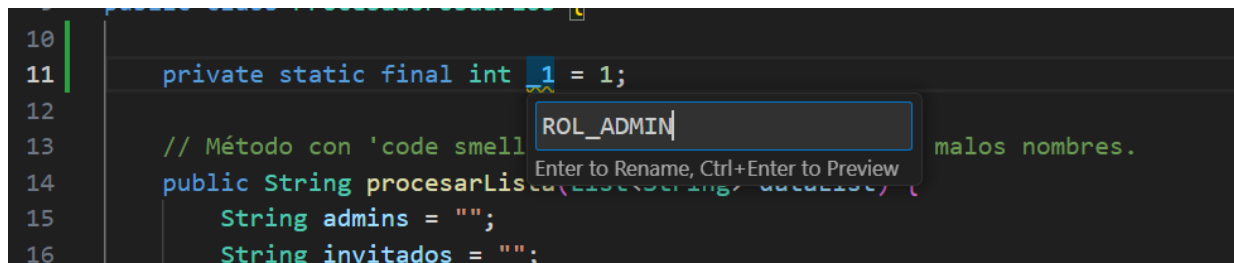
1. **Refactorización de "Números Mágicos":** El análisis estático alertó sobre el uso de literales numéricos (1 y 2) dentro de la lógica condicional, lo cual dificulta la comprensión de qué representan esos valores. Para corregirlo, se ubica el cursor sobre el valor 1 y se utiliza la herramienta de refactorización automatizada del IDE. Se accede mediante el menú contextual *Refactor...*



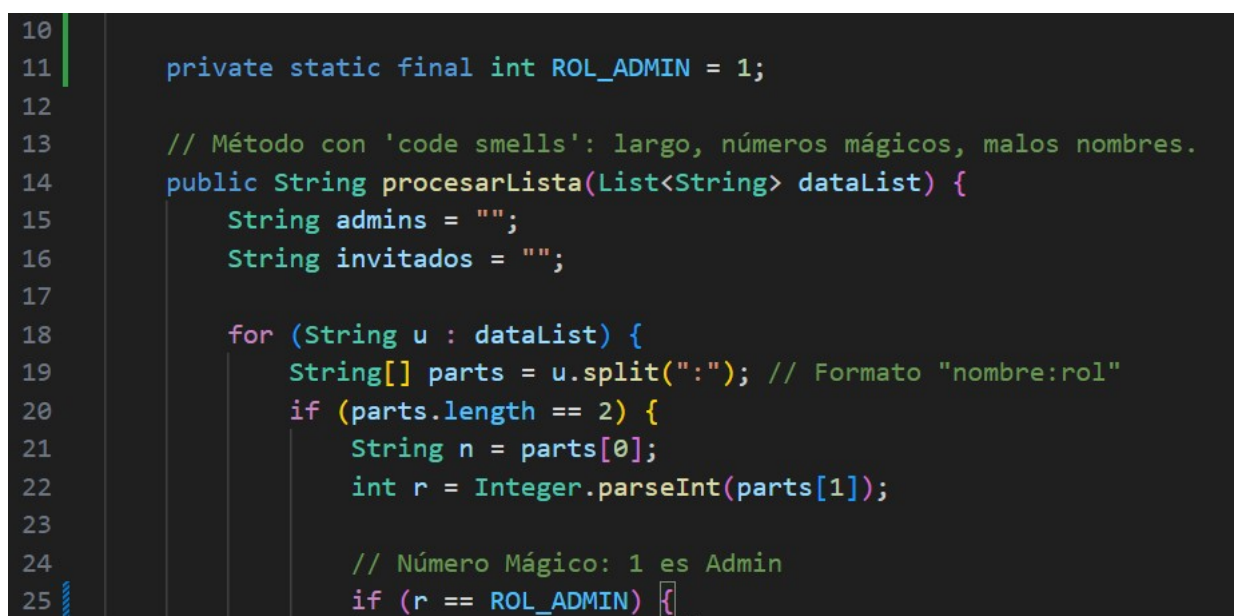
Dentro de las opciones de refactorización, se selecciona *Extract to constant*. Esta herramienta permite reemplazar todas las ocurrencias del valor por una constante con nombre significativo.



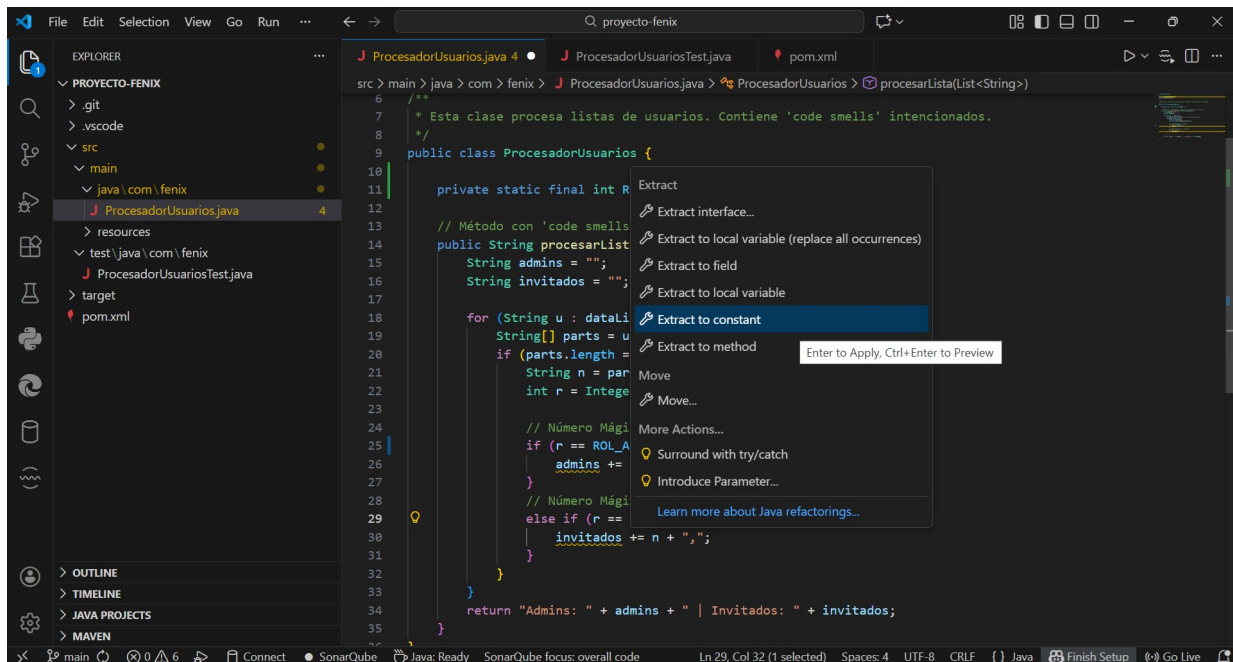
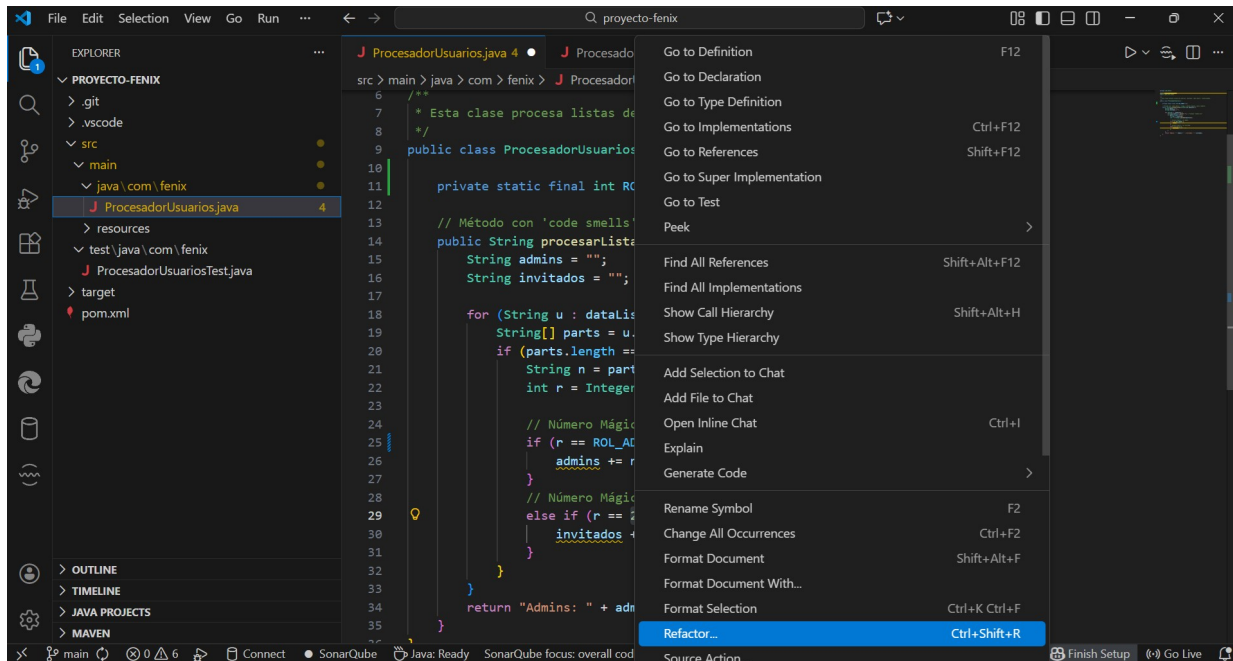
Se asigna el nombre *ROL_ADMIN* a la constante, siguiendo la convención de nomenclatura *UPPER_SNAKE_CASE* para constantes en Java. El IDE se encarga de declarar la variable *private static final int* automáticamente.



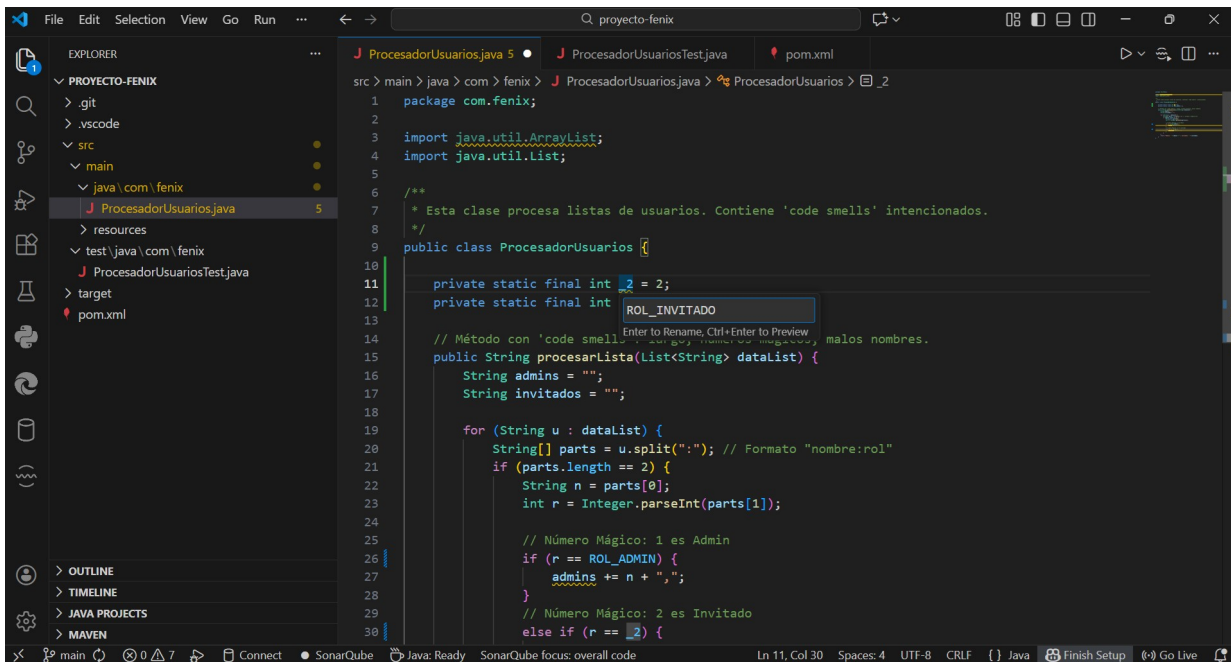
Tras confirmar, se observa que el código ha sustituido el número 1 por la constante *ROL_ADMIN*, mejorando instantáneamente la semántica de la comparación.



Se repite el mismo procedimiento para el valor 2, que representa el rol de invitado. Se selecciona el literal y se invoca nuevamente la herramienta de extracción de constantes.

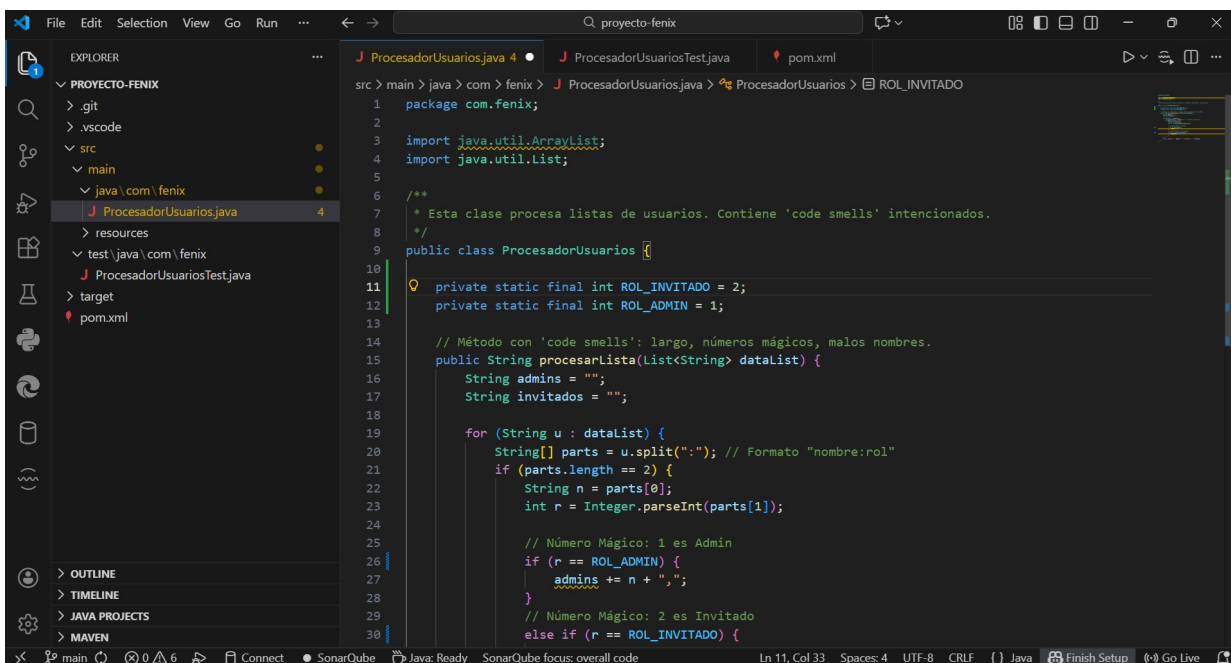


Se nombra esta segunda constante como `ROL_INVITADO`.



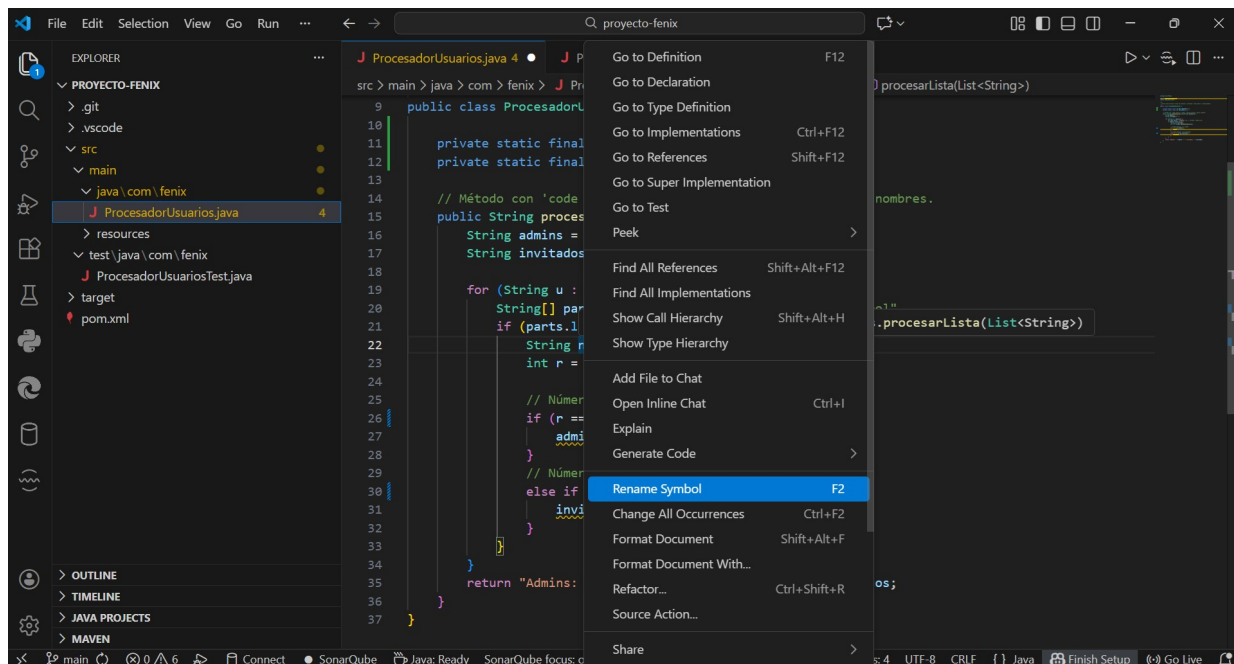
```
src > main > java > com > fenix > J ProcesadorUsuarios > _2
1 package com.fenix;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 /**
7  * Esta clase procesa listas de usuarios. Contiene 'code smells' intencionados.
8  */
9 public class ProcesadorUsuarios {
10
11     private static final int _2 = 2;
12     private static final int ROL_INVITADO
13
14     // Método con 'code smell'
15     public String procesarLista(List<String> dataList) {
16         String admins = "";
17         String invitados = "";
18
19         for (String u : dataList) {
20             String[] parts = u.split(":"); // Formato "nombre:rol"
21             if (parts.length == 2) {
22                 String n = parts[0];
23                 int r = Integer.parseInt(parts[1]);
24
25                 // Número Mágico: 1 es Admin
26                 if (r == ROL_ADMIN) {
27                     admins += n + ",";
28                 }
29                 // Número Mágico: 2 es Invitado
30                 else if (r == 2) {
```

Al finalizar este paso, el código ya no contiene números sin contexto en la lógica de negocio, sino constantes descriptivas declaradas al inicio de la clase.

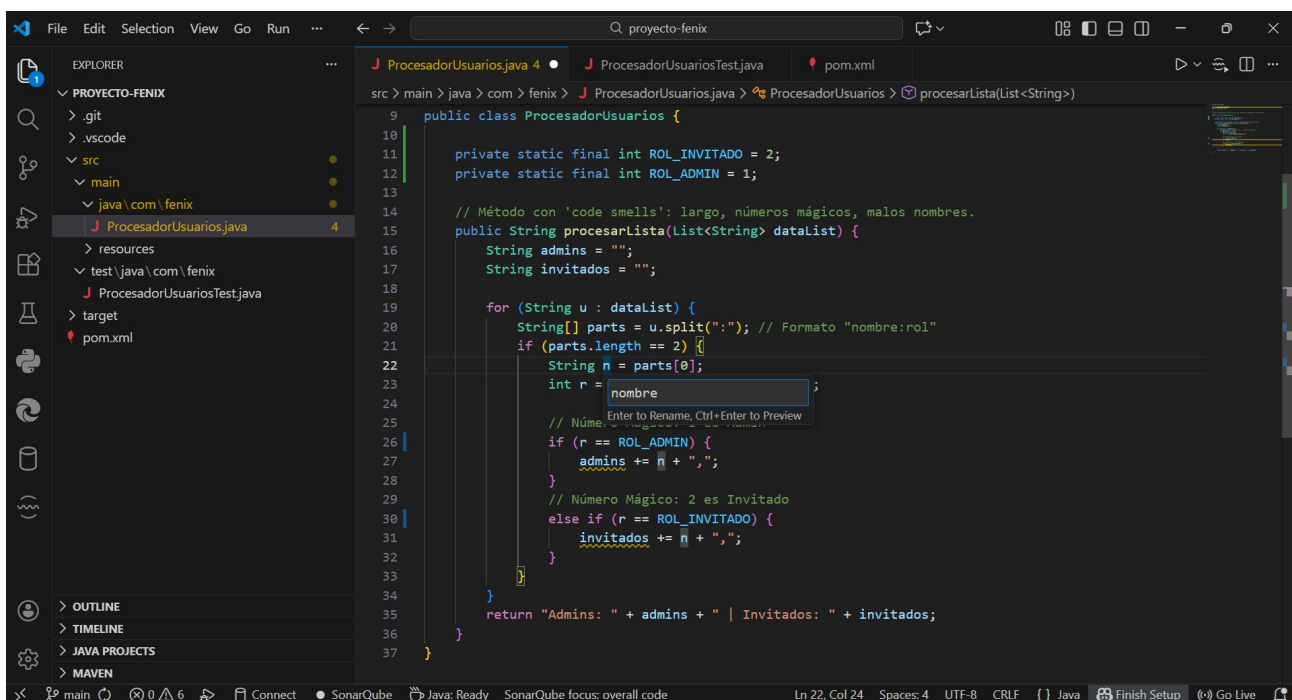


```
src > main > java > com > fenix > J ProcesadorUsuarios > ROL_INVITADO
1 package com.fenix;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 /**
7  * Esta clase procesa listas de usuarios. Contiene 'code smells' intencionados.
8  */
9 public class ProcesadorUsuarios {
10
11     private static final int ROL_INVITADO = 2;
12     private static final int ROL_ADMIN = 1;
13
14     // Método con 'code smells': largo, números mágicos, malos nombres.
15     public String procesarLista(List<String> dataList) {
16         String admins = "";
17         String invitados = "";
18
19         for (String u : dataList) {
20             String[] parts = u.split(":"); // Formato "nombre:rol"
21             if (parts.length == 2) {
22                 String n = parts[0];
23                 int r = Integer.parseInt(parts[1]);
24
25                 // Número Mágico: 1 es Admin
26                 if (r == ROL_ADMIN) {
27                     admins += n + ",";
28                 }
29                 // Número Mágico: 2 es Invitado
30                 else if (r == ROL_INVITADO) {
```

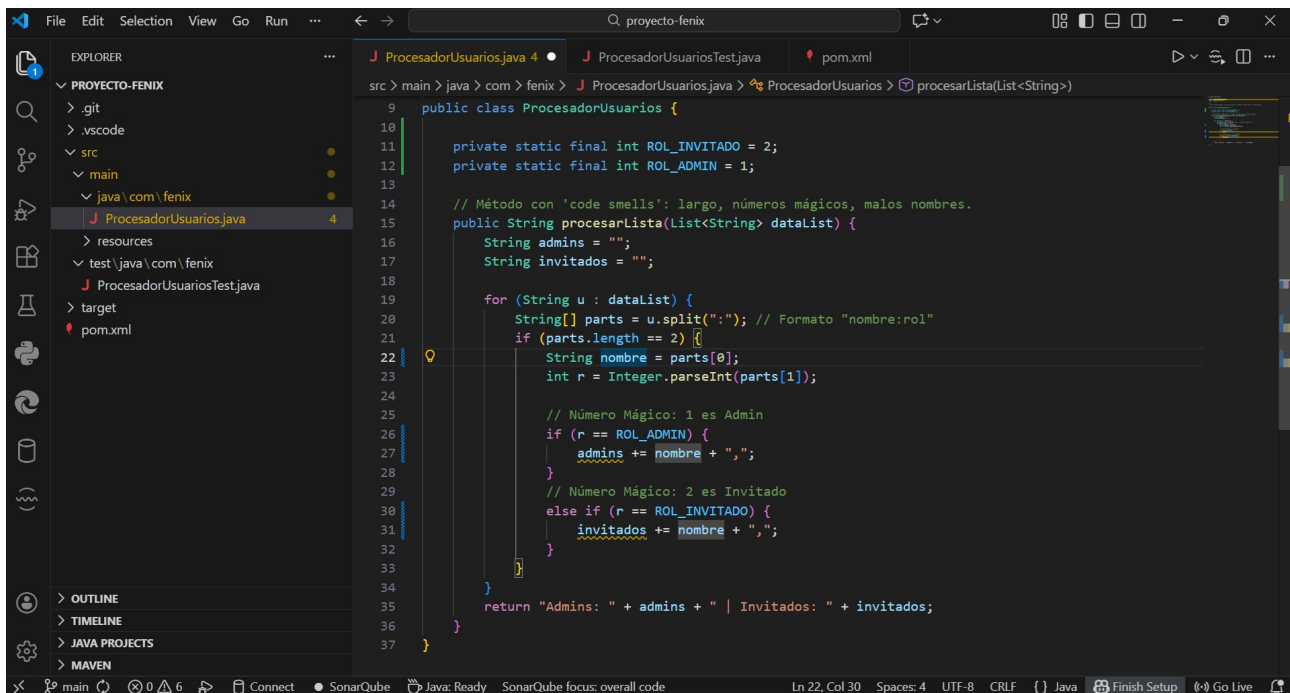
2. **Mejora de Nombres de Variables (Rename):** El siguiente "smell" identificado es el uso de nombres de variables poco descriptivos como *r*, *n* o *u*. Se comienza por la variable *r*, que almacena el rol numérico. Se utiliza la función *Rename Symbol* (F2) del IDE, que asegura que el cambio de nombre se propague a todas las referencias de la variable de forma segura.



3. **Renombrado de Variables Locales (n -> nombre):** Siguiendo con la mejora de la semántica, se identifica la variable *n* como un nombre poco descriptivo para almacenar el nombre del usuario. Se procede a su refactorización mediante la herramienta de renombrado.

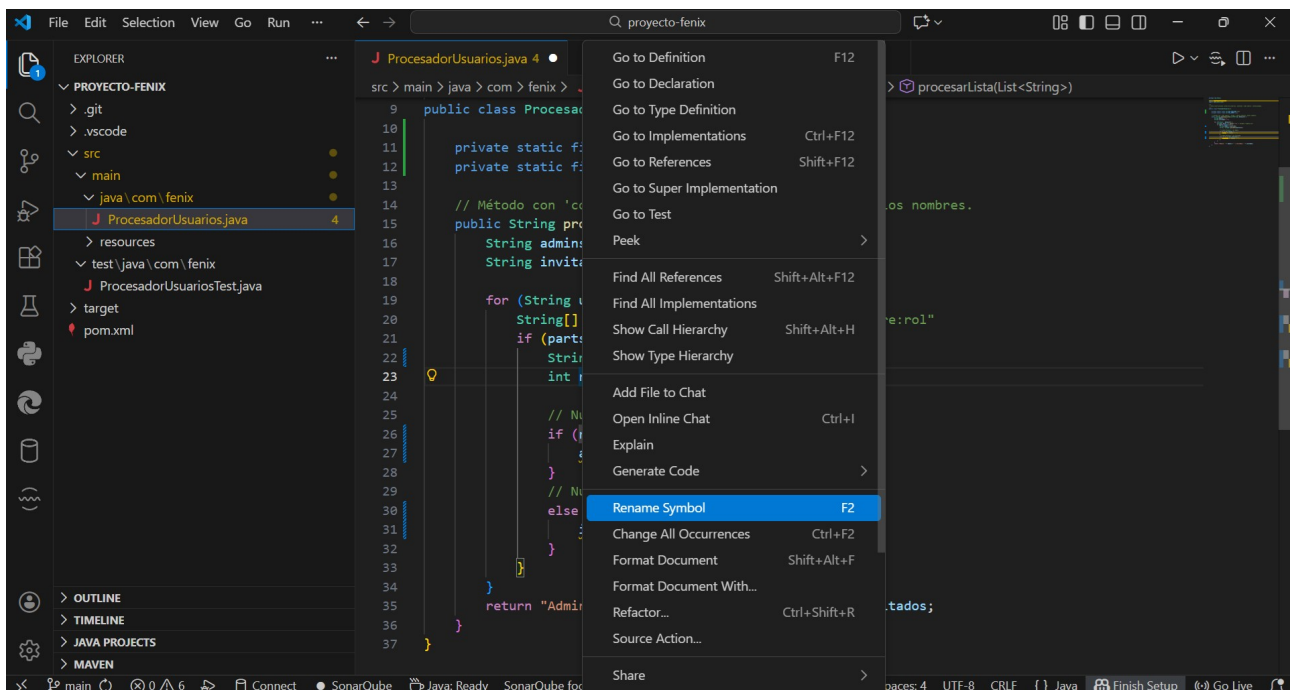


Tras la operación, el código refleja el cambio *String nombre = parts[0]*, haciendo evidente el propósito de la variable sin necesidad de comentarios adicionales.

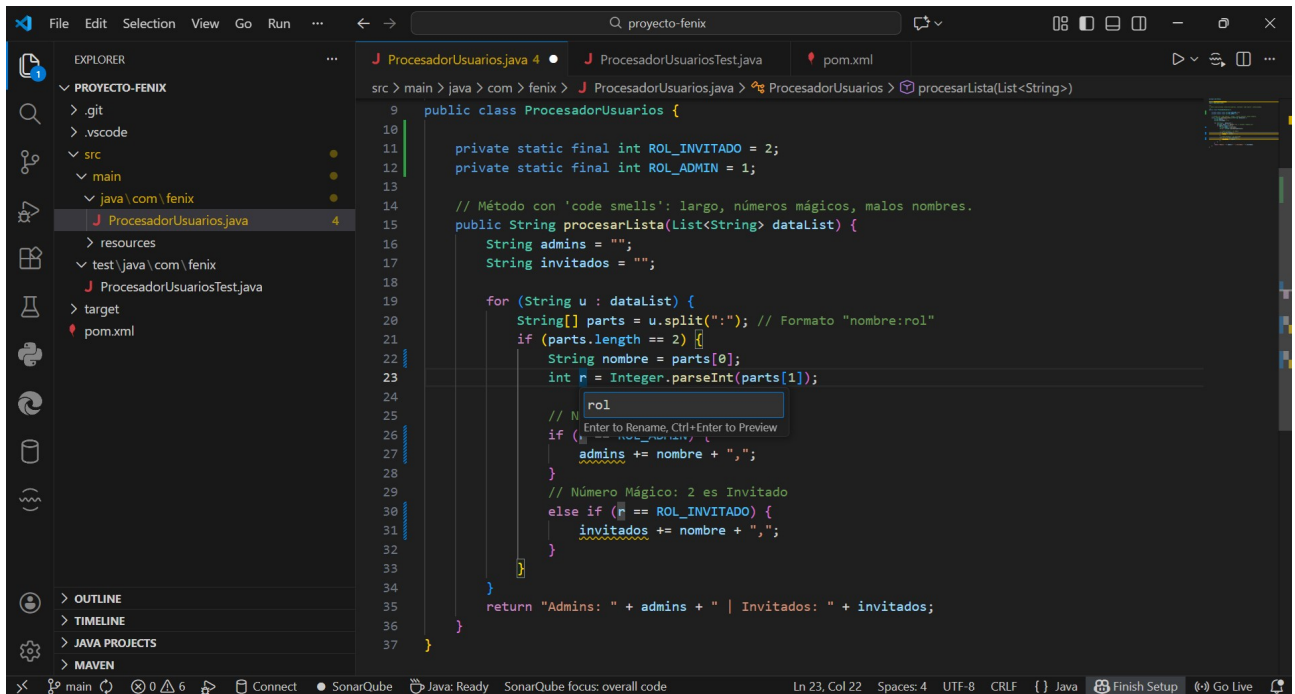


```
9 public class ProcesadorUsuarios {
10
11     private static final int ROL_INVITADO = 2;
12     private static final int ROL_ADMIN = 1;
13
14     // Método con 'code smells': largo, números mágicos, malos nombres.
15     public String procesarLista(List<String> dataList) {
16         String admins = "";
17         String invitados = "";
18
19         for (String u : dataList) {
20             String[] parts = u.split(":"); // Formato "nombre:rol"
21             if (parts.length == 2) {
22                 String nombre = parts[0];
23                 int r = Integer.parseInt(parts[1]);
24
25                 // Número Mágico: 1 es Admin
26                 if (r == ROL_ADMIN) {
27                     admins += nombre + ",";
28                 }
29                 // Número Mágico: 2 es Invitado
30                 else if (r == ROL_INVITADO) {
31                     invitados += nombre + ",";
32                 }
33             }
34         }
35         return "Admins: " + admins + " | Invitados: " + invitados;
36     }
37 }
```

- 4. Renombrado de Variables Locales (*r* -> *rol*):** De igual manera, se completa el cambio de la variable *r* (iniciado anteriormente) por *rol*. El IDE actualiza automáticamente tanto la declaración como las referencias en las condiciones lógicas *if (rol == ...)* y el *parsing* del entero.



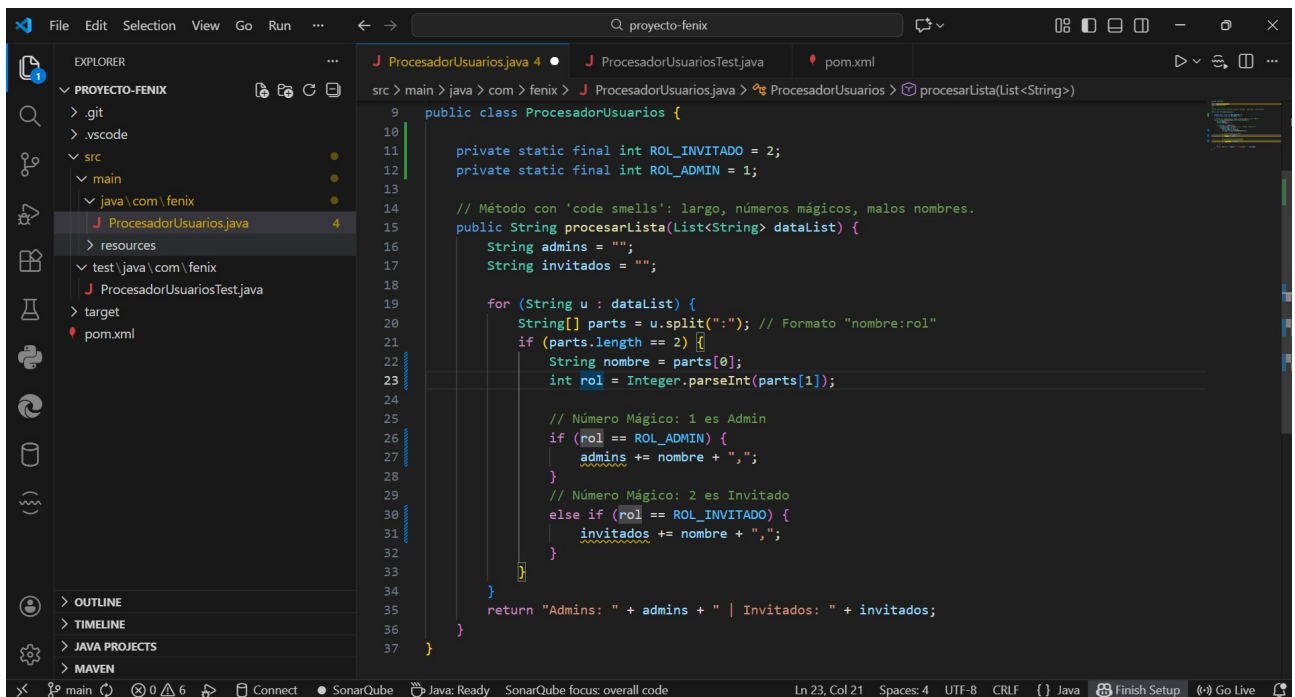
```
9 public class ProcesadorUsuarios {
10
11     private static final int ROL_INVITADO = 2;
12     private static final int ROL_ADMIN = 1;
13
14     // Método con 'code smells': largo, números mágicos, malos nombres.
15     public String procesarLista(List<String> dataList) {
16         String admins = "";
17         String invitados = "";
18
19         for (String u : dataList) {
20             String[] parts = u.split(":"); // Formato "nombre:rol"
21             if (parts.length == 2) {
22                 String nombre = parts[0];
23                 int r = Integer.parseInt(parts[1]);
24
25                 // Número Mágico: 1 es Admin
26                 if (r == ROL_ADMIN) {
27                     admins += nombre + ",";
28                 }
29                 // Número Mágico: 2 es Invitado
30                 else if (r == ROL_INVITADO) {
31                     invitados += nombre + ",";
32                 }
33             }
34         }
35         return "Admins: " + admins + " | Invitados: " + invitados;
36     }
37 }
```



This screenshot shows the VS Code editor with the file `ProcesadorUsuarios.java` open. The Explorer sidebar on the left shows the project structure: `PROYECTO-FENIX` with subfolders `.git`, `.vscode`, `src` (containing `main` and `test`), and `target`. The `main` folder contains `java\com\fenix` with `ProcesadorUsuarios.java` and `ProcesadorUsuariosTest.java`. The `test` folder contains `ProcesadorUsuariosTest.java`. The `target` folder contains `pom.xml`. The main editor shows the `ProcesadorUsuarios.java` file with the following code:

```
9 public class ProcesadorUsuarios {
10
11     private static final int ROL_INVITADO = 2;
12     private static final int ROL_ADMIN = 1;
13
14     // Método con 'code smells': largo, números mágicos, malos nombres.
15     public String procesarLista(List<String> dataList) {
16         String admins = "";
17         String invitados = "";
18
19         for (String u : dataList) {
20             String[] parts = u.split(":"); // Formato "nombre:rol"
21             if (parts.length == 2) {
22                 String nombre = parts[0];
23                 int r = Integer.parseInt(parts[1]);
24
25                 // Número Mágico: 2 es Invitado
26                 if (r == ROL_INVITADO) {
27                     admins += nombre + ",";
28                 }
29                 // Número Mágico: 1 es Admin
30                 else if (r == ROL_ADMIN) {
31                     invitados += nombre + ",";
32                 }
33             }
34         }
35         return "Admins: " + admins + " | Invitados: " + invitados;
36     }
37 }
```

A tooltip is visible over the `rol` variable on line 23, showing the text "rol" and "Enter to Rename, Ctrl+Enter to Preview". The status bar at the bottom indicates the file is at line 23, column 22, with 4 spaces, UTF-8 encoding, and CRLF line endings.

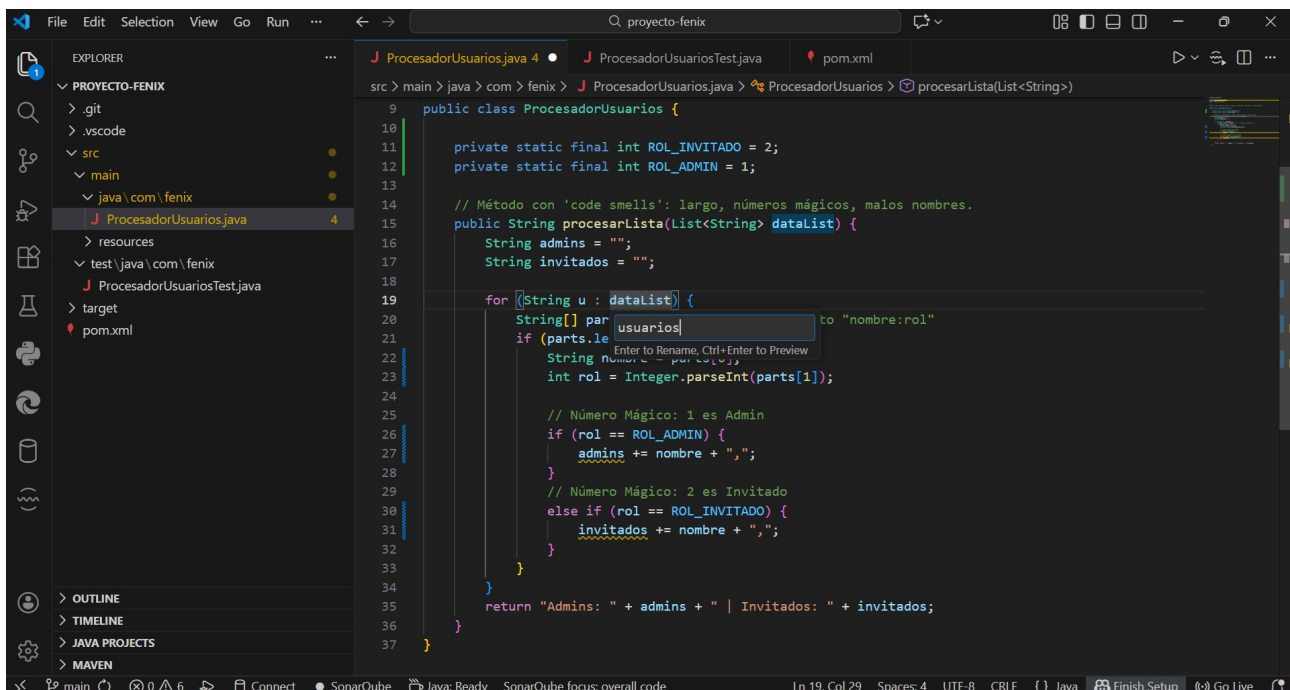
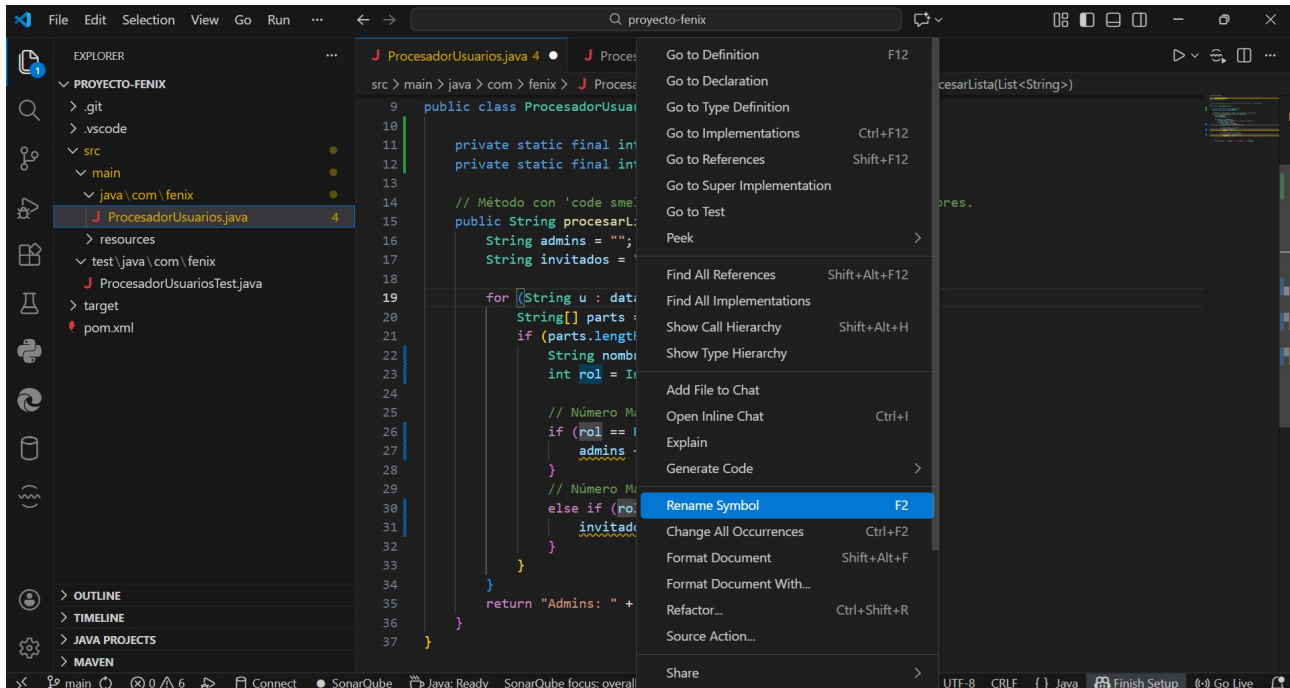


This screenshot shows the same VS Code editor with the file `ProcesadorUsuarios.java` open. The Explorer sidebar on the left shows the project structure: `PROYECTO-FENIX` with subfolders `.git`, `.vscode`, `src` (containing `main` and `test`), and `target`. The `main` folder contains `java\com\fenix` with `ProcesadorUsuarios.java` and `ProcesadorUsuariosTest.java`. The `test` folder contains `ProcesadorUsuariosTest.java`. The `target` folder contains `pom.xml`. The main editor shows the `ProcesadorUsuarios.java` file with the following code:

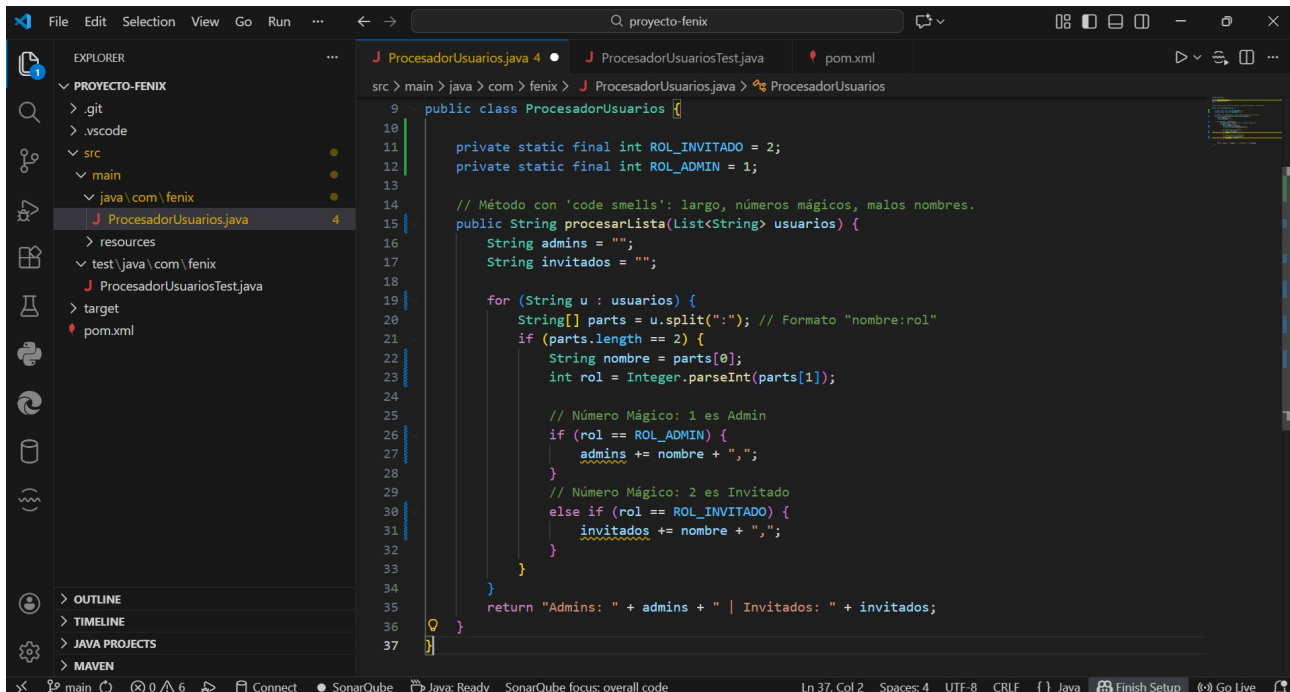
```
9 public class ProcesadorUsuarios {
10
11     private static final int ROL_INVITADO = 2;
12     private static final int ROL_ADMIN = 1;
13
14     // Método con 'code smells': largo, números mágicos, malos nombres.
15     public String procesarLista(List<String> dataList) {
16         String admins = "";
17         String invitados = "";
18
19         for (String u : dataList) {
20             String[] parts = u.split(":"); // Formato "nombre:rol"
21             if (parts.length == 2) {
22                 String nombre = parts[0];
23                 int rol = Integer.parseInt(parts[1]);
24
25                 // Número Mágico: 1 es Admin
26                 if (rol == ROL_ADMIN) {
27                     admins += nombre + ",";
28                 }
29                 // Número Mágico: 2 es Invitado
30                 else if (rol == ROL_INVITADO) {
31                     invitados += nombre + ",";
32                 }
33             }
34         }
35         return "Admins: " + admins + " | Invitados: " + invitados;
36     }
37 }
```

The status bar at the bottom indicates the file is at line 23, column 21, with 4 spaces, UTF-8 encoding, and CRLF line endings.

5. Renombrado de Parámetros (dataList -> usuarios): Por último, se aborda el nombre del parámetro de entrada del método, *dataList*. Al tratarse de una lista de cadenas que representan usuarios, el nombre actual es genérico y poco informativo. Se utiliza nuevamente la herramienta *Rename Symbol* para asignarle el identificador *usuarios*.

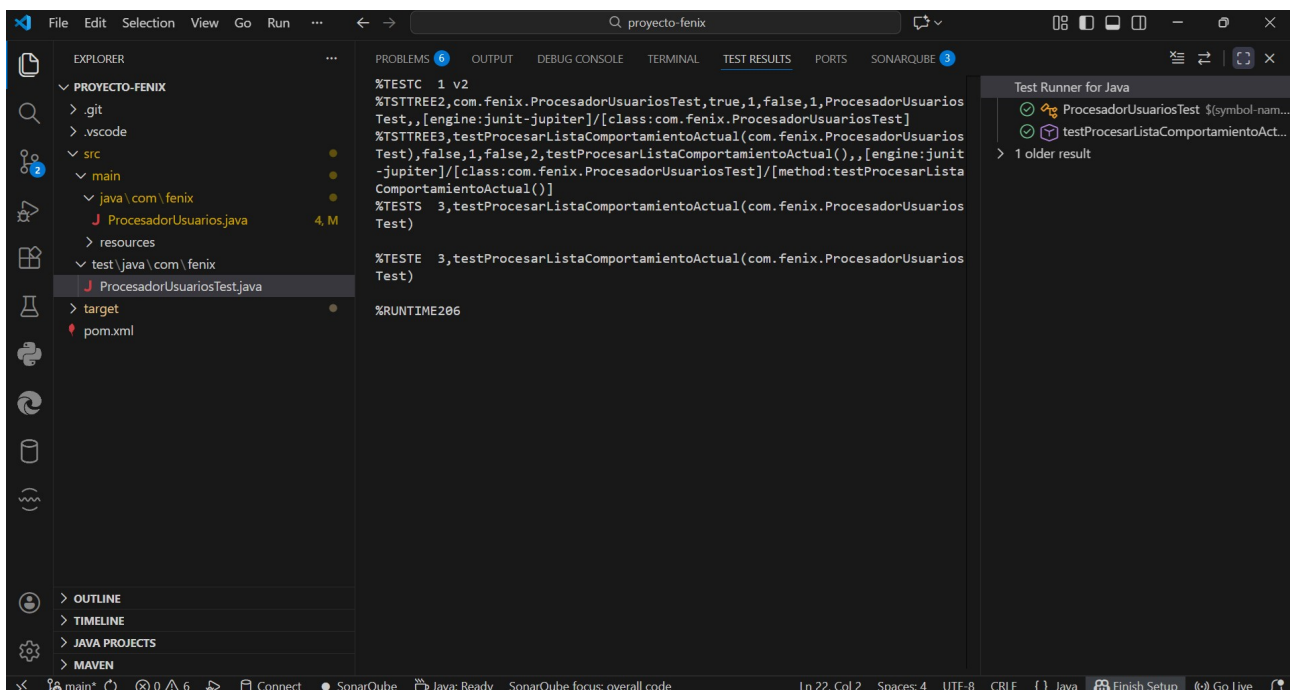


El resultado final de esta etapa de limpieza es un código donde la firma del método *procesarLista*(*List<String> usuarios*) y su lógica interna utilizan terminología del dominio del problema (*usuarios, nombre, rol*) en lugar de abreviaturas técnicas.



```
9 public class ProcesadorUsuarios {
10
11     private static final int ROL_INVITADO = 2;
12     private static final int ROL_ADMIN = 1;
13
14     // Método con 'code smells': largo, números mágicos, malos nombres.
15     public String procesarLista(List<String> usuarios) {
16         String admins = "";
17         String invitados = "";
18
19         for (String u : usuarios) {
20             String[] parts = u.split(":"); // Formato "nombre:rol"
21             if (parts.length == 2) {
22                 String nombre = parts[0];
23                 int rol = Integer.parseInt(parts[1]);
24
25                 // Número Mágico: 1 es Admin
26                 if (rol == ROL_ADMIN) {
27                     admins += nombre + ",";
28                 }
29                 // Número Mágico: 2 es Invitado
30                 else if (rol == ROL_INVITADO) {
31                     invitados += nombre + ",";
32                 }
33             }
34         }
35         return "Admins: " + admins + " | Invitados: " + invitados;
36     }
37 }
```

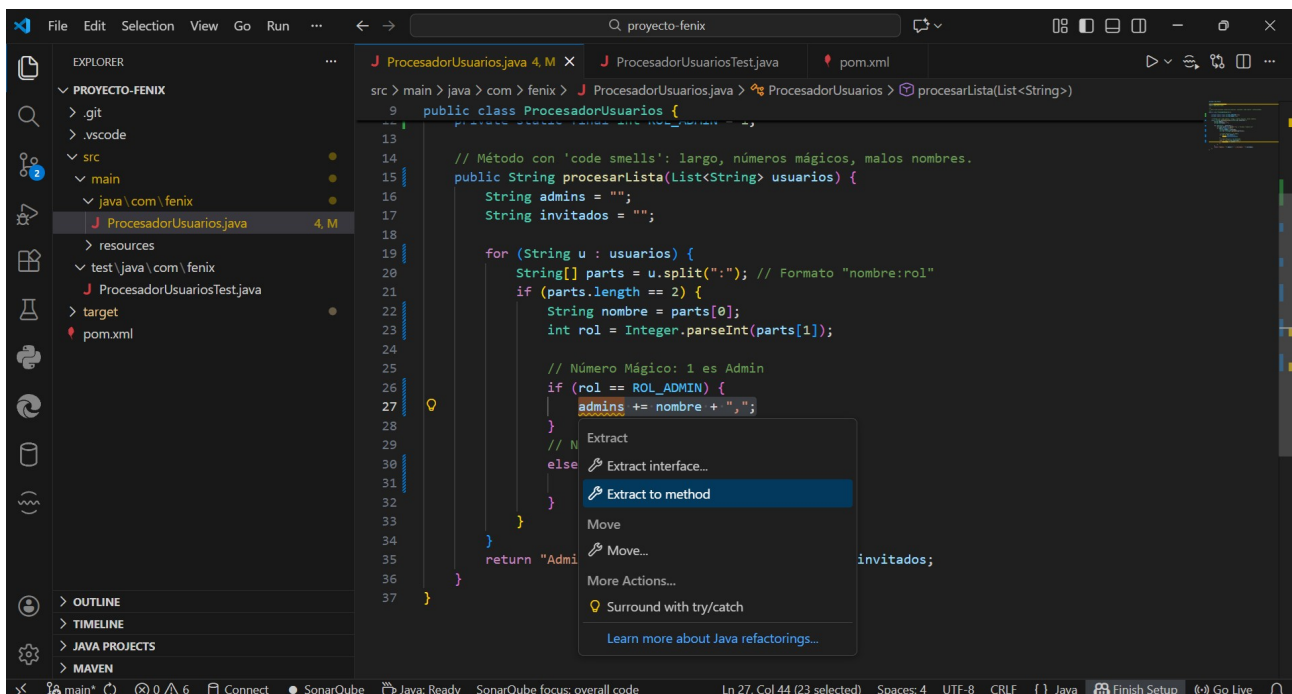
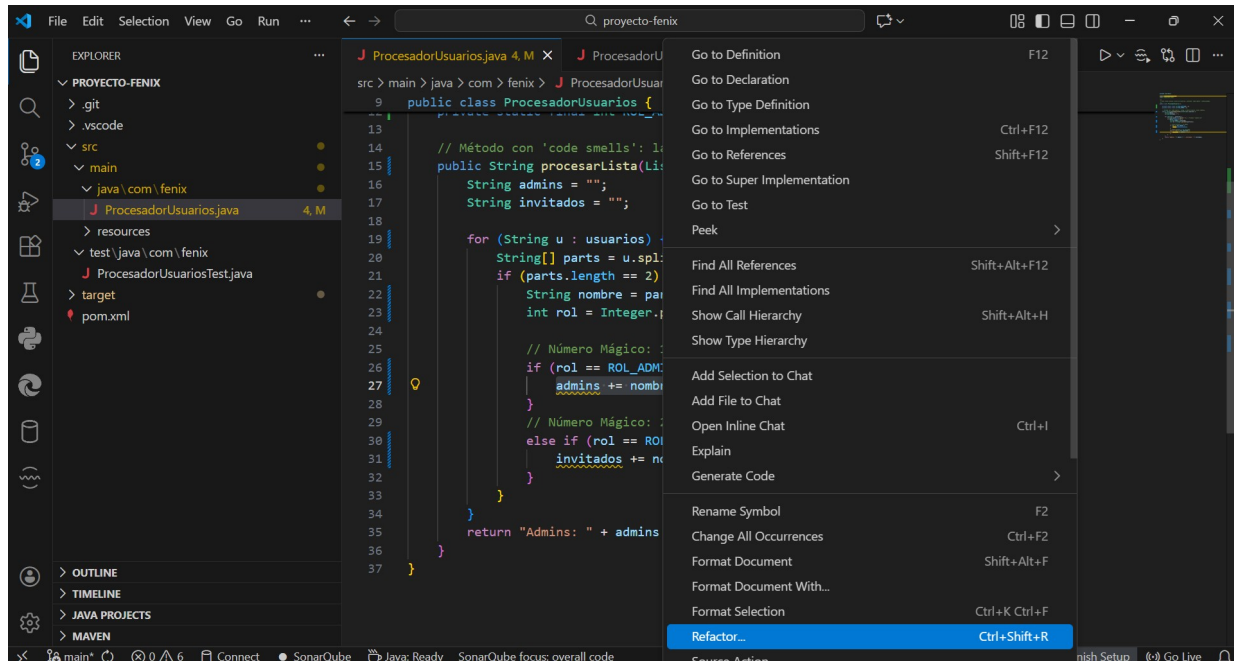
6. Verificación de Seguridad (Regression Testing): Tras realizar múltiples cambios estructurales en el código (extracción de constantes y renombrado de variables), es imperativo verificar que la funcionalidad no se ha visto alterada. Se ejecuta nuevamente la clase de prueba *ProcesadorUsuariosTest*.



```
%TESTC 1 v2
%TTT2,com.fenix.ProcesadorUsuariosTest,true,1,false,1,ProcesadorUsuarios
Test,,[engine:junit-jupiter]/[class:com.fenix.ProcesadorUsuariosTest]
%TTT3,testProcesarListaComportamientoActual(com.fenix.ProcesadorUsuarios
Test),false,1,false,2,testProcesarListaComportamientoActual(),,[engine:junit
-jupiter]/[class:com.fenix.ProcesadorUsuariosTest]/[method:testProcesarLista
ComportamientoActual()]
%TESTS 3,testProcesarListaComportamientoActual(com.fenix.ProcesadorUsuarios
Test)
%TESTE 3,testProcesarListaComportamientoActual(com.fenix.ProcesadorUsuarios
Test)
%RUNTIME206
```

El resultado exitoso de la prueba confirma que la refactorización ha sido segura: el código es ahora más legible ("Clean Code") pero mantiene exactamente el mismo comportamiento externo que la versión heredada original.

7. **Extracción de Método (Admin):** Para reducir la complejidad del método principal y cumplir con el principio de responsabilidad única, se selecciona el bloque de código encargado de procesar a los administradores. A través del menú *Refactor*, se elige la opción *Extract to method*.

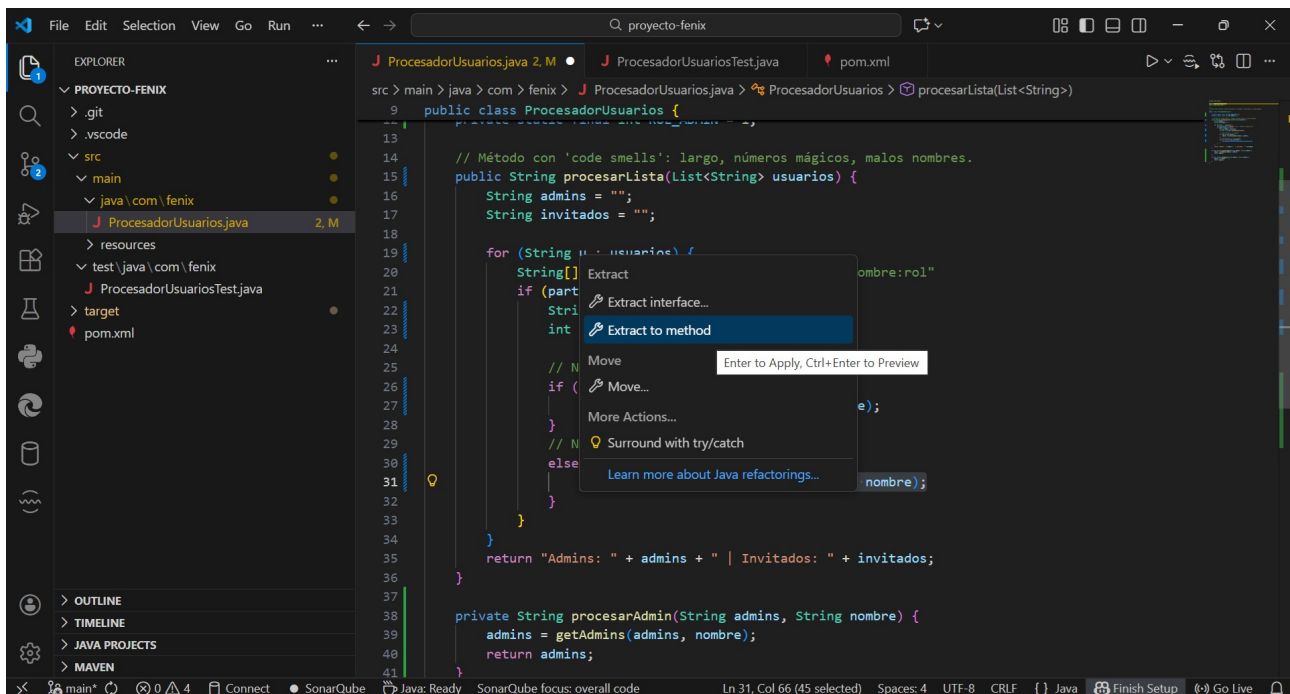


Se nombra al nuevo método privado como *procesarAdmin*. Durante este proceso, se presentó una incidencia técnica: debido a que la lógica de concatenación es sintácticamente idéntica en ambos bloques, el IDE intentó aplicar el método *procesarAdmin* también al bloque de invitados. Para corregir esta "falsa duplicidad", se procedió a restaurar manualmente la línea *invitados += nombre + ", "*; dentro del *else if*, asegurando que cada rol mantenga su propio flujo lógico antes de continuar.

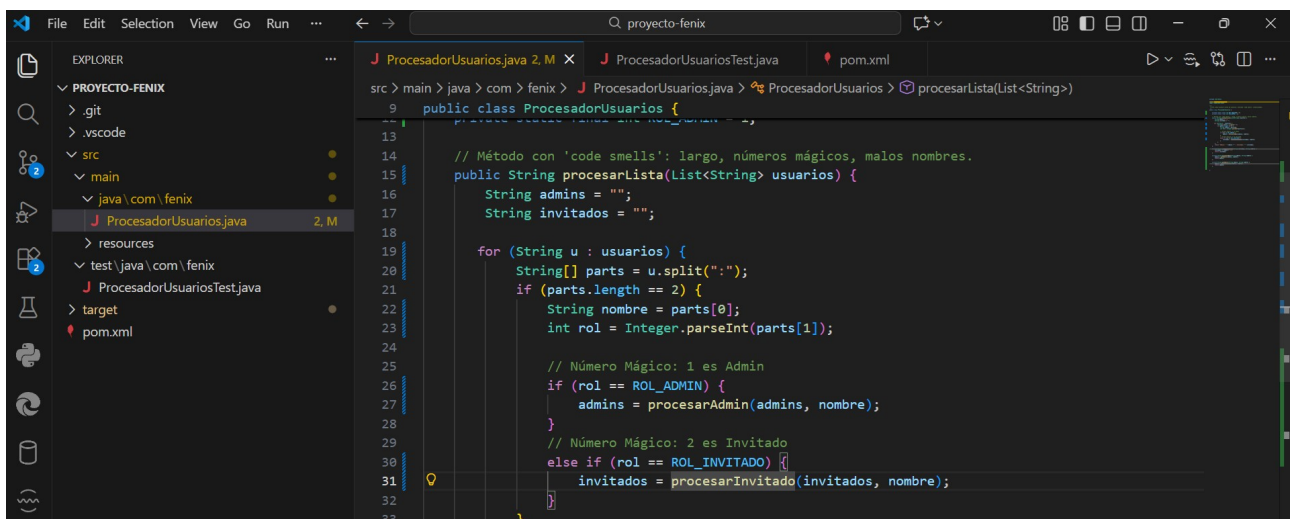
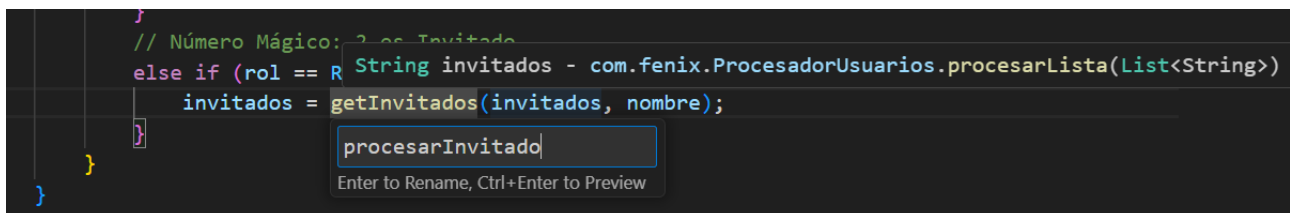
```
24
25 // Número Mágico: 1 es Admin
26 if (rol == ROL_ADMIN) {
27     admins = getAdmins(admins, nombre);
28 }
29 // Número Mág
30 else if (rol == ROL_INVITADO) {
31     invitados = getAdmins(invitados, nombre);
```

8. Extracción de Método (Invitado): Con el código restaurado, se procede a aislar la lógica restante. Se selecciona el cuerpo del bloque *else if* correspondiente al rol de invitado y se repite la operación de extracción.

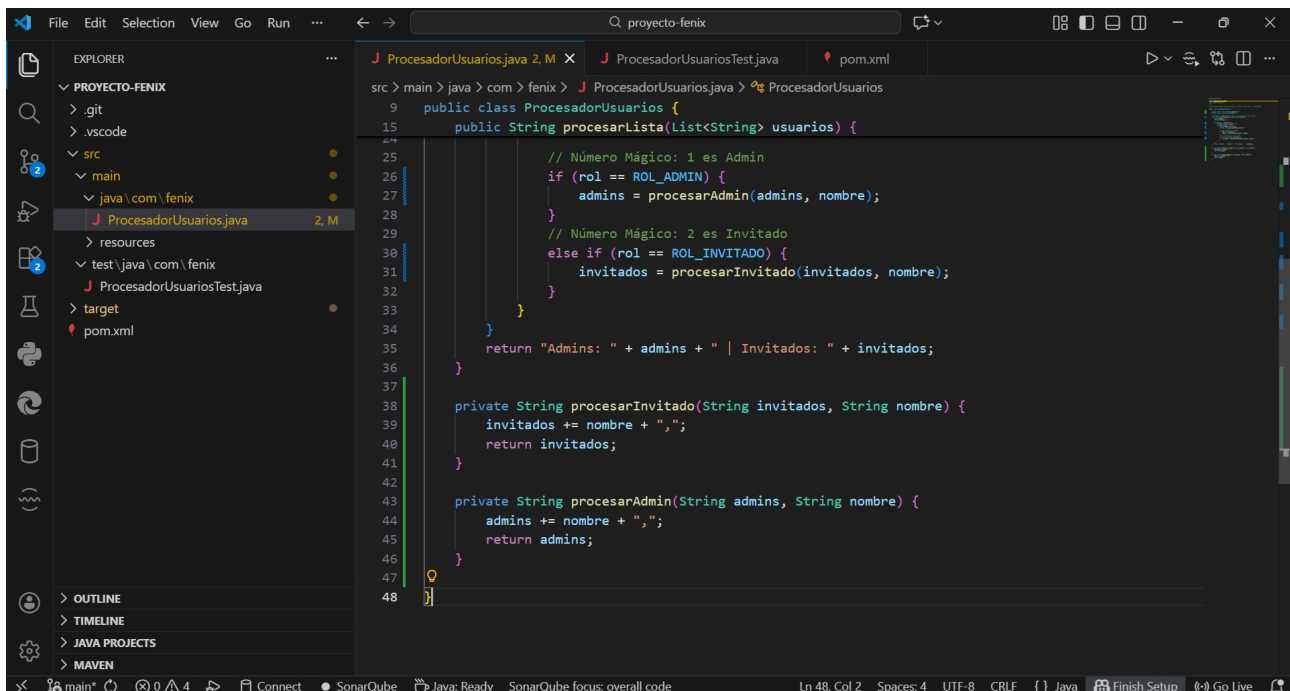
The screenshot shows the VS Code editor with the file `ProcesadorUsuarios.java` open. The `procesarLista` method is visible, containing a loop that processes a list of users. The `else if` block for `ROL_INVITADO` is highlighted, showing the logic for concatenating the invited user's name to the `invitados` string. The `procesarAdmin` method is also visible, showing the logic for concatenating the admin's name to the `admins` string. The status bar at the bottom indicates the current position is `Ln 33, Col 14`.



Se asigna el nombre *procesarInvitado* a este nuevo método. Ahora, el método original delega la lógica de negocio en dos métodos auxiliares privados, mejorando drásticamente la legibilidad.

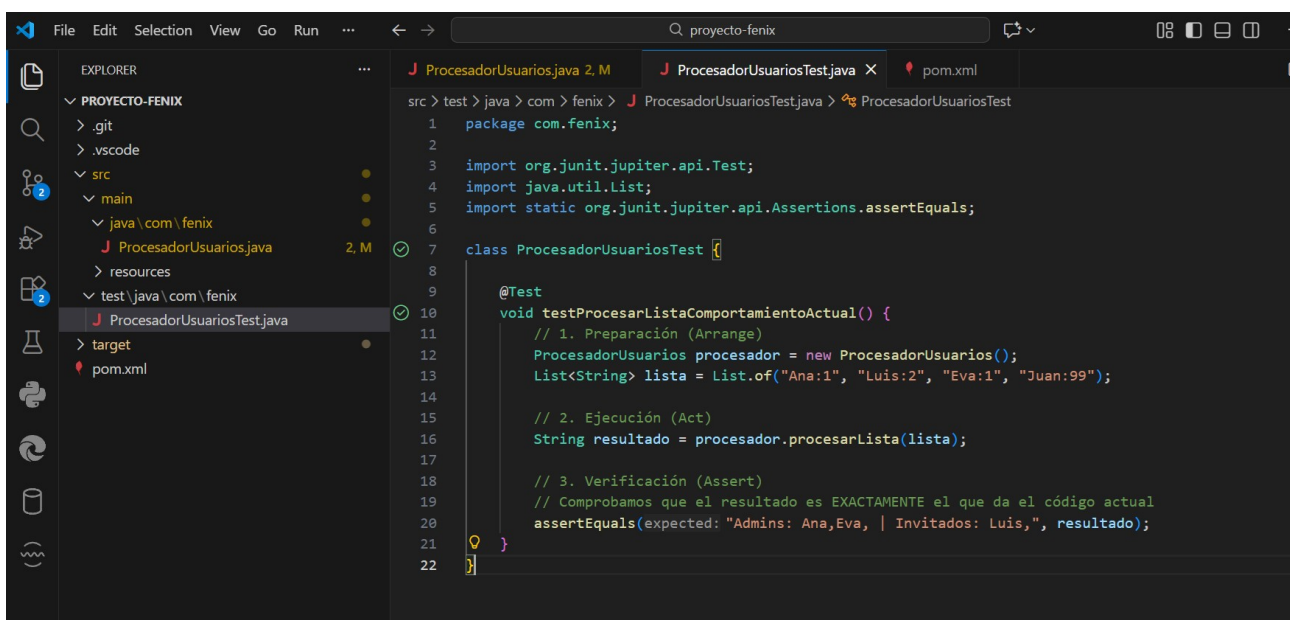


9. Revisión de la Estructura Final: Al examinar la clase *ProcesadorUsuarios*, se observa que el método público *procesarLista* es ahora conciso y fácil de leer. Los detalles de implementación se han movido a la parte inferior de la clase, encapsulados en *procesarInvitado* y *procesarAdmin*.



```
src > main > java > com > fenix > J ProcesadorUsuariosTest.java > ProcesadorUsuarios
9 public class ProcesadorUsuarios {
15     public String procesarLista(List<String> usuarios) {
16         // Número Mágico: 1 es Admin
17         if (rol == ROL_ADMIN) {
18             admins = procesarAdmin(admins, nombre);
19         }
20         // Número Mágico: 2 es Invitado
21         else if (rol == ROL_INVITADO) {
22             invitados = procesarInvitado(invitados, nombre);
23         }
24     }
25     return "Admins: " + admins + " | Invitados: " + invitados;
26 }
27
28 private String procesarInvitado(String invitados, String nombre) {
29     invitados += nombre + ",";
30     return invitados;
31 }
32
33 private String procesarAdmin(String admins, String nombre) {
34     admins += nombre + ",";
35     return admins;
36 }
37
38 }
39
40
41
42
43
44
45
46
47
48 }
```

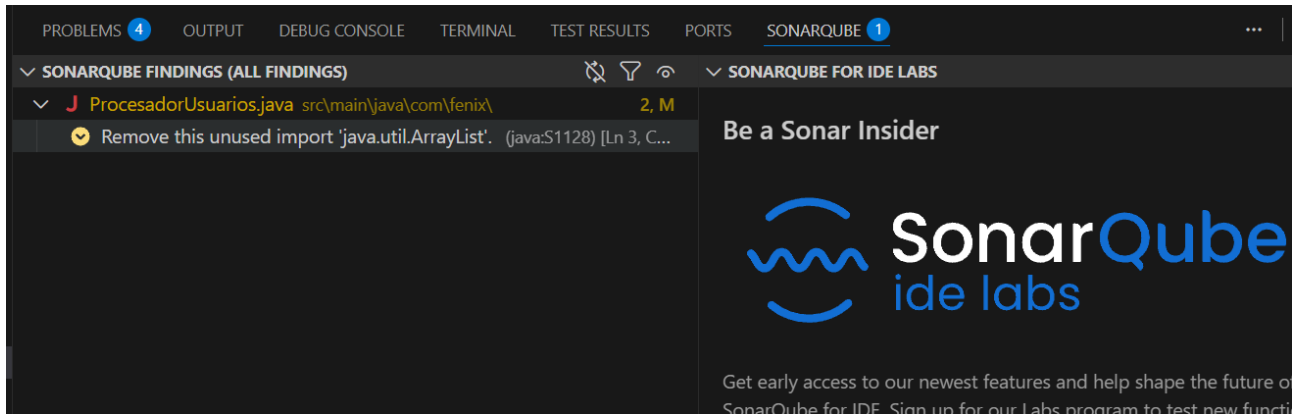
10. Verificación Final (Test de Regresión): Como dicta la metodología de refactorización, tras realizar cambios estructurales significativos (Extract Method), se ejecutan nuevamente las pruebas unitarias.



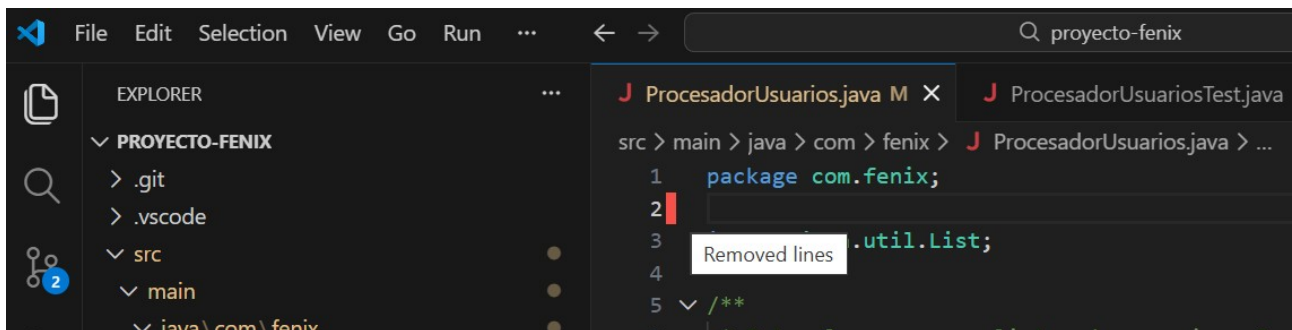
```
src > test > java > com > fenix > J ProcesadorUsuariosTest.java > ProcesadorUsuariosTest
1 package com.fenix;
2
3 import org.junit.jupiter.api.Test;
4 import java.util.List;
5 import static org.junit.jupiter.api.Assertions.assertEquals;
6
7 class ProcesadorUsuariosTest {
8
9     @Test
10     void testProcesarListaComportamientoActual() {
11         // 1. Preparación (Arrange)
12         ProcesadorUsuarios procesador = new ProcesadorUsuarios();
13         List<String> lista = List.of("Ana:1", "Luis:2", "Eva:1", "Juan:99");
14
15         // 2. Ejecución (Act)
16         String resultado = procesador.procesarLista(lista);
17
18         // 3. Verificación (Assert)
19         // Comprobamos que el resultado es EXACTAMENTE el que da el código actual
20         assertEquals(expected: "Admins: Ana,Eva, | Invitados: Luis,", resultado);
21     }
22 }
```

El test pasa correctamente, confirmando que la división del código en submétodos no ha alterado el comportamiento del programa.

11.Estado del Análisis Estático: Se consulta el panel de *SonarQube* para verificar el impacto de los cambios. Se observa que las advertencias sobre complejidad y duplicidad han desaparecido, persistiendo únicamente el aviso sobre la importación no utilizada (que se abordará en la limpieza final), lo que indica una mejora sustancial en la calidad del código.



Dado que la lista se maneja mediante la interfaz *List* y utilidades de *Arrays* o *List.of*, la importación de *ArrayList* es innecesaria. Se procede a eliminar la línea correspondiente para dejar el archivo libre de advertencias.

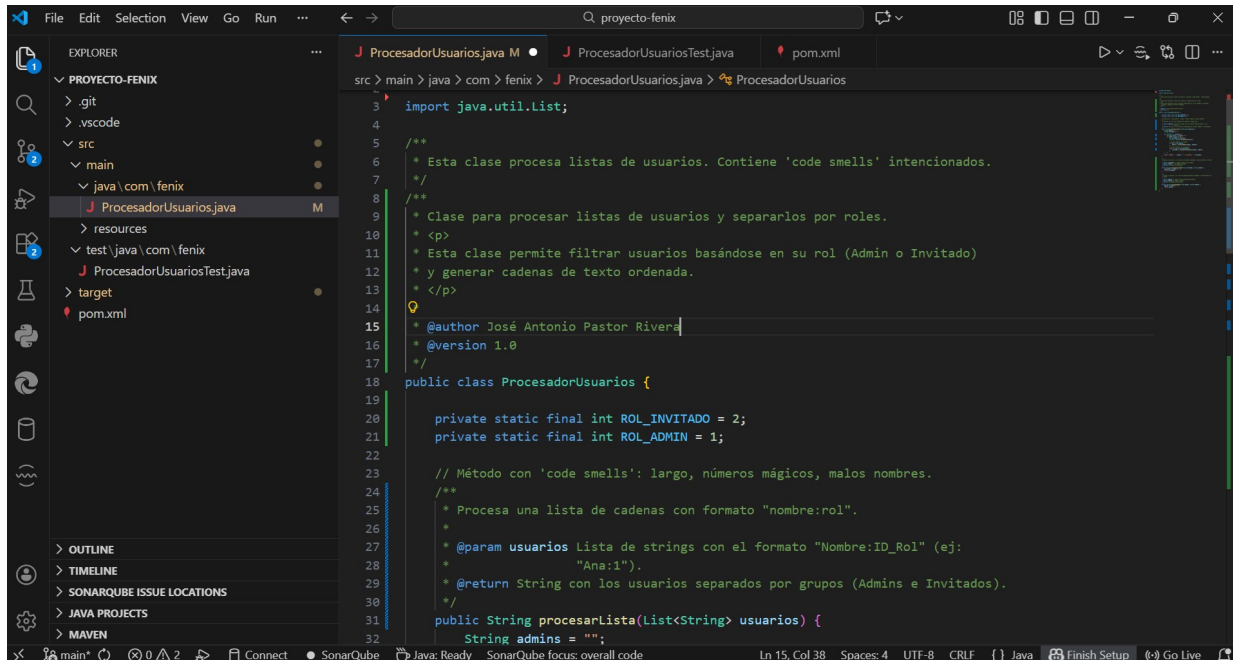


Parte 4: Documentación

Tarea 5: Generar documentación

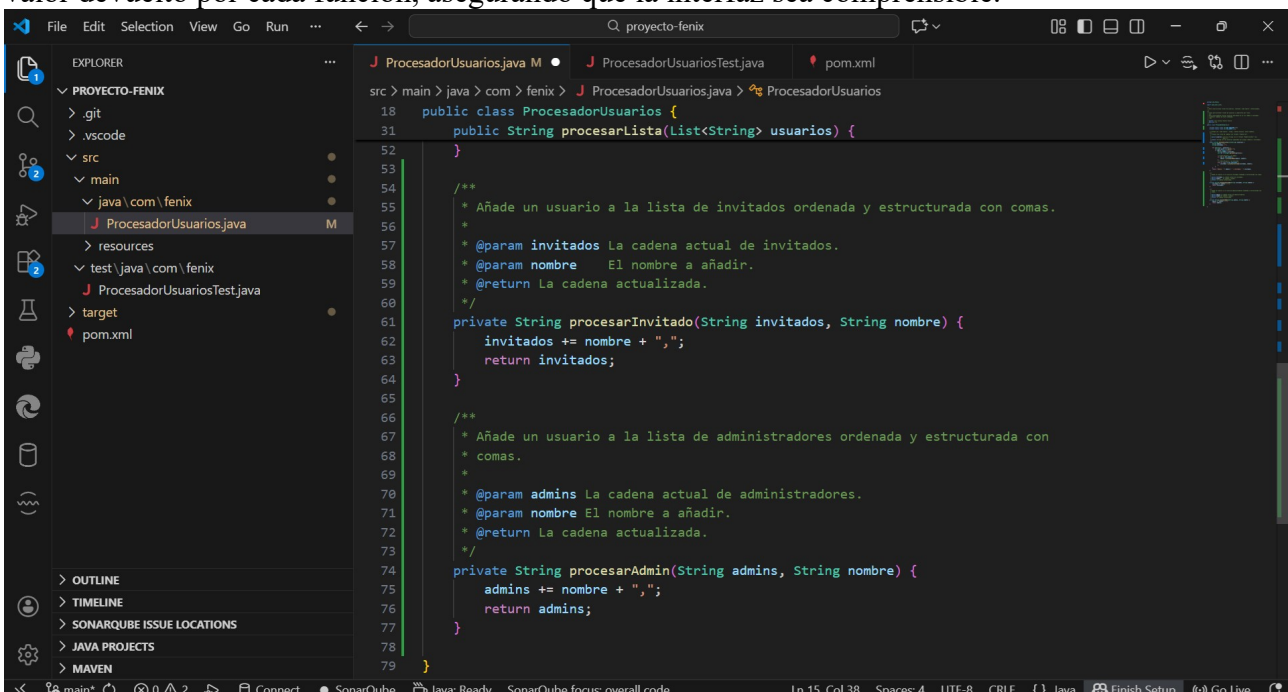
Objetivo: Incorporar comentarios técnicos estandarizados y generar la documentación HTML del proyecto utilizando las herramientas del entorno de desarrollo.

1. **Escritura de Comentarios Javadoc:** Siguiendo las especificaciones, se accede a la clase *ProcesadorUsuarios.java* ya refactorizada. Se insertan comentarios en formato Javadoc (`/** ... */`) en la cabecera de la clase, indicando la descripción general, el autor (`@author`) y la versión (`@version`)



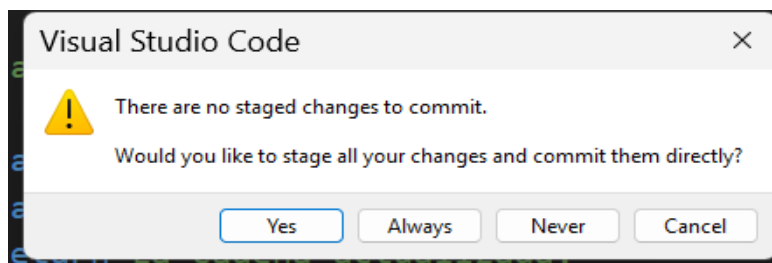
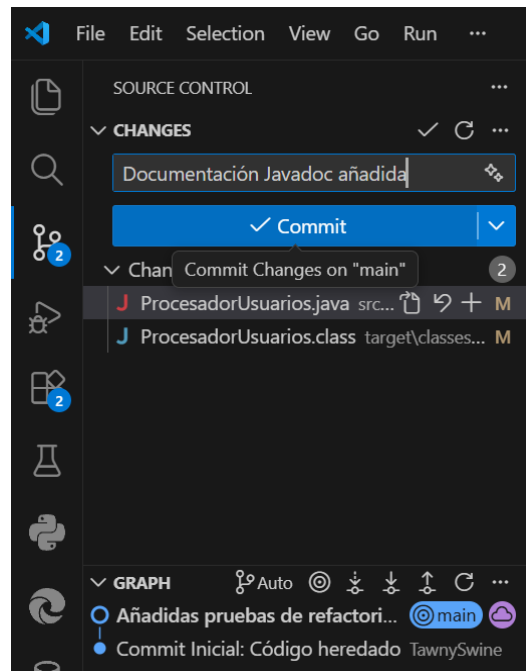
```
3 import java.util.List;
4
5 /**
6  * Esta clase procesa listas de usuarios. Contiene 'code smells' intencionados.
7  */
8 /**
9  * Clase para procesar listas de usuarios y separarlos por roles.
10  * <p>
11  * Esta clase permite filtrar usuarios basándose en su rol (Admin o Invitado)
12  * y generar cadenas de texto ordenada.
13  * </p>
14  */
15 * @author José Antonio Pastor Rivera
16 * @version 1.0
17 */
18 public class ProcesadorUsuarios {
19
20     private static final int ROL_INVITADO = 2;
21     private static final int ROL_ADMIN = 1;
22
23     // Método con 'code smells': largo, números mágicos, malos nombres.
24     /**
25      * Procesa una lista de cadenas con formato "nombre:rol".
26      *
27      * @param usuarios Lista de strings con el formato "Nombre:ID_Rol" (ej:
28      * "Ana:1").
29      * @return String con los usuarios separados por grupos (Admins e Invitados).
30      */
31     public String procesarLista(List<String> usuarios) {
32         String admins = "";
```

Posteriormente, se documentan los métodos públicos y privados. Se utilizan las etiquetas `@param` para describir los argumentos de entrada (*usuarios*, *invitados*, *nombre*) y `@return` para explicar el valor devuelto por cada función, asegurando que la interfaz sea comprensible.

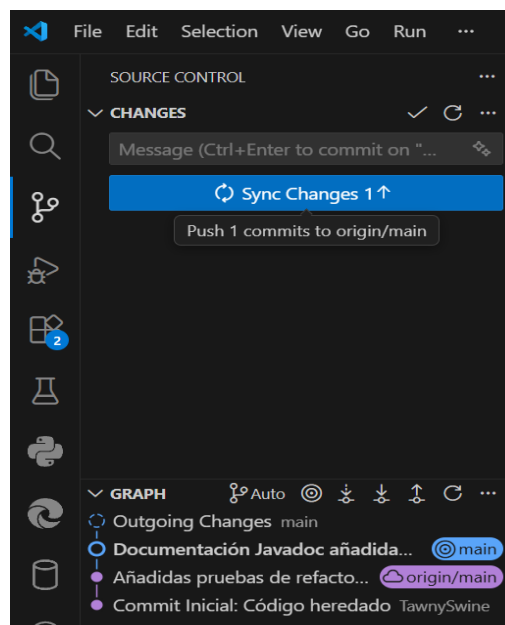


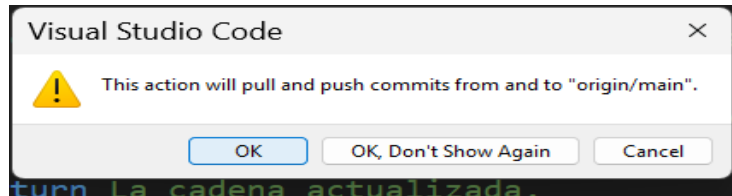
```
18 public class ProcesadorUsuarios {
19     public String procesarLista(List<String> usuarios) {
20     }
21
22     /**
23      * Añade un usuario a la lista de invitados ordenada y estructurada con comas.
24      *
25      * @param invitados La cadena actual de invitados.
26      * @param nombre El nombre a añadir.
27      * @return La cadena actualizada.
28      */
29     private String procesarInvitado(String invitados, String nombre) {
30         invitados += nombre + ",";
31         return invitados;
32     }
33
34     /**
35      * Añade un usuario a la lista de administradores ordenada y estructurada con
36      * comas.
37      *
38      * @param admins La cadena actual de administradores.
39      * @param nombre El nombre a añadir.
40      * @return La cadena actualizada.
41      */
42     private String procesarAdmin(String admins, String nombre) {
43         admins += nombre + ",";
44         return admins;
45     }
46 }
```

2. **Control de Versiones (Commit Solicitado):** Una vez completada la fase de escritura, tal como solicita el enunciado, se procede a guardar los cambios. En el panel de control de código fuente, se realiza el *stage* del archivo modificado y se crea un *commit* con el mensaje exacto: "Documentación Javadoc añadida".



Acto seguido, se sincronizan los cambios con el repositorio remoto (*Push*) para reflejar la actualización en GitHub.



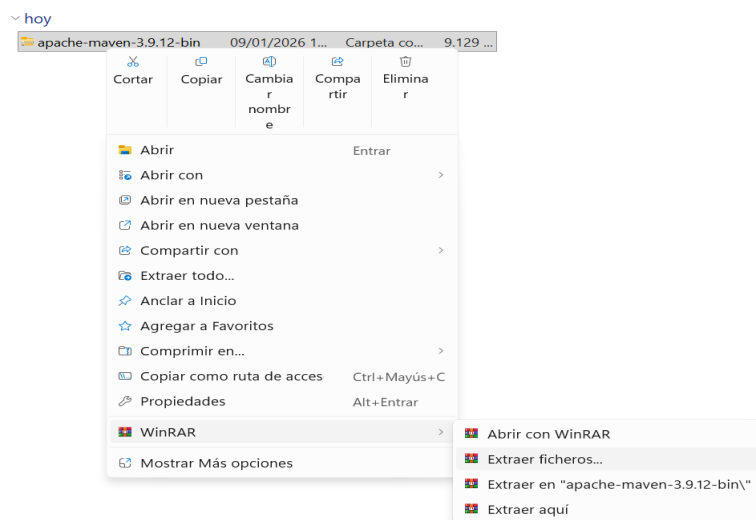


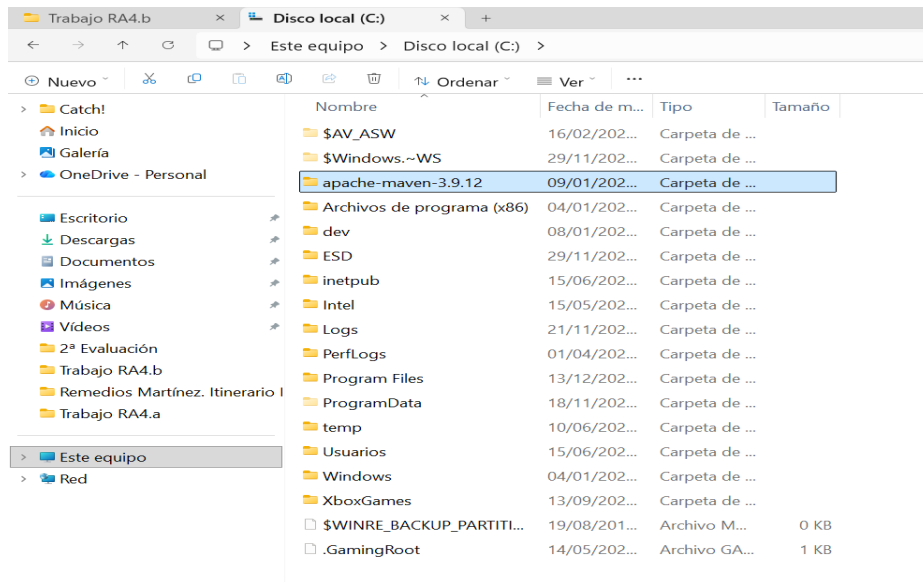
3. Configuración de la Herramienta de Generación (Maven): Para cumplir el requisito de generar la documentación HTML en este entorno, se requiere el uso de **Apache Maven** externo. Se descarga la última versión binaria desde el sitio oficial.

The screenshot shows the Apache Maven download page for version 3.9.12. The page includes a sidebar with navigation links like 'Downloads', 'Documentation', and 'Community'. The main content area features the title 'Apache Maven 3.9.12' and a table of download links and checksums.

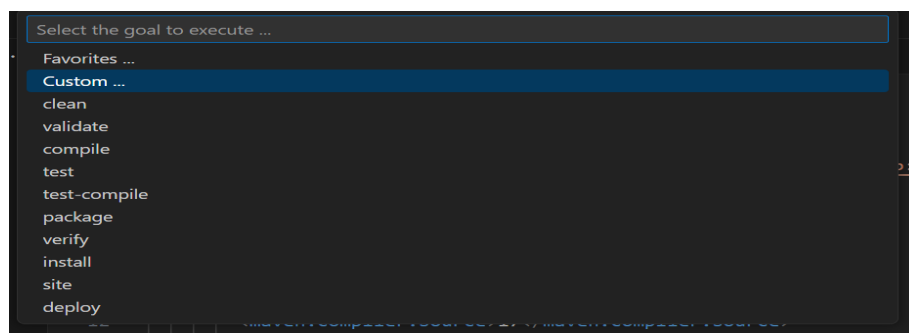
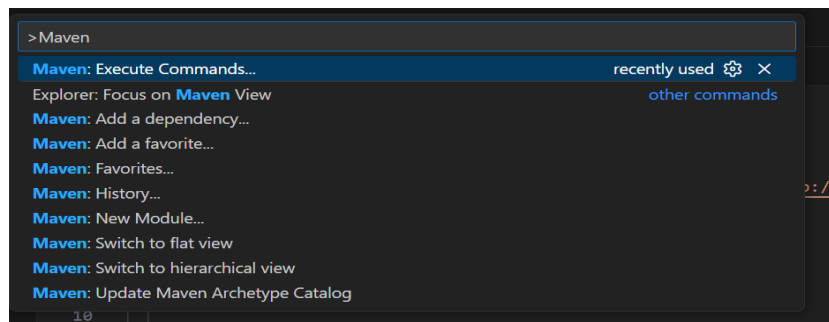
	Link	Checksums	Signature
Binary tar.gz archive	apache-maven-3.9.12-bin.tar.gz	apache-maven-3.9.12-bin.tar.gz.sha512	apache-maven-3.9.12-bin.tar.gz.asc
Binary zip archive	apache-maven-3.9.12-bin.zip	apache-maven-3.9.12-bin.zip.sha512	apache-maven-3.9.12-bin.zip.asc
Source tar.gz archive	apache-maven-3.9.12-src.tar.gz	apache-maven-3.9.12-src.tar.gz.sha512	apache-maven-3.9.12-src.tar.gz.asc
Source zip archive	apache-maven-3.9.12-src.zip	apache-maven-3.9.12-src.zip.sha512	apache-maven-3.9.12-src.zip.asc

Se descomprime el archivo en el disco local para tener acceso a las herramientas de línea de comandos.

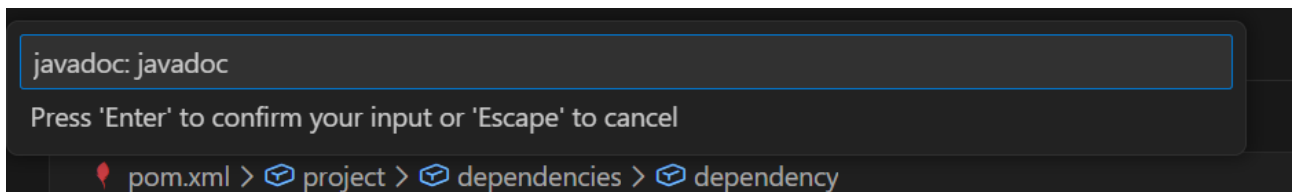




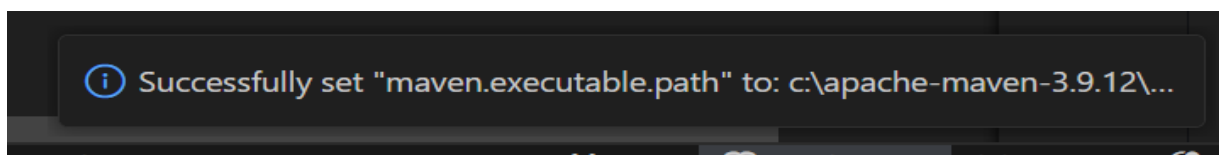
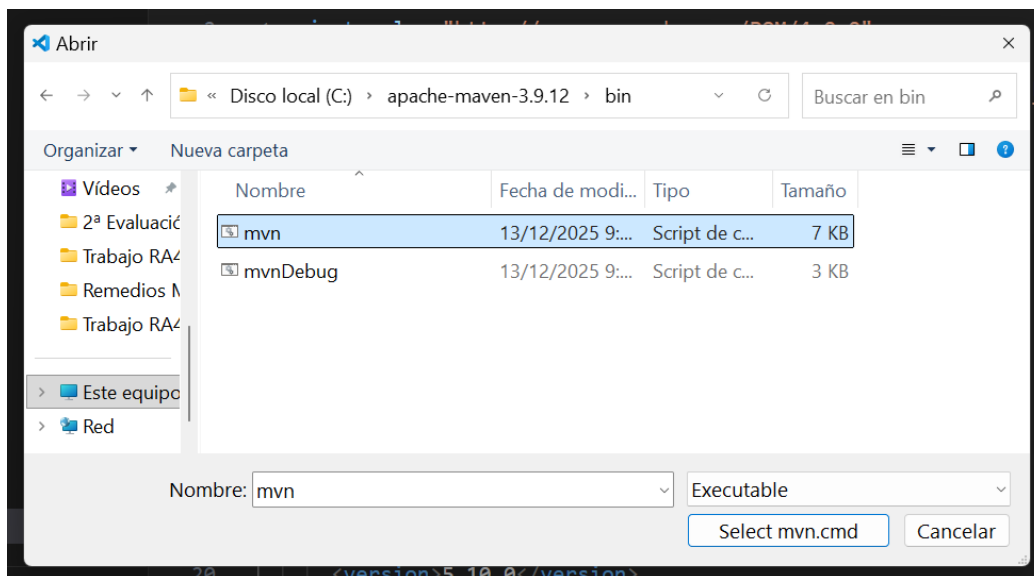
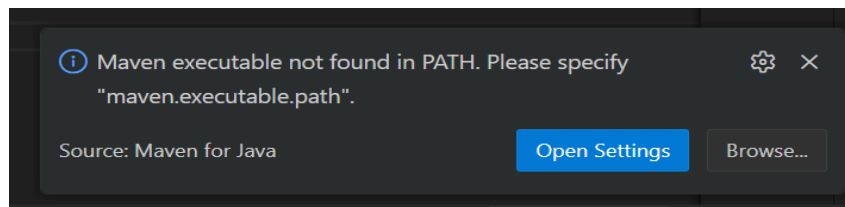
- 4. Generación de Documentación desde el IDE:** Se utiliza la integración de Maven en Visual Studio Code. Desde la paleta de comandos, se accede a *Maven: Execute Commands* y se selecciona la ejecución de un objetivo personalizado (*Custom*).



Se introduce el comando `javadoc:javadoc`. Esta instrucción ordena al IDE que utilice el plugin de Javadoc para procesar los comentarios escritos anteriormente y construir el sitio HTML.

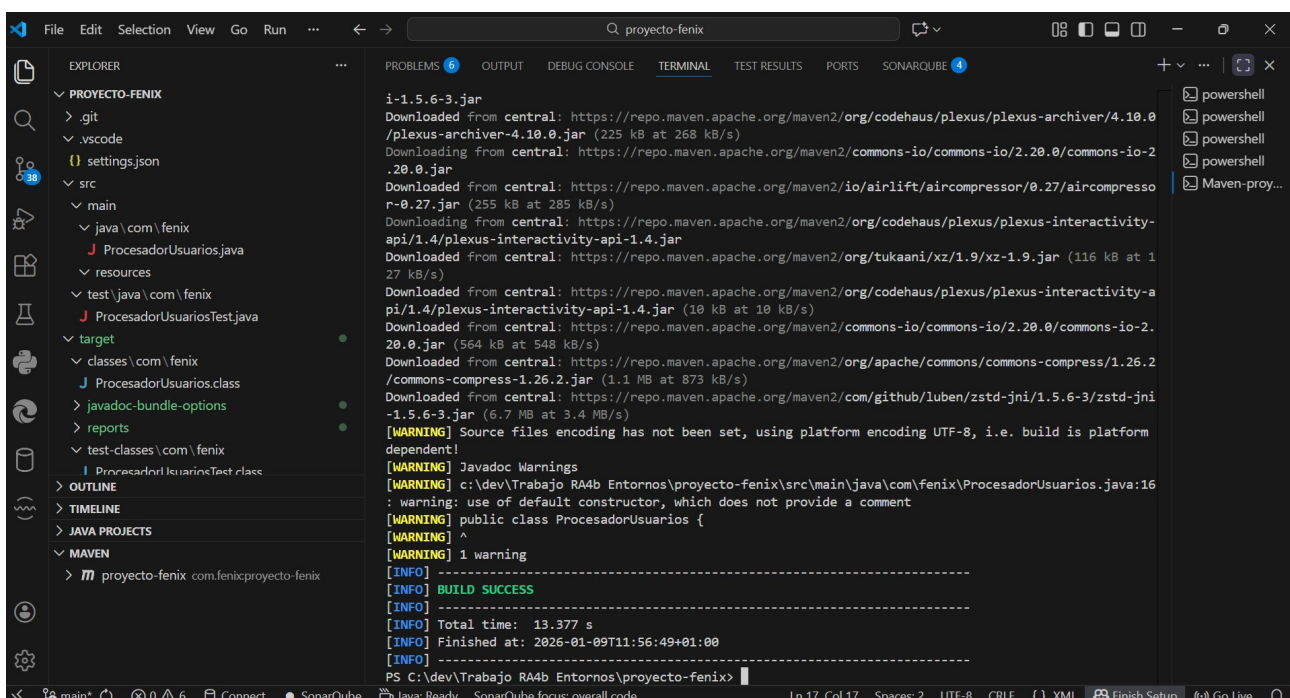


Al detectarse que el IDE no localiza el ejecutable de Maven en el sistema (*Maven executable not found*), se procede a configurar la ruta manualmente en los ajustes de la extensión (`maven.executable.path`), apuntando al archivo `mvn.cmd` descargado previamente.

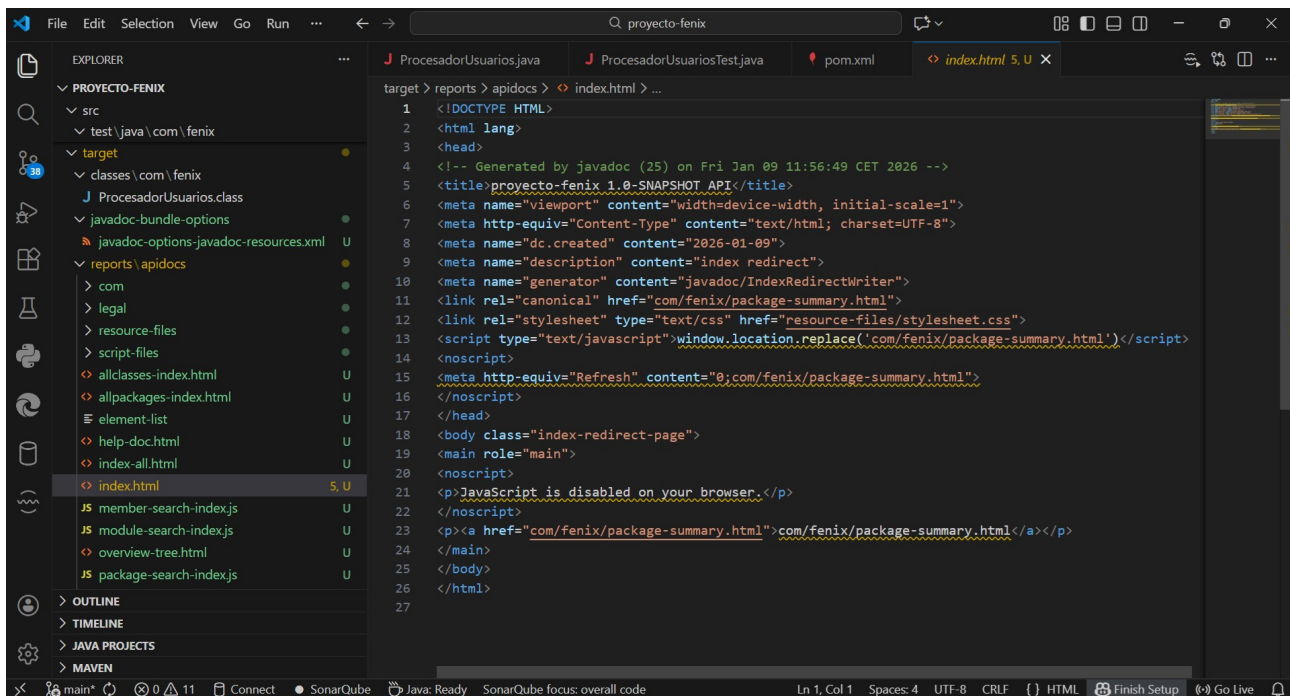


Con la configuración corregida, se lanza nuevamente el proceso. La terminal muestra la descarga de plugins y la generación de los reportes, finalizando con el mensaje *BUILD SUCCESS*.

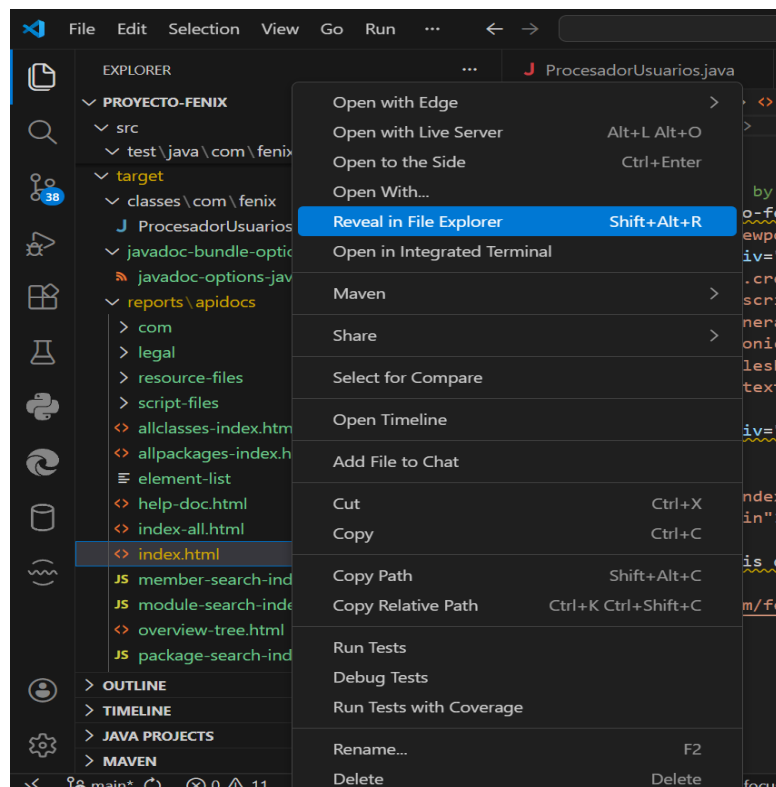
```
PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS SONARQUBE 4 +
PS C:\dev\Trabajo RA4b Entornos\proyecto-fenix> & "c:\apache-maven-3.9.12\bin\mvn.cmd" javadoc:javadoc -
f "c:\dev\Trabajo RA4b Entornos\proyecto-fenix\pom.xml"
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.fenix:proyecto-fenix >-----
[INFO] Building proyecto-fenix 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] >>> javadoc:3.12.0:javadoc (default-cli) > generate-sources @ proyecto-fenix >>>
[INFO]
[INFO] <<< javadoc:3.12.0:javadoc (default-cli) < generate-sources @ proyecto-fenix <<<
[INFO]
[INFO] --- javadoc:3.12.0:javadoc (default-cli) @ proyecto-fenix ---
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-
util/1.4.1/maven-resolver-util-1.4.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-u
til/1.4.1/maven-resolver-util-1.4.1.pom (2.8 kB at 8.7 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver/
1.4.1/maven-resolver-1.4.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver/1
.4.1/maven-resolver-1.4.1.pom (18 kB at 454 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/33/maven-pa
rent-33.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/33/maven-par
ent-33.pom (44 kB at 1.1 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/apache/21/apache-21.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/apache/21/apache-21.pom (17 kB
at 535 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-
api/1.4.1/maven-resolver-api-1.4.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-a
pi/1.4.1/maven-resolver-api-1.4.1.pom (2.6 kB at 73 kB/s)
```



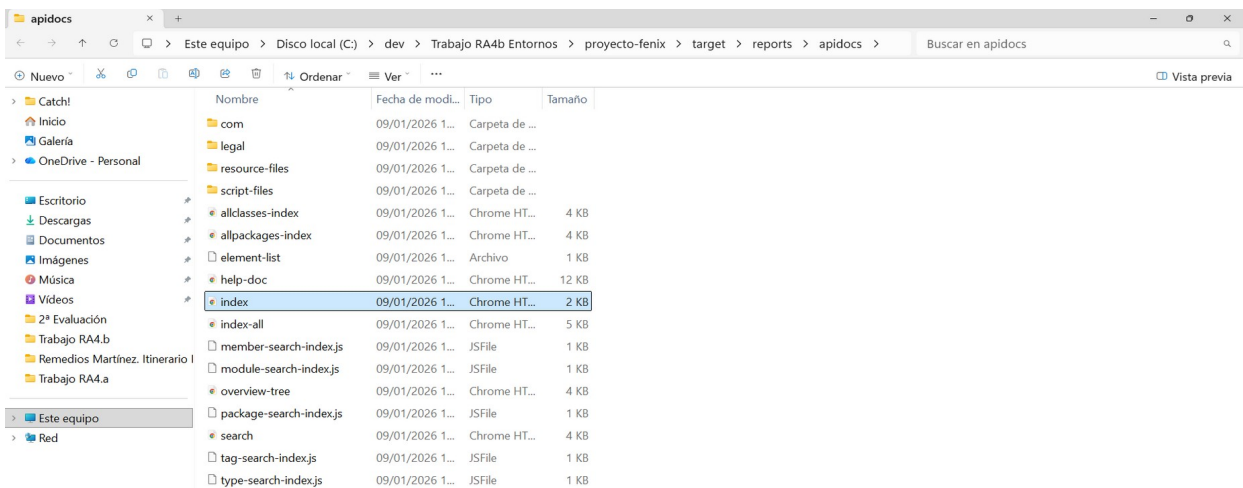
5. Verificación del Resultado HTML: Se navega a la carpeta generada *target/reports/apidocs*. Se localiza el archivo *index.html*, que sirve como entrada a la documentación web.



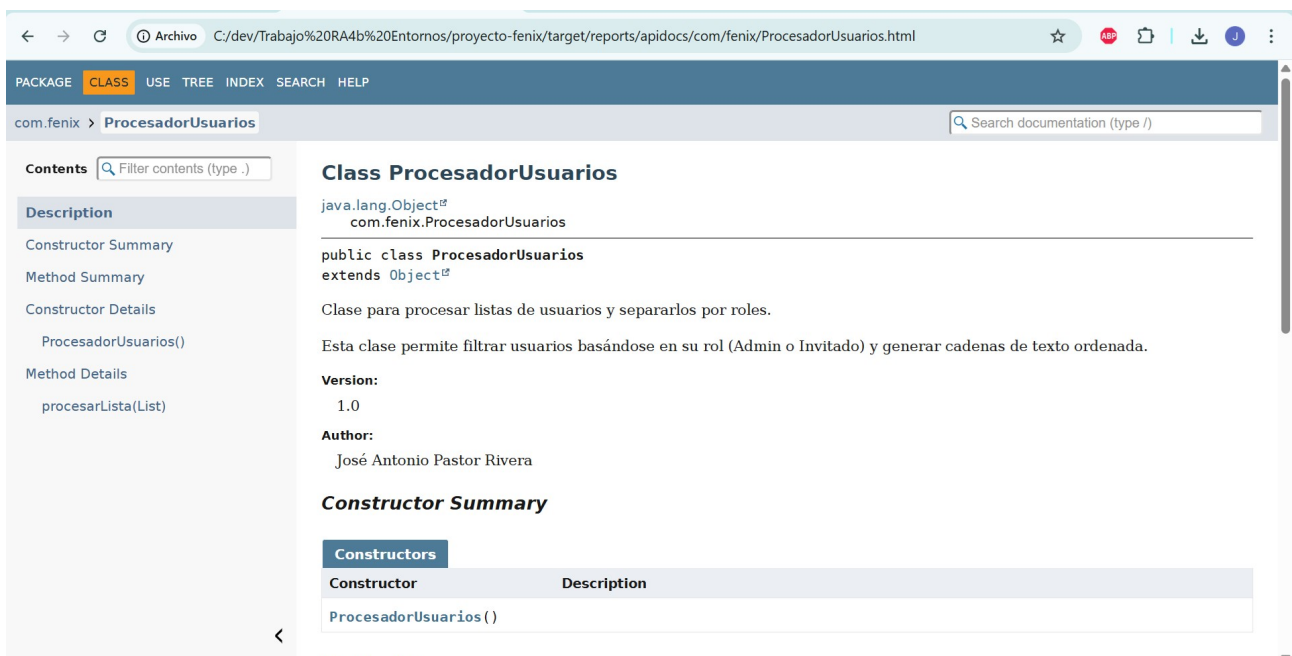
Localizamos la ubicación del archivo desde el IDE.



Confirmamos que está todo bien.



Al abrir el archivo, se visualiza la documentación generada. Se comprueba que el resumen del paquete, la información de la clase (*@author*, *@version*) y el detalle de los métodos (*@param*, *@return*) se han renderizado correctamente a partir del código fuente.



← → ↻ Archivo C:/dev/Trabajo%20RA4b%20Entornos/proyecto-fenix/target/reports/apidocs/com/fenix/ProcesadorUsuarios.html ☆ 🔍 📄 ⬇️ ⋮

PACKAGE CLASS USE TREE INDEX SEARCH HELP

com.fenix > **ProcesadorUsuarios** 🔍 Search documentation (type /)

Contents 🔍 Filter contents (type .)

- Description
- Constructor Summary
- Method Summary**
- Constructor Details
 - ProcesadorUsuarios()
- Method Details
 - procesarLista(List)

Method Summary

All Methods Instance Methods Concrete Methods

Modifier and Type	Method	Description
String	procesarLista(List<String> usuarios)	Procesa una lista de cadenas con formato "nombre:rol".

Methods inherited from class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

ProcesadorUsuarios

```
public ProcesadorUsuarios()
```

Method Details

procesarLista

```
public String procesarLista(List<String> usuarios)
```

← → ↻ Archivo C:/dev/Trabajo%20RA4b%20Entornos/proyecto-fenix/target/reports/apidocs/com/fenix/ProcesadorUsuarios.html ☆ 🔍 📄 ⬇️ ⋮

PACKAGE CLASS USE TREE INDEX SEARCH HELP

com.fenix > **ProcesadorUsuarios** 🔍 Search documentation (type /)

Contents 🔍 Filter contents (type .)

- Description
- Constructor Summary
- Method Summary
- Constructor Details**
- Method Details
 - procesarLista(List)

Constructor Details

ProcesadorUsuarios

```
public ProcesadorUsuarios()
```

Method Details

procesarLista

```
public String procesarLista(List<String> usuarios)
```

Procesa una lista de cadenas con formato "nombre:rol".

Parameters:

usuarios - Lista de strings con el formato "Nombre:ID_Rol" (ej: "Ana:1").

Returns:

String con los usuarios separados por grupos (Admins e Invitados).

Copyright © 2026. All rights reserved.

6. Refinamiento Técnico (Codificación UTF-8): Durante la generación, se detectó una advertencia sobre la codificación de caracteres. Para asegurar la máxima calidad y evitar problemas con caracteres especiales, se define explícitamente la codificación **UTF-8** en el archivo *pom.xml* añadiendo las propiedades `<project.build.sourceEncoding>` y `<project.reporting.outputEncoding>`.

```

15
16 public class ProcesadorUsuarios {
17
18     /**
19      * Constructor por defecto de la clase ProcesadorUsuarios.
20      */
21
22     private static final int ROL_INVITADO = 2;
23     private static final int ROL_ADMIN = 1;
24

```

```

PS C:\dev\Trabajo RA4b Entornos\proyecto-fenix> & "c:\apache-maven-3.9.12\bin\mvn.cmd" javadoc:javadoc -
f "c:\dev\Trabajo RA4b Entornos\proyecto-fenix\pom.xml"
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.fenix:proyecto-fenix >-----
[INFO] Building proyecto-fenix 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] >>> javadoc:3.12.0:javadoc (default-cli) > generate-sources @ proyecto-fenix >>>
[INFO]
[INFO] <<< javadoc:3.12.0:javadoc (default-cli) < generate-sources @ proyecto-fenix <<<
[INFO]
[INFO] --- javadoc:3.12.0:javadoc (default-cli) @ proyecto-fenix ---
[WARNING] Source files encoding has not been set, using platform encoding UTF-8, i.e. build is platform
dependent!
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time:  1.605 s
[INFO] Finished at: 2026-01-09T12:13:58+01:00
[INFO] -----
PS C:\dev\Trabajo RA4b Entornos\proyecto-fenix>

```

```

<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>

  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
</properties>

```

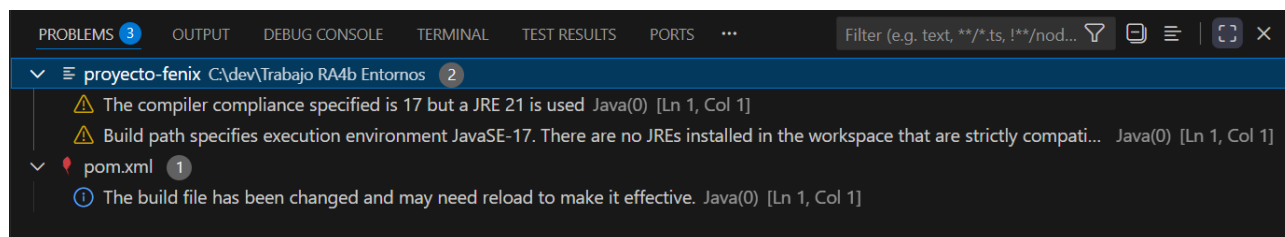
Se regenera la documentación para confirmar que el proceso es limpio y libre de advertencias.

```

PS C:\dev\Trabajo RA4b Entornos\proyecto-fenix> & "c:\apache-maven-3.9.12\bin\mvn.cmd" javadoc:javadoc -
f "c:\dev\Trabajo RA4b Entornos\proyecto-fenix\pom.xml"
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.fenix:proyecto-fenix >-----
[INFO] Building proyecto-fenix 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] >>> javadoc:3.12.0:javadoc (default-cli) > generate-sources @ proyecto-fenix >>>
[INFO]
[INFO] <<< javadoc:3.12.0:javadoc (default-cli) < generate-sources @ proyecto-fenix <<<
[INFO]
[INFO] --- javadoc:3.12.0:javadoc (default-cli) @ proyecto-fenix ---
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time:  1.461 s
[INFO] Finished at: 2026-01-09T12:17:31+01:00
[INFO] -----

```

Durante el desarrollo, la pestaña de problemas reportó un error de conformidad porque el proyecto estaba configurado para Java 17, pero el entorno de ejecución utilizaba la versión 21.



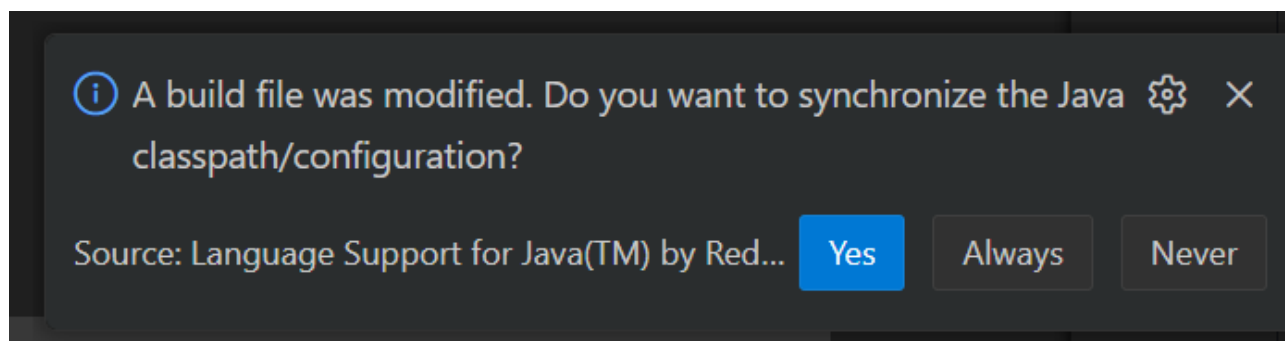
Al revisar el archivo de configuración pom.xml, confirmamos que las propiedades de compilación apuntaban incorrectamente a la versión antigua (17).

```
<properties>  
  <maven.compiler.source>17</maven.compiler.source>  
  <maven.compiler.target>17</maven.compiler.target>
```

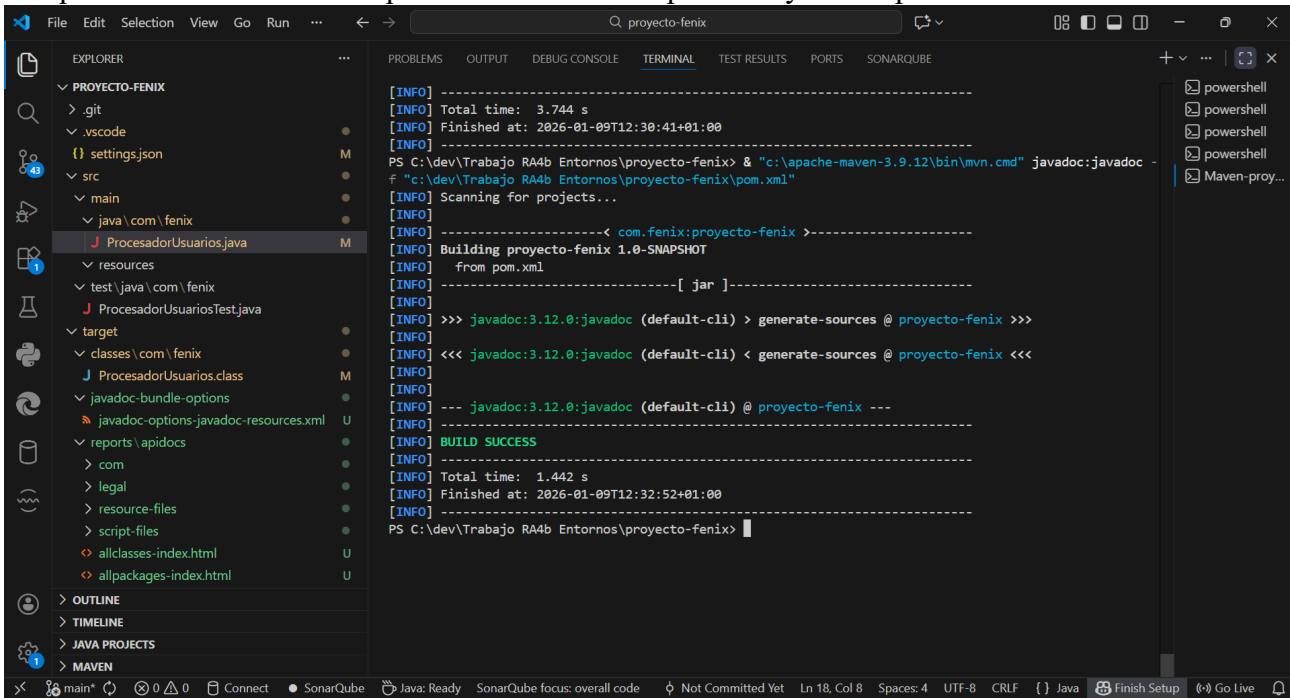
Para corregirlo, modificamos las etiquetas <maven.compiler.source> y <target> en el archivo, actualizando el valor a 21 para que coincidiera con nuestro JDK.

```
<properties>  
  <maven.compiler.source>21</maven.compiler.source>  
  <maven.compiler.target>21</maven.compiler.target>
```

Inmediatamente tras guardar el cambio, Visual Studio Code detectó la modificación en el archivo de construcción y solicitó actualizar la configuración del proyecto.



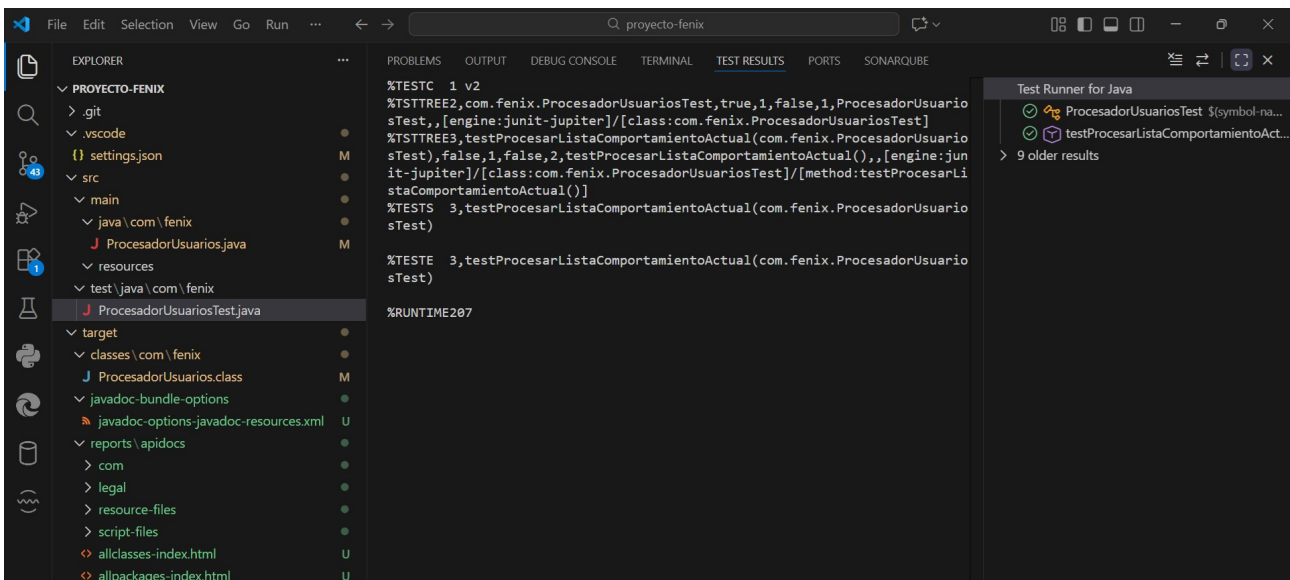
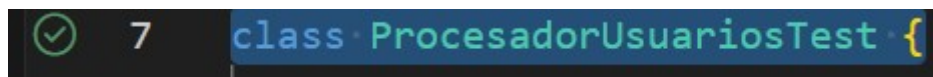
Finalmente, tras aceptar la sincronización, ejecutamos de nuevo la construcción del proyecto y comprobamos en la terminal que el error había desaparecido y la compilación fue exitosa.



The screenshot shows the VS Code interface with the 'proyecto-fenix' project open. The Explorer panel on the left shows the project structure, including 'ProcesadorUsuarios.java' and 'ProcesadorUsuariosTest.java'. The Terminal panel on the right displays the output of a Maven build, which is successful. The output includes information about the build time, the version of the project (1.0-SNAPSHOT), and the successful execution of the 'generate-sources' goal. The status bar at the bottom indicates that the project is built and ready for execution.

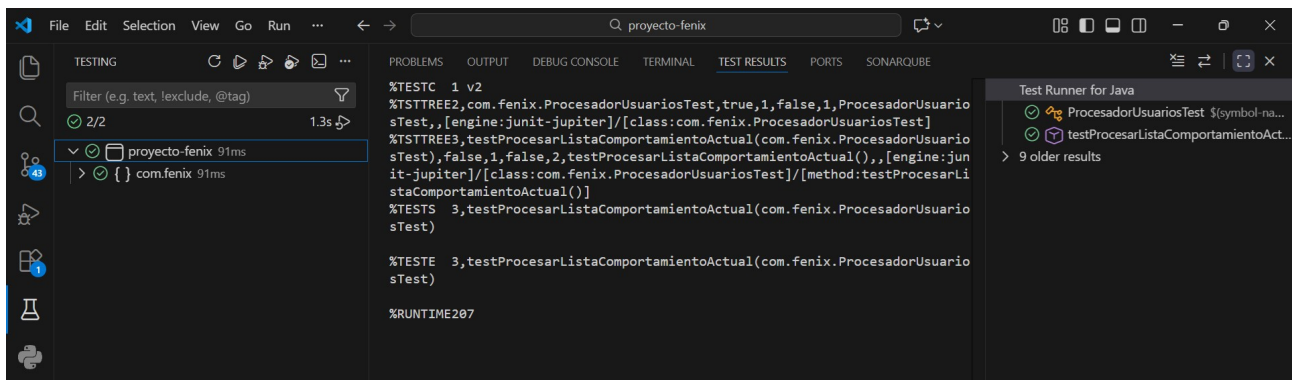
```
[INFO] -----
[INFO] Total time: 3.744 s
[INFO] Finished at: 2026-01-09T12:30:41+01:00
[INFO] -----
PS C:\dev\Trabajo RA4b Entornos\proyecto-fenix> & "c:\apache-maven-3.9.12\bin\mvn.cmd" javadoc:javadoc -
f "c:\dev\Trabajo RA4b Entornos\proyecto-fenix\pom.xml"
[INFO] Scanning for projects...
[INFO] -----< com.fenix:proyecto-fenix >-----
[INFO] Building proyecto-fenix 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] >>> javadoc:3.12.0:javadoc (default-cli) > generate-sources @ proyecto-fenix >>>
[INFO]
[INFO] <<< javadoc:3.12.0:javadoc (default-cli) < generate-sources @ proyecto-fenix <<<
[INFO]
[INFO] --- javadoc:3.12.0:javadoc (default-cli) @ proyecto-fenix ---
[INFO]
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.442 s
[INFO] Finished at: 2026-01-09T12:32:52+01:00
[INFO] -----
PS C:\dev\Trabajo RA4b Entornos\proyecto-fenix>
```

7. Validación Final y Cierre: Antes de la entrega, se ejecutan las pruebas unitarias (*ProcesadorUsuariosTest*) para garantizar que la inclusión de comentarios y cambios en el *pom.xml* no han afectado a la lógica del programa.

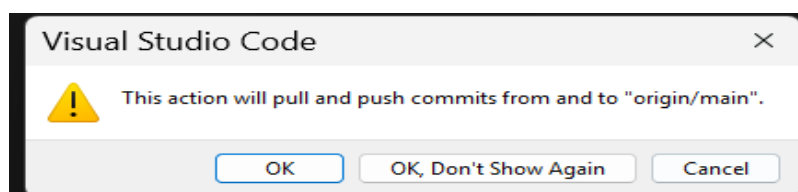
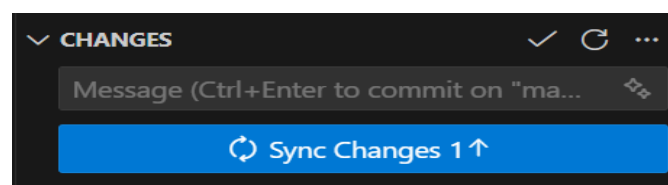
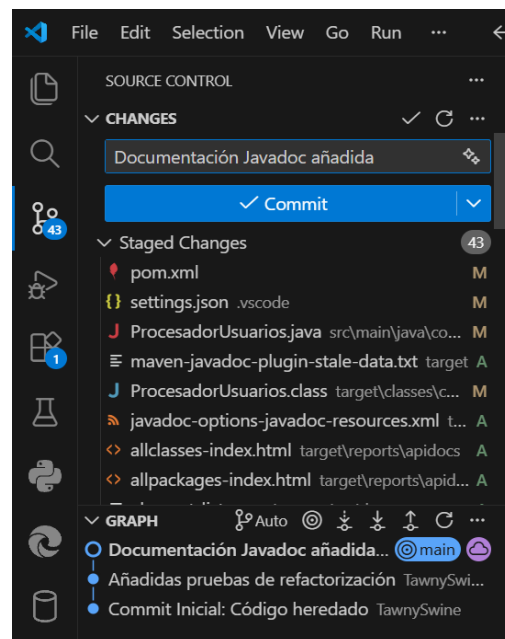
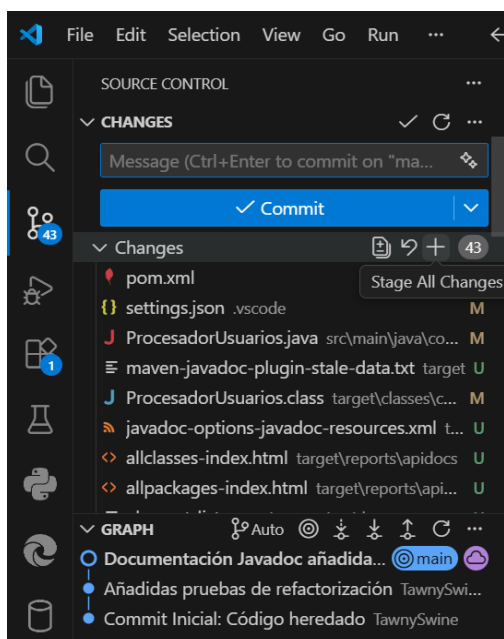


The screenshot shows the VS Code interface with the 'proyecto-fenix' project open. The Explorer panel on the left shows the project structure, including 'ProcesadorUsuariosTest.java'. The Test Results panel on the right displays the results of the unit tests, showing that all tests passed. The Test Runner for Java is visible, showing the test results for 'ProcesadorUsuariosTest' and 'testProcesarListaComportamientoActual'.

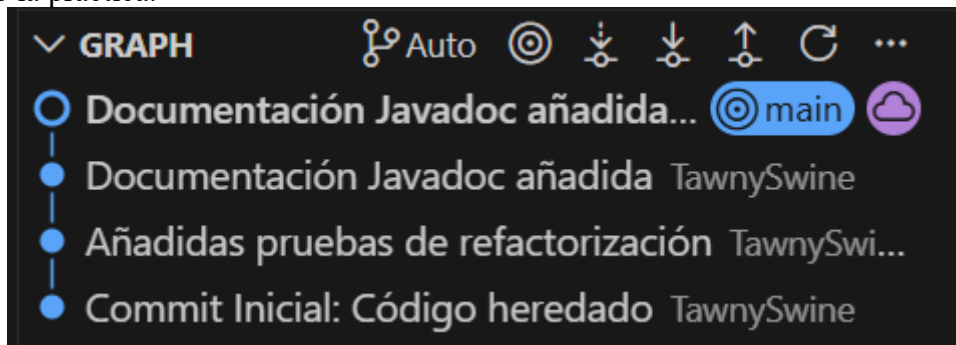
```
%TESTC 1 v2
%TSTTREE2,com.fenix.ProcesadorUsuariosTest,true,1,false,1,ProcesadorUsuario
sTest,,[engine:junit-jupiter]/[class:com.fenix.ProcesadorUsuariosTest]
%TSTTREE3,testProcesarListaComportamientoActual(com.fenix.ProcesadorUsuario
sTest),false,1,false,2,testProcesarListaComportamientoActual(),[engine:jun
it-jupiter]/[class:com.fenix.ProcesadorUsuariosTest]/[method:testProcesarLi
staComportamientoActual()]
%TESTS 3,testProcesarListaComportamientoActual(com.fenix.ProcesadorUsuario
sTest)
%TESTE 3,testProcesarListaComportamientoActual(com.fenix.ProcesadorUsuario
sTest)
%RUNTIME207
```



Finalmente, se realiza un último *commit* y *push* que incluye los ajustes de configuración y la verificación de que el proyecto está completo, sincronizado y documentado en el repositorio remoto.



Se verifica en el historial (*Graph*) que la rama local *main* y la remota *origin/main* están alineadas, concluyendo la práctica.



Parte 5: Integración Continua

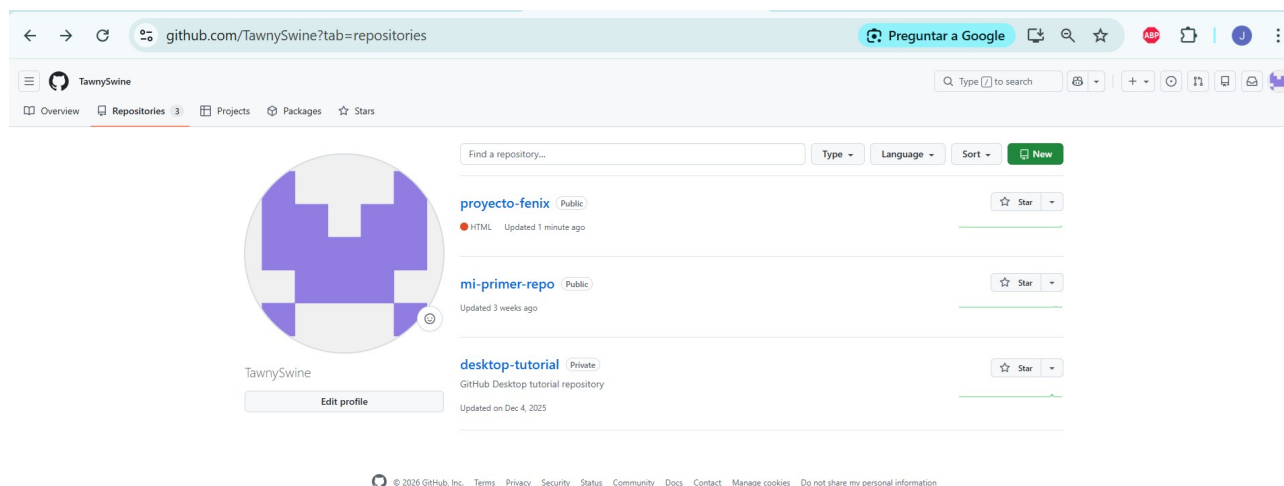
Tarea 6: Usar herramientas de integración continua

Objetivo: Automatizar el ciclo de vida del desarrollo de software (específicamente las fases de compilación y pruebas) mediante la implementación de un servidor de integración continua. La finalidad es asegurar que cada cambio introducido en el código fuente sea verificado automáticamente antes de integrarse en la rama principal, garantizando así la estabilidad del proyecto.

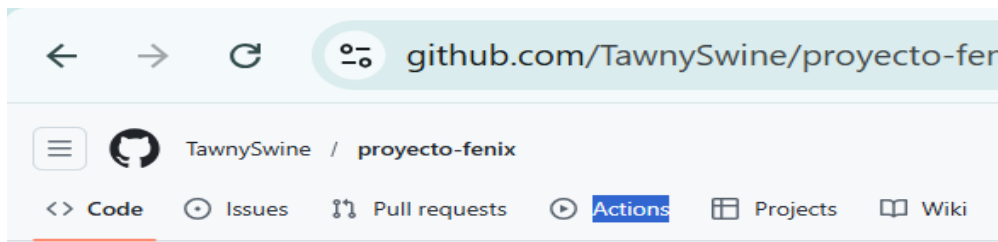
Para este propósito, se ha seleccionado GitHub Actions, la herramienta de CI/CD (Integración y Despliegue Continuos) nativa de la plataforma donde se aloja el repositorio remoto. Esta solución permite definir flujos de trabajo (workflows) directamente en el repositorio sin necesidad de configurar servidores externos. A continuación, se describe el proceso de configuración y verificación realizado.

1. Selección de la herramienta CI.

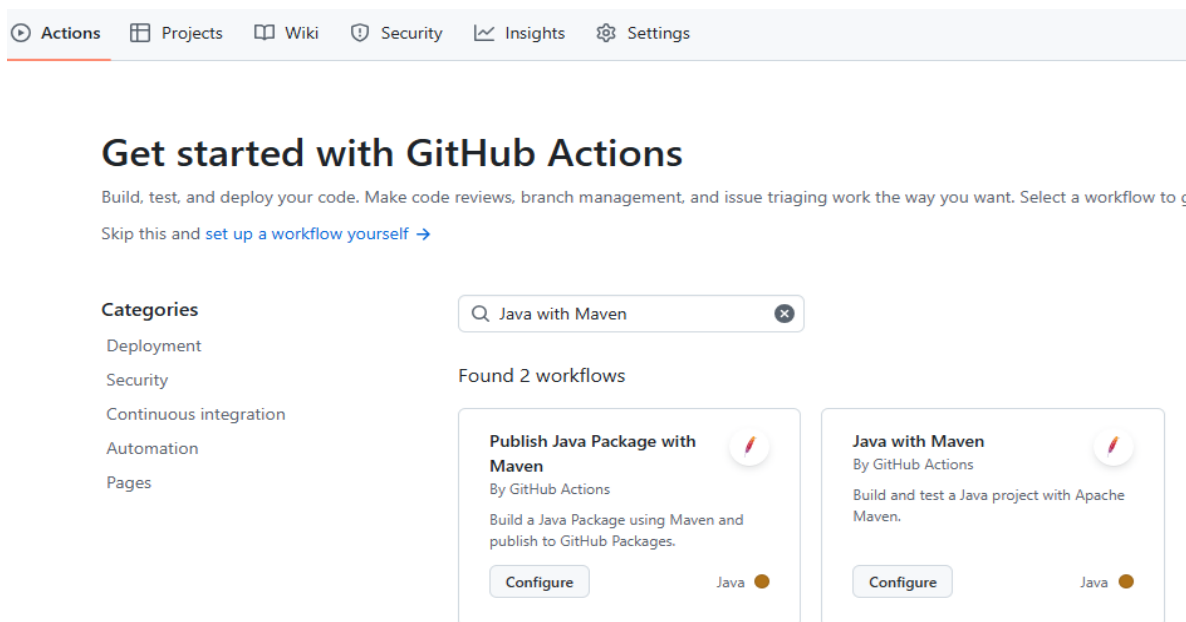
Desde el perfil de GitHub, se accedió al repositorio del proyecto.



En la pestaña Actions, GitHub sugirió varios flujos de trabajo (workflows) basándose en el lenguaje del proyecto.

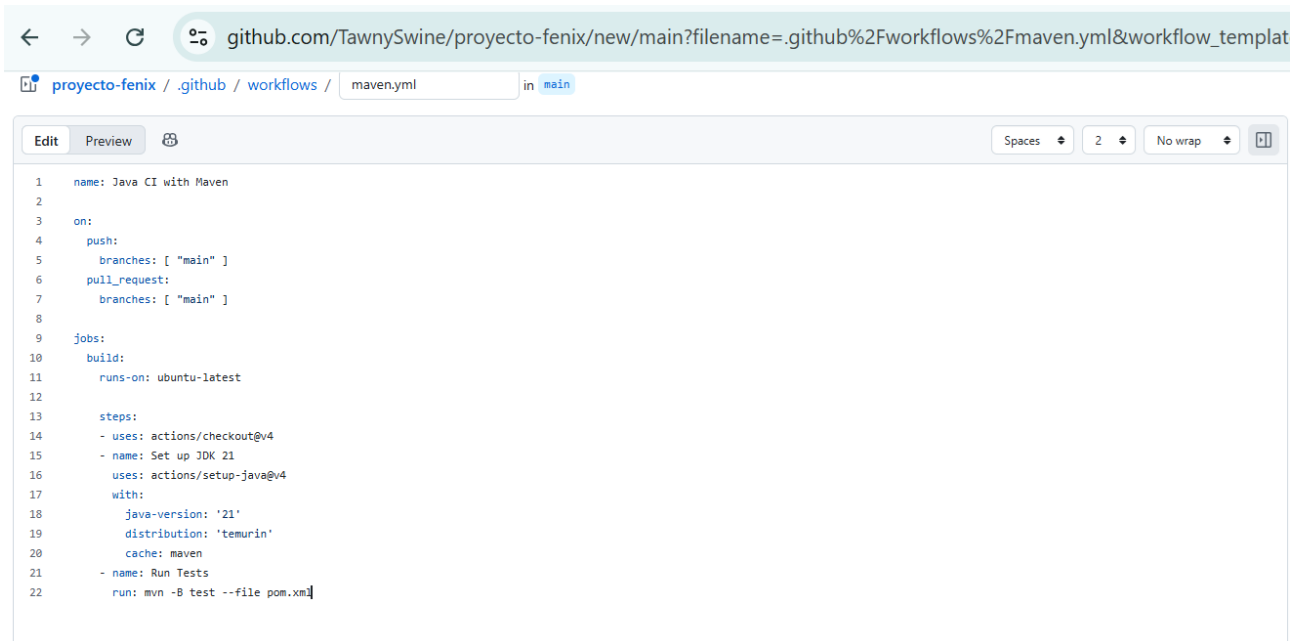


Se seleccionó la plantilla "Java with Maven", ya que es el gestor de dependencias utilizado.



2. Configuración del Workflow.

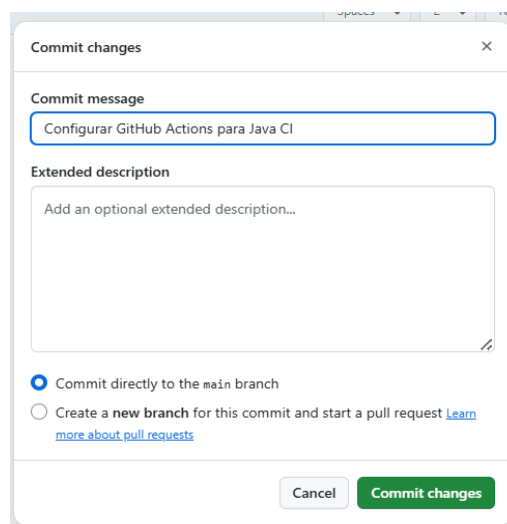
Para comenzar, seleccionamos la plantilla de Maven. GitHub genera automáticamente un archivo YAML con la configuración necesaria para ejecutar las pruebas. Revisamos que el script incluye el comando *mvn test*.



The screenshot shows a web browser displaying a GitHub repository page for 'TawnySwine/proyecto-fenix'. The URL is 'github.com/TawnySwine/proyecto-fenix/new/main?filename=.github%2Fworkflows%2Fmaven.yml&workflow_templat'. The breadcrumb navigation shows 'proyecto-fenix / .github / workflows / maven.yml' in the 'main' branch. The file content is a YAML workflow configuration for 'Java CI with Maven'. It triggers on 'push' to the 'main' branch or 'pull_request' to 'main'. The workflow runs on 'ubuntu-latest' and includes two steps: 'Set up JDK 21' using 'actions/setup-java@v4' with 'java-version: 21', 'distribution: temurin', and 'cache: maven'; and 'Run Tests' using 'run: mvn -B test --file pom.xml'.

```
1 name: Java CI with Maven
2
3 on:
4   push:
5     branches: [ "main" ]
6   pull_request:
7     branches: [ "main" ]
8
9 jobs:
10  build:
11    runs-on: ubuntu-latest
12
13    steps:
14      - uses: actions/checkout@v4
15        name: Set up JDK 21
16        uses: actions/setup-java@v4
17        with:
18          java-version: '21'
19          distribution: 'temurin'
20          cache: maven
21      - name: Run Tests
22        run: mvn -B test --file pom.xml
```

Una vez verificado el contenido del archivo, procedemos a guardarlo en el repositorio. Escribimos un mensaje descriptivo para el commit, como "Configurar GitHub Actions para Java CI", indicando que haremos el commit directamente en la rama *main*.



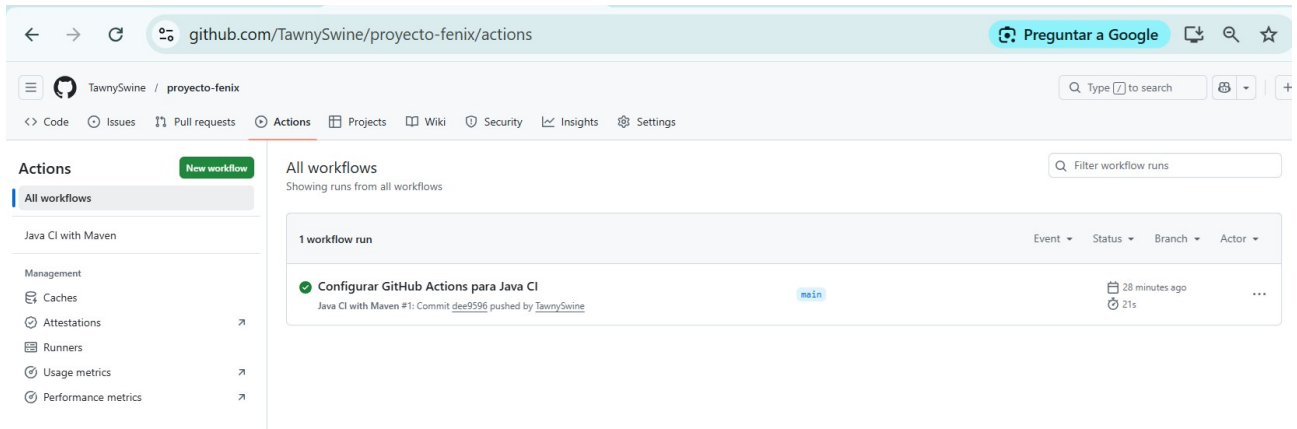
The screenshot shows the 'Commit changes' dialog box. The 'Commit message' field contains the text 'Configurar GitHub Actions para Java CI'. The 'Extended description' field is empty with the placeholder text 'Add an optional extended description...'. Below the fields, there are two radio button options: 'Commit directly to the main branch' (which is selected) and 'Create a new branch for this commit and start a pull request'. At the bottom right, there are two buttons: 'Cancel' and 'Commit changes'.

Para confirmar la operación y guardar el archivo definitivamente en el servidor, pulsamos el botón verde de confirmación.

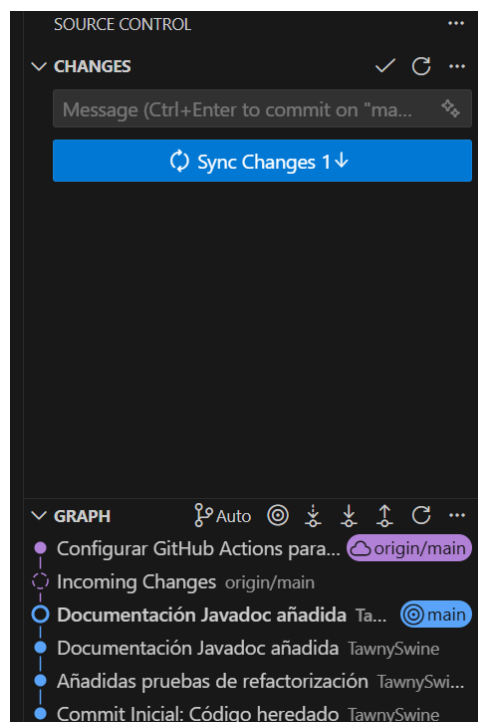
Commit changes...

3. Verificación Inicial y Sincronización.

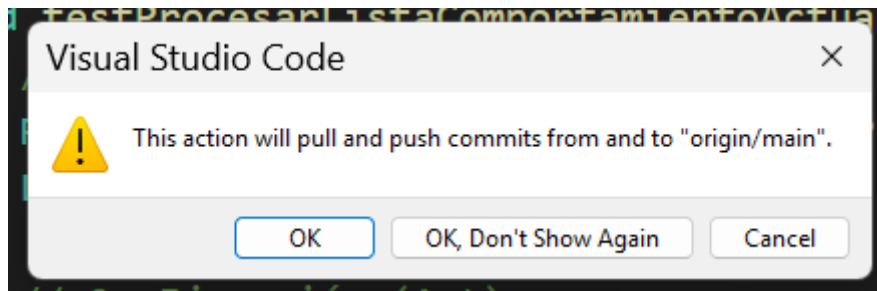
Inmediatamente después de crear el archivo, vamos a la pestaña Actions para comprobar que el sistema ha detectado el nuevo flujo de trabajo. Como se observa a continuación, la primera ejecución (creación del archivo) ha sido exitosa (check verde).



Dado que el archivo YAML se creó en el servidor remoto (GitHub), nuestro entorno local (VS Code) está desactualizado. Al abrir el control de versiones, vemos en el gráfico que hay cambios entrantes (Incoming Changes) pendientes de descargar.



Procedemos a sincronizar el proyecto. VS Code nos advierte que esta acción realizará un pull (descarga) y un push (subida) para alinear ambas versiones. Aceptamos para tener el archivo de workflow en nuestro ordenador.

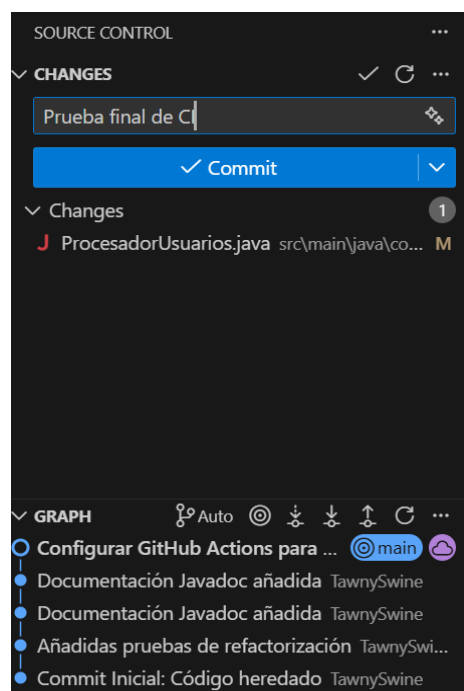


4. Prueba de Disparo Automático (Modificación del Código)

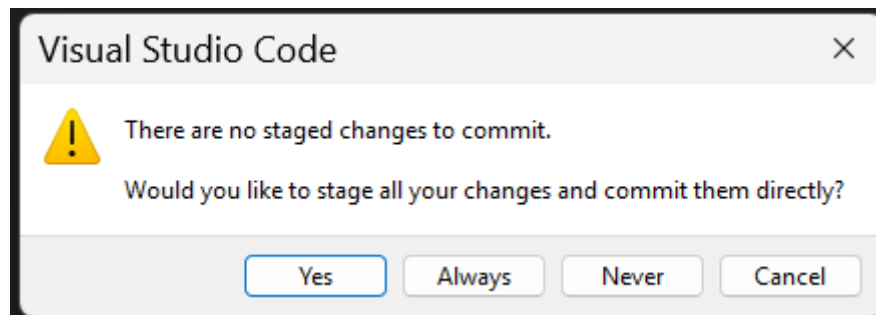
Con el entorno sincronizado, realizamos la prueba funcional. Editamos una clase del proyecto (por ejemplo, *ProcesadorUsuarios.java*) añadiendo un comentario trivial para simular un cambio en el código fuente.

```
79  }  
80  //Cambio para probar la integración continua (CI)  
81  }
```

Guardamos el archivo y vamos a la pestaña de control de código fuente. Escribimos el mensaje del commit: "Prueba final de CI", preparándonos para registrar el cambio.

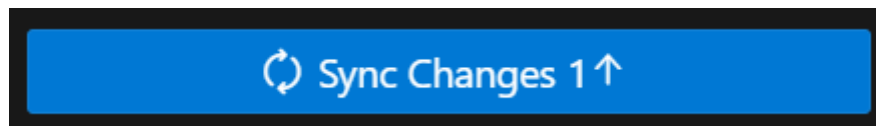


Al intentar hacer commit sin haber añadido los cambios al área de preparación (*stage*), el entorno nos pregunta si queremos hacerlo todo en un solo paso. Confirmamos pulsando "Yes".

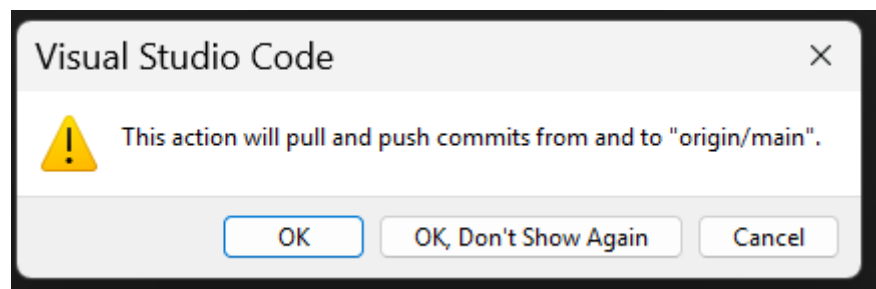


5. Envío de Cambios (Push)

Una vez realizado el commit local, aparece el botón azul "Sync Changes" con un "1" indicando que tenemos una confirmación lista para enviar al servidor.

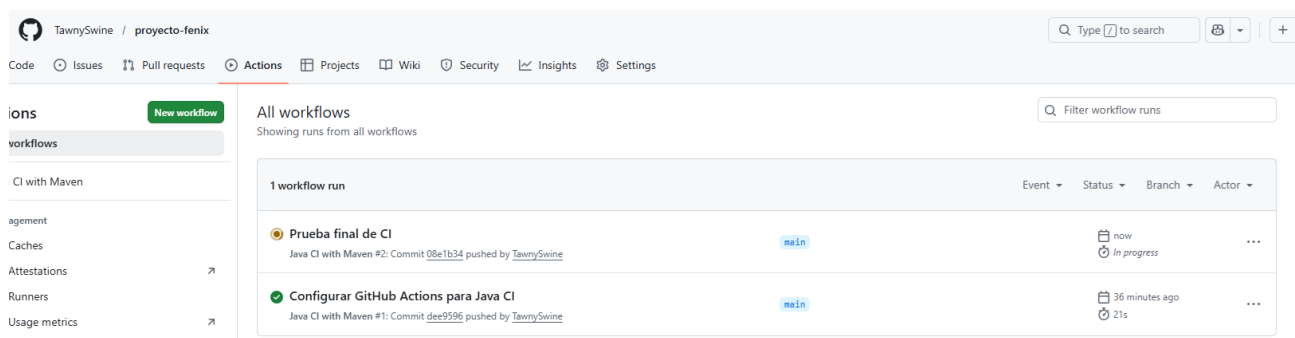


Al pulsar el botón, confirmamos la acción de envío (Push). Este es el paso crucial, ya que al llegar el código nuevo a GitHub, el servidor debería "despertar" y ejecutar las pruebas automáticamente.



6. Verificación Final (Resultado del Test)

Volvemos rápidamente a la pestaña Actions de GitHub. Observamos que ha aparecido una nueva ejecución con nuestro mensaje "Prueba final de CI" marcada en color amarillo, lo que indica que el servidor está ejecutando las pruebas en ese preciso instante.



Tras unos segundos de espera, el proceso finaliza. El icono cambia a verde, confirmando que GitHub Actions ha descargado el proyecto, ha ejecutado Maven y todas las pruebas han pasado satisfactoriamente sin intervención manual.

TawnySwine / proyecto-fenix

CodeIssuesPull requestsActionsProjectsWikiSecurityInsightsSettings

New workflow

workflows

CI with Maven

gement

aches

testations

unners

usage metrics

erformance metrics

All workflows

Showing runs from all workflows

1 workflow run

Event

Status

Branch

Actor

Prueba final de CI

Java CI with Maven #2: Commit 08e1b34 pushed by TawnySwine

main

now

14s

...

Configurar GitHub Actions para Java CI

Java CI with Maven #1: Commit dee9596 pushed by TawnySwine

main

37 minutes ago

21s

...